

Week 1 · Session 1

Foundations

Data pre-processing for reliable machine learning

Where Are We in the Bootcamp?

Build the end-to-end ML workflow: clean data → train → evaluate.

Week 1

S1 Foundations & Preprocessing

S2 Regression Models

S3 Classification Basics

S4 Naive Bayes & Trees

S5 SVM & Kernels

S6 Clustering & PCA

Week 2

S7 Deep Learning

Week 3

S8 NLP Fundamentals

S9 Tokenization

S10 Text Analysis & NER

S11 Language Modeling

S12 Embeddings & RNNs

S13 Seq2Seq & NMT

Week 4

S14 Generative AI

S15 RAG Systems

S16 MLOps

Week 1 · Session 1

Foundations

Data Pre-Processing

Data Pre-Processing

Train/test splits

Feature scaling

Encoding & imputation

Data Pre-Processing

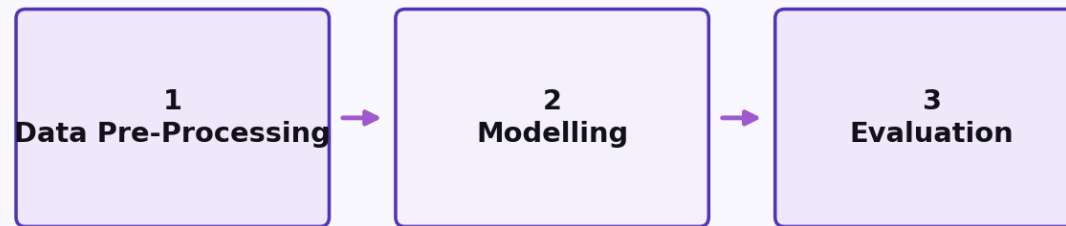
What we will cover in this session

- Data Pre-Processing
- Train/test splits
- Feature scaling
- Encoding & imputation
- Data leakage

The Machine Learning Process

The 3 main steps

The Machine Learning Process



Step 1 · Data Pre-Processing

Load the data · Clean the data

Step 2 · Modelling

Build the model · Train the model

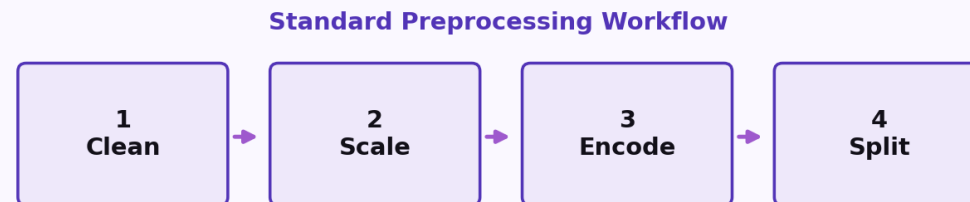
Step 3 · Evaluation

Calculate performance metrics · Make a final prediction

Step 1: Data Pre-Processing

The Machine Learning Process

- Load the data
- Clean the data
- Scale numeric features
- Encode categorical features
- Split into training and test :



Step 2: Modelling

The Machine Learning Process

- Build the model
- Train the model
- Make predictions

Step 3: Evaluation

The Machine Learning Process

- Calculate performance metrics
- Make a final prediction

Data Preprocessing Template

Standard Workflow

Clean → Scale → Encode → Split

Data preprocessing before modeling — a repeatable checklist before you train.

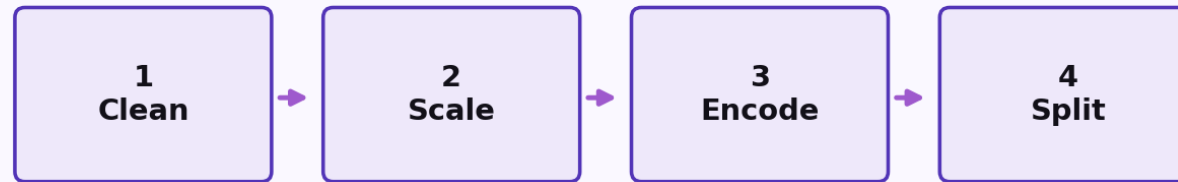
Start by loading the dataset, then follow the workflow in order.

Standard Workflow

Data Preprocessing Template

Data preprocessing before modeling

Standard Preprocessing Workflow



1. Clean

Handle missing values, errors, and noise.

2. Scale

Normalize or standardize numeric features.

3. Encode

Turn categories into numeric representations.

4. Split

Separate training and test data.

Workflow Checklist

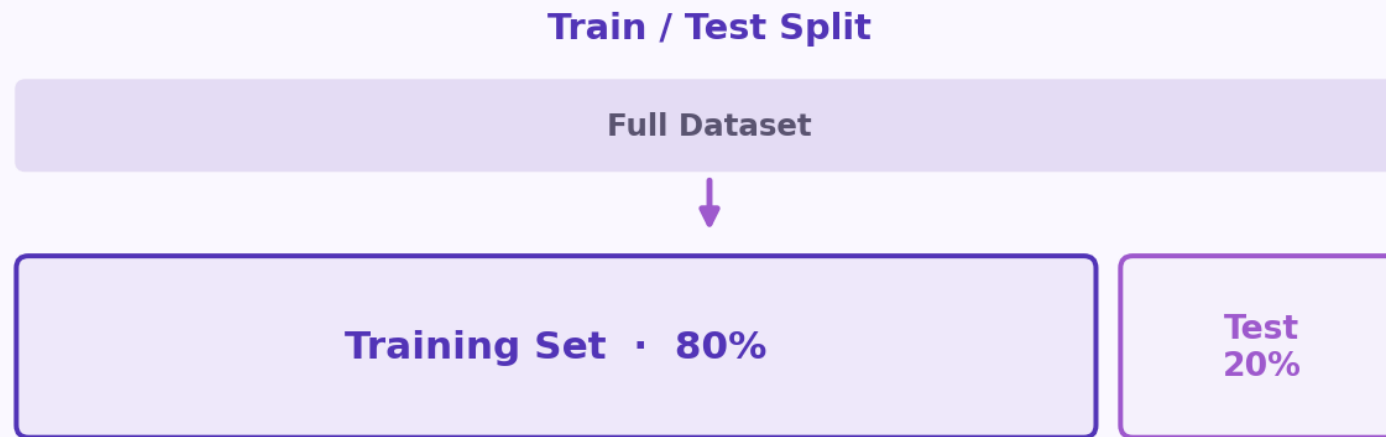
Data Preprocessing Template

- Load the dataset.
- Clean — fix missing values, duplicates, and inconsistent records.
- Scale — put numeric features on a comparable range.
- Encode — convert categorical variables into model-ready form.
- Split — create train/test sets before fitting the model.

Training Set and Test Set

Key Idea

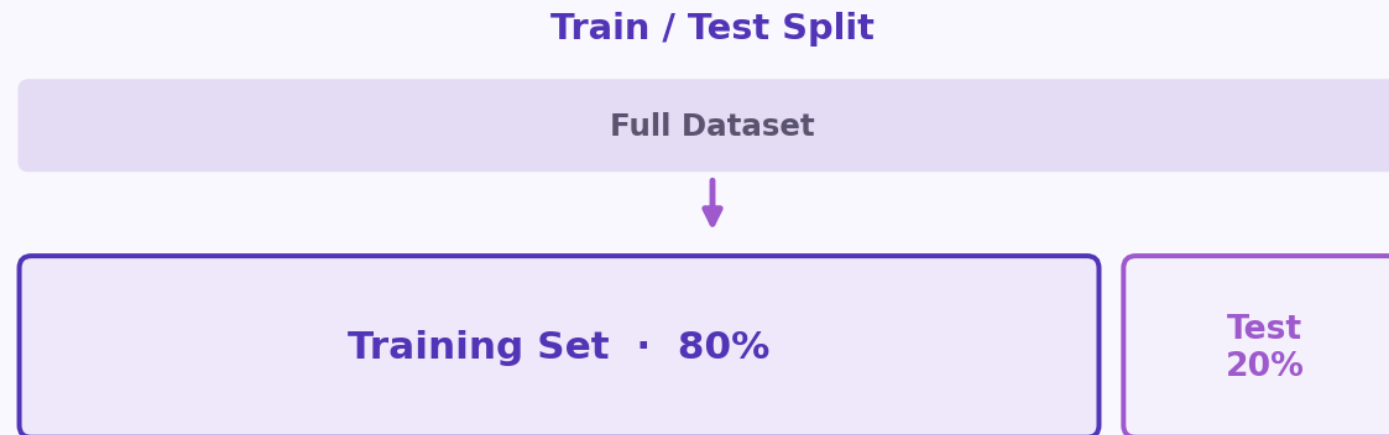
Hold-out partition — train wide, test narrow



Typical split: 80% train · 20% test

Training Set and Test Set

Definitions

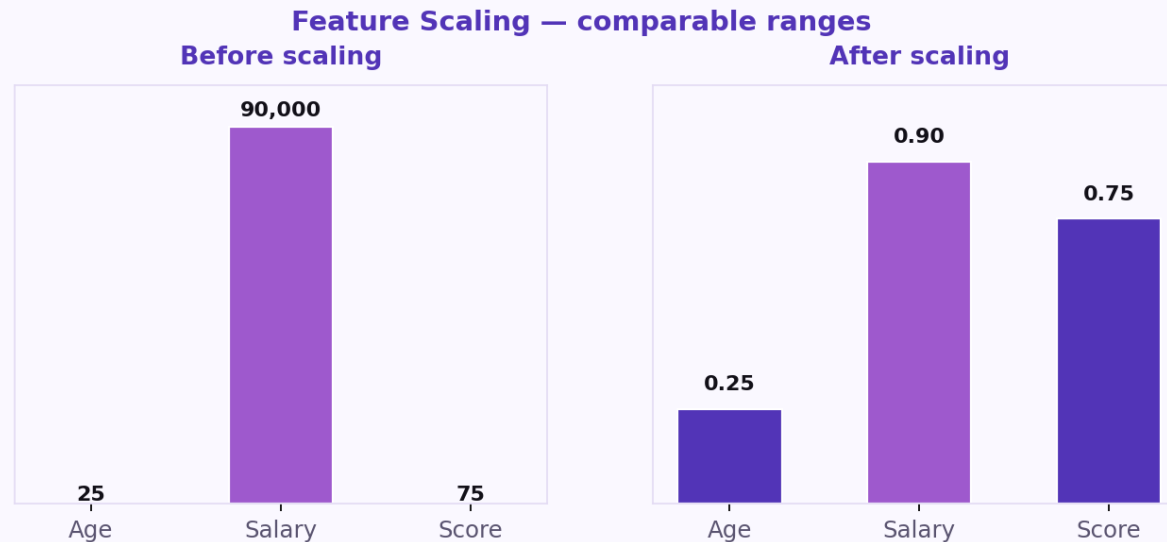


- Never tune hyperparameters on the test set — it becomes part of training indirectly.
- Stratified splits preserve class balance in classification tasks.
- We train on one part of the data, then test on another part to make sure the model generalizes well.

Feature Scaling

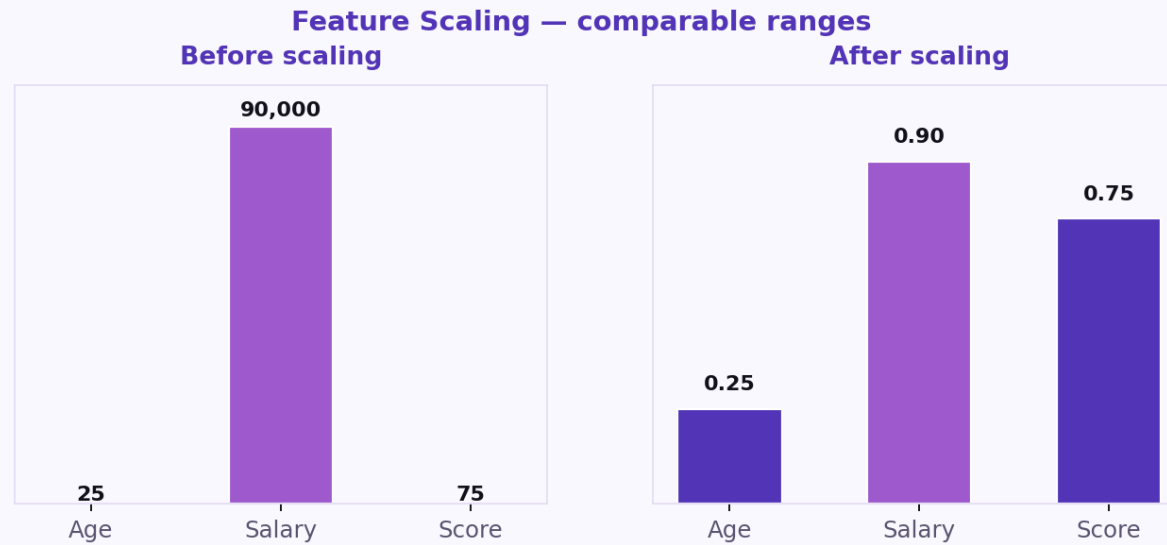
What is Feature Scaling?

Feature scaling means adjusting feature values to a similar range using methods like standardization and normalization.



Feature Scaling

Why it matters



- It prevents large-value features from dominating smaller-value features.
- It improves performance for distance-based and gradient-based algorithms (e.g., KNN, SVM, Gradient Descent).
- It helps training converge faster.

Example (Before vs After Scaling)

Worked example

Student	Height (cm)	Weight (kg)	Height (scaled)	Weight (scaled)
A	150	50	0.25	0.22
B	170	70	0.58	0.67
C	185	85	0.83	1.00

Before scaling, height and weight have different numeric ranges. After scaling, both features are on a comparable scale and contribute more fairly.

Feature Scaling Methods

Feature Scaling

Comparable range

Features brought to a comparable range.

After scaling

Heights are comparable.

Normalization

Scales to $[0, 1]$ using min–max — sensitive to outliers.

Standardization

Uses mean 0 and std 1 — preferred when features are roughly Gaussian.

Robust scaling

Uses median and IQR when outliers dominate.

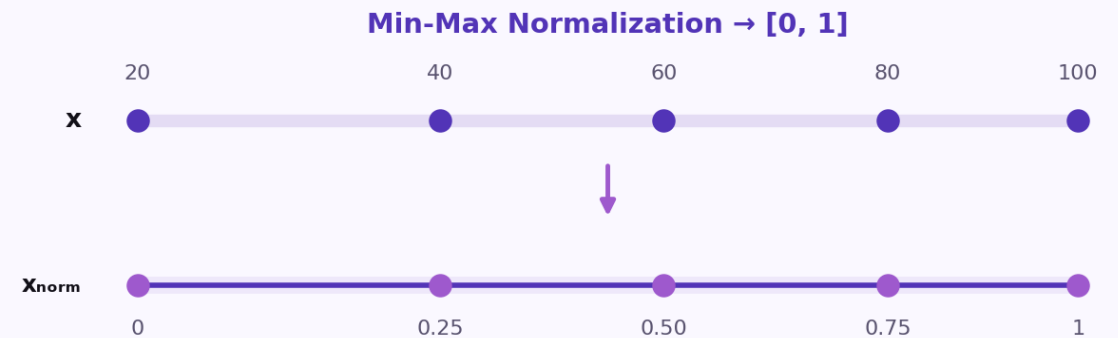
1) Normalization (Min-Max Scaling)

Feature Scaling Methods

Normalization transforms each value to a range between 0 and 1.

Formula

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$



- Useful for models that depend on distances or absolute magnitudes.
- Common choice for algorithms such as KNN and K-Means.

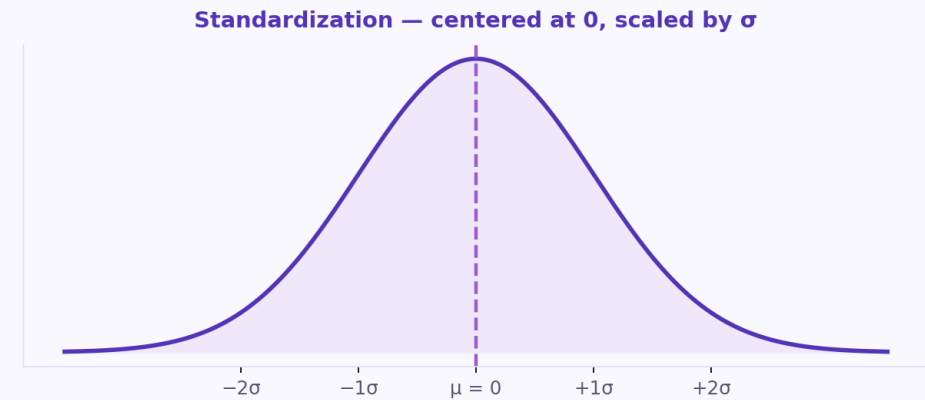
2) Standardization (Z-score Scaling)

Feature Scaling Methods

Standardization centers data around 0 with a standard deviation of 1.

Formula

$$z = \frac{x - \mu}{\sigma}$$



- Preferred when features are close to a Gaussian distribution.
- Works well with Logistic Regression, SVM, and Linear Regression.

Choosing the Right Scaling Method

Practical guidance

- Use standardization for SVM, logistic regression, PCA, and gradient-based training.
- Tree-based models are scale-invariant — scaling is optional.
- When in doubt, fit the scaler on train only, then transform train and test.

Normalization

Use Normalization when feature values have very different ranges and you want all values mapped to $[0, 1]$.

Typical method: Min-Max Scaling.

Normalization vs Standardization

When to use each

Normalization

Good for distance-based models where magnitude differences strongly affect results.

Maps values to $[0, 1]$ · typical method: Min-Max Scaling.

Standardization

Use when features are expected to follow (or be close to) a Gaussian distribution, or when the dataset contains outliers.

Keeps data centered around 0 and scales variability to 1.

- Often preferred for linear models and optimization-based algorithms.

Feature Scaling Example (Normalization)

Worked example

Given feature values: [20, 40, 60, 80, 100]

Comparable range

Features brought to a comparable range.

After scaling

Heights are comparable.

$$x_{\text{norm}} = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

$$x_{\text{min}} = 20 \cdot x_{\text{max}} = 100$$

20

0.00

40

0.25

60

0.50

80

0.75

100

1.00

Before vs After Normalization

Feature Scaling Example

$$x_{\text{norm}} = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

$x_{\text{min}} = 20 \cdot x_{\text{max}} = 100$

Original value (x)	Normalized value (x _{norm})
20	0.00
40	0.25
60	0.50
80	0.75
100	1.00

This table shows how all values are scaled to the range [0, 1].

Feature Scaling Example (Standardization)

Worked example

Given feature values: [20, 40, 60, 80, 100]

Comparable range

Features brought to a comparable range.

After scaling

Heights are comparable.

z =

$x - \mu$

σ

Mean: $\mu = 60$ · Standard deviation: $\sigma = 28.28$

20

-1.41

40

-0.71

60

0.00

80

0.71

100

1.41

Before vs After Standardization

Feature Scaling Example

$$z = \frac{x - \mu}{\sigma}$$

Mean: $\mu = 60$ · Standard deviation: $\sigma = 28.28$

Original value (x)	Standardized value (z)
20	-1.41
40	-0.71
60	0.00
80	0.71
100	1.41

This table shows that standardized values are centered around 0 and measured in units of standard deviation.

Categorical Data

What is Categorical Data?

Categorical data describes values that belong to named groups rather than continuous numbers.

Category Types

Ordinal vs Nominal

Type	Description	Example
Ordinal	Categories have a meaningful order.	Education Level: High School < Bachelor's < Master's < PhD
Nominal	Categories have no natural order.	Payment Method: Cash, Card, Bank Transfer

Choose encoding based on whether order matters.

Example 1: Label Encoding

Encoding categorical data

Raw category	Encoded value
Cash	0
Card	1
Bank Transfer	2

Each category is mapped to a unique integer.

Example 2: One-Hot Encoding

Encoding categorical data

Raw category	Cash	Card	Bank Transfer
Cash	1	0	0
Card	0	1	0
Bank Transfer	0	0	1

Most machine learning algorithms work with numerical inputs only, so categorical values must be converted into numbers.

How to Deal with Categorical Data (Ordinal Encoding)

What is Ordinal Encoding?

Ordinal Encoding converts ordered categories into integers that preserve their ranking.

Encoding Comparison

Choosing an encoding

Three ways to encode categorical variables



[Label Encoding](#) · [One-Hot Encoding](#) · [Ordinal Encoding](#)

How to Apply Ordinal Encoding

Practical steps

Use Ordinal Encoding only when categories have a true order.

- Step 1: Identify the correct category order.
- Step 2: Assign an integer to each level based on that order.
- Step 3: Replace the original text values with encoded numbers.

Example: Ordinal Encoding

Ordinal Encoding

Satisfaction level	Encoded value
Low	1
Medium	2
High	3

This method keeps the order information, which is important for many machine learning models.

How to Deal with Categorical Data (One-Hot Encoding)

What is One-Hot Encoding?

One-Hot Encoding converts each category into a separate binary column (0 or 1).

How to Apply One-Hot Encoding

Practical steps

Use One-Hot Encoding for nominal categories (no natural order).

- Step 1: List all unique categories.
- Step 2: Create one column for each category.
- Step 3: Put 1 in the matching category column and 0 in all others.

Example: One-Hot Encoding

One-Hot Encoding

Payment method	Cash	Card	Bank Transfer
Cash	1	0	0
Card	0	1	0
Bank Transfer	0	0	1

This method avoids creating a false ranking between categories.

Avoiding the Dummy Variable Trap

One-Hot Encoding

It helps prevent the dummy variable trap (perfect multicollinearity), especially in linear models.

- Step 1: Start with One-Hot encoded columns.
- Step 2: Drop one category as a reference (baseline).
- Step 3: Keep the remaining binary columns.

Original categories: Payment Method = [Cash, Card, Bank Transfer]. · Reference category (dropped): Cash.

Example (Reference = Cash)

Dummy variable trap

Payment method	Card	Bank Transfer
Cash	0	0
Card	1	0
Bank Transfer	0	1

In this setup, Cash is represented when all remaining columns are 0.

Encoding Methods Overview

Beyond label / one-hot / ordinal

Encoding type	Idea	Best use case
Frequency Encoding	Replace each category with how often it appears.	Large datasets with many categories
Count Encoding	Similar to frequency, but uses raw counts directly.	Tree-based models with high-cardinality features
Target Encoding	Replace category with target mean for that category.	Supervised tasks (use with care to avoid leakage)
Binary Encoding	Convert category index to binary digits across columns.	High-cardinality data with fewer columns than one-hot
Hash Encoding	Use a hash function to map categories into fixed columns.	Very large and dynamic category sets

1) Frequency Encoding (City)

Encoding methods

City	Frequency
Riyadh	0.50
Jeddah	0.33
Dammam	0.17

2) Count Encoding (City)

Encoding methods

City	Count
Riyadh	3
Jeddah	2
Dammam	1

3) Target Encoding (City)

Encoding methods

City	Mean target
Riyadh	0.80
Jeddah	0.40
Dammam	0.20

4) Binary Encoding (City)

Encoding methods

City	Category ID	Binary code
Riyadh	1	01
Jeddah	2	10
Dammam	3	11

5) Hash Encoding (4 Buckets)

Encoding methods

City	Hash bucket
Riyadh	2
Jeddah	0
Dammam	3

Handling Missing Data

What is Missing Data?

Missing data refers to values that are absent or incomplete in one or more features of a dataset.

- It can introduce bias into model training.
- It may reduce prediction accuracy.
- It can cause errors in analysis if not handled properly.

Types of Missing Data

MCAR · MAR · MNAR

MCAR

X	Y
Missing	Y
X	Missing
X	Y
Missing	Y

Random gaps
no pattern

Types of Missing Data

MAR

X↑	Missing
X↑	Missing
X	Y
X	Y
X	Y

Depends on
observed X

MNAR

X	Missing
X	Missing
X	Y
X	Y
X	Y

Depends on
missing Y itself

MCAR = random · MAR = explained by observed features · MNAR = tied to the missing value itself.

Why Do We Have Missing Data?

Handling Missing Data

Missing values can appear for many practical reasons during data collection and processing.

Common Causes

Why values go missing

Cause	Description	Example
Human Error	Data is skipped, entered incorrectly, or forgotten.	A user leaves the Age field empty in a form.
Device/Sensor Failure	Measurement tools fail or disconnect.	A medical sensor stops recording heart rate temporarily.
Data Integration Issues	Missing fields appear when combining data from multiple sources.	One database has salary, another does not.
Privacy or Refusal	Participants choose not to share sensitive information.	A customer does not provide income details.
System/Transmission Errors	Data is lost during storage, transfer, or export.	Network interruption causes missing rows in logs.
Conditional Missingness	Some fields are only relevant for specific groups.	Pregnancy_Status is empty for male patients.

Understanding why data is missing helps us choose the right treatment method (deletion, imputation, or advanced modeling).

Missing Completely at Random (MCAR)

Handling Missing Data · MCAR

MCAR means missing values occur randomly and are independent of both observed and unobserved variables.

- MCAR has no systematic pattern.
- In practice, MCAR is often an unrealistic assumption in real-world datasets.
- Before assuming MCAR, review data collection workflow and consult domain experts.
- If MCAR is reasonable, simple imputation (mean/median/mode) can work.

Illustrative Data Example

Handling Missing Data · MCAR

Student ID	Age	Score
101	20	85
102	Missing	88
103	21	Missing
104	23	91
105	Missing	76
106	22	Missing

Missing values appear across different rows and columns with no clear pattern, which is consistent with MCAR.

Missing at Random (MAR)

Handling Missing Data · MAR

MAR means missingness is related to observed features in the dataset, but not to the missing feature itself.

- Missingness depends on other observed variables.
- The missing value itself is not the direct cause of the gap.
- More realistic than MCAR in many real-world datasets.
- Model-based imputation that uses observed features usually works better than simple mean/median fills.

Imputation Techniques for MAR

Handling Missing Data · MAR

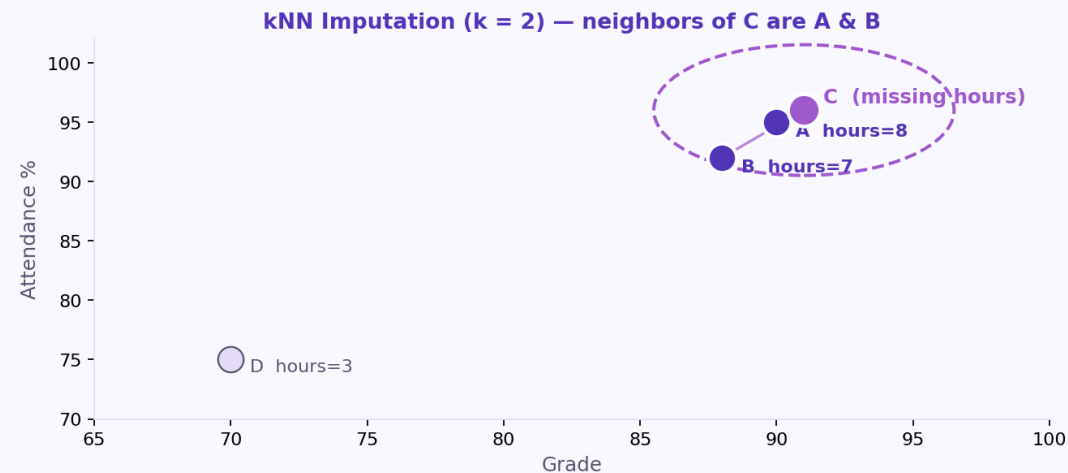
Method	Idea	Why it works for MAR
kNN Imputation	Fills missing values using the nearest similar records.	Uses observed features to find similar patterns.
MissForest	Uses random-forest models iteratively to predict missing values.	Captures non-linear relationships from observed data effectively.

Both methods leverage observed features — exactly what MAR assumes is available and informative.

kNN Imputation ($k = 2$)

Handling Missing Data · MAR

For student C, the two nearest students based on Grade and Attendance are A and B.



- Use observed features (Grade, Attendance) to find the nearest neighbors.
- Impute student C's missing value from students A and B (e.g. average).

kNN Neighbor Table

Handling Missing Data · MAR

Student	Grade	Attendance (%)	Study Hours
A	90	95	8
B	88	92	7
C	91	96	Missing
D	70	75	3

Student C is closest to A and B on Grade and Attendance — so we ignore distant student D.

kNN Imputed Result

Handling Missing Data · MAR

$$\text{Study Hours}_C = (8 + 7) / 2 = 7.5$$

Average the Study Hours of the two nearest neighbors (A and B) to fill the missing value for student C.

- Neighbor A → Study Hours = 8
- Neighbor B → Study Hours = 7
- Imputed value for C → 7.5

Example: MissForest

Handling Missing Data · MAR

MissForest trains a Random Forest model using known rows to predict the missing value for student C.

- Treat the incomplete column (Study Hours) as the target.
- Train on complete rows (A, B, D) using observed features such as Grade and Attendance.
- Predict Study Hours for student C from the learned non-linear relationships.
- Iterate across features with missing values until predictions stabilize.

MissForest Result

Handling Missing Data · MAR

Study Hours_C $\approx$ 7.6

The Random Forest prediction for student C's missing Study Hours is approximately 7.6.

- Uses non-linear patterns learned from complete rows.
- Often more accurate than simple neighbor averages when relationships are complex.

Example: MAR Grades Table

Handling Missing Data · MAR

Student	Grade (Observed)	Study Hours
1	95	Missing
2	92	Missing
3	88	4
4	75	6
5	70	7

MAR is common in practice because missingness can often be explained by available variables.

Missing Not at Random (MNAR)

Handling Missing Data · MNAR

MNAR is usually the most complex type because missingness depends on the missing value itself or unobserved variables.

- MNAR has a systematic pattern tied to hidden information.
- Standard imputation can be unreliable without additional data or strong domain assumptions.
- Often needs specialized models, sensitivity analysis, or collecting the missing information directly.
- Before treating MNAR like MAR/MCAR, validate the missingness mechanism with domain experts.

Example: MNAR Stress Reports

Handling Missing Data · MNAR

Participant	Reported Stress Level
1	2
2	Missing
3	3
4	Missing
5	1

Missing stress reports may be tied to the stress level itself — people with higher stress may skip the question.

Practical Tip: Preserve Missingness Pattern

Handling Missing Data · MNAR

Preserve the missingness pattern with an indicator column

Participant	Stress (imputed)	Was_Missing
1	2.0	0
2	4.1	1
3	3.0	0
4	4.3	1

Add an indicator column so the model can still learn from the fact that a value was missing.

Data Leakage

Keep the test set unseen

Data leakage happens when information from outside the training process — especially the test set — influences model fitting or decisions.



- Never use test labels or test-only signals when choosing features, models, or hyperparameters.
- Fit scalers and encoders on the training set only, then transform train and test.

Rule: treat the test set as future data you have not seen yet.
Peeking at the test set creates optimistic scores that will not hold in production.

Week 1 · Session 2

Regression Models

From linear fits to regularized models

Where Are We in the Bootcamp?

After preprocessing — learn models that predict continuous targets.

Week 1

S1 Foundations & Preprocessing

S2 **Regression Models**

S3 Classification Basics

S4 Naive Bayes & Trees

S5 SVM & Kernels

S6 Clustering & PCA

Week 2

S7 Deep Learning

Week 3

S8 NLP Fundamentals

S9 Tokenization

S10 Text Analysis & NER

S11 Language Modeling

S12 Embeddings & RNNs

S13 Seq2Seq & NMT

Week 4

S14 Generative AI

S15 RAG Systems

S16 MLOps

Week 1 · Session 2

Regression Models

Regression Models

Linear regression

OLS & R^2

Residual diagnostics

Polynomial regression

Regression Models

What we will cover in this session

- Linear regression
- OLS & R^2
- Residual diagnostics
- Polynomial regression
- SVR
- Decision Tree
- Random Forest

What is Simple Linear Regression?

Regression · Simple Linear Regression

Simple Linear Regression models the relationship between one independent variable x and one dependent variable y using a straight line.

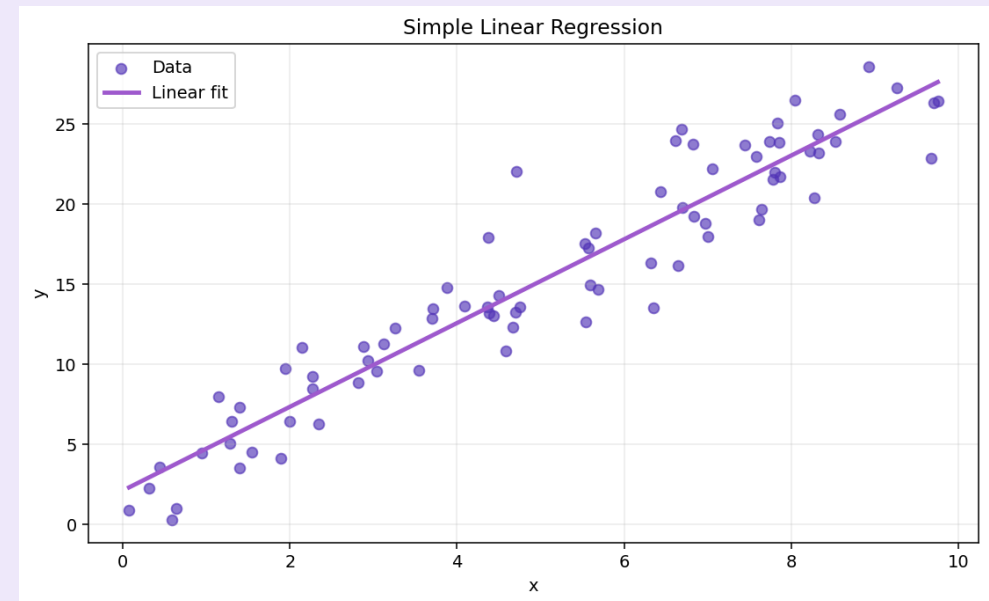
Formula

$$y = b_0 + b_1x$$

b_0 = intercept (value of y when $x = 0$) · b_1 = slope (change in y per unit change in x)

- x = input feature · y = target to predict
- Finds the straight line that best fits the data.

One feature → one straight-line fit. Multiple features come next.



Practical Example

Simple Linear Regression

Suppose x = Hours of Study and y = Exam Score.

$$y = 40 + 5x$$

- Slope $b_1 = 5$: +1 study hour $\rightarrow$ +5 score points.
- Intercept $b_0 = 40$: score when hours = 0.
- Example: 3 hours $\rightarrow \hat{y} = 40 + 5(3) = 55$.

Meaning of Each Symbol

Simple Linear Regression

Symbol	Meaning
y	Predicted output (dependent variable).
x	Input feature (independent variable).
b_0	Intercept: predicted value of y when $x = 0$.
b_1	Slope: expected change in y when x increases by 1 unit.

The regression line is the best-fit line that minimizes prediction errors.

Multiple Linear Regression

Linear Regression · Multiple Features

Multiple Linear Regression describes the relationship between one dependent variable y and two or more independent variables.

$$y = b_0 + b_1x_1 + b_2x_2 + \cdots + b_nx_n$$

Each b_j is the effect of feature x_j holding other features fixed.

- Predict y from several inputs at once.
- Still linear in the coefficients $b_0 \dots b_n$.

OLS still minimizes the sum of squared residuals.

House Price Example

Multiple Linear Regression

Predict house price from size and bedrooms.

$$y = 50 + 2.5x_1 + 15x_2$$

y = price · x_1 = size (m²) · x_2 = bedrooms

- +1 m² → price +2.5 (other variables fixed).
- +1 bedroom → price +15 (other variables fixed).

Interpret each coefficient holding other variables constant.

House Price Calculations

Multiple Linear Regression · Example

House	Size x_1 (m ²)	Bedrooms x_2	Predicted price y (\$1000)
A	100	2	$50 + 2.5(100) + 15(2) = 330$
B	120	3	$50 + 2.5(120) + 15(3) = 395$
C	80	1	$50 + 2.5(80) + 15(1) = 265$

Same formula, different inputs → different predicted prices.

What is OLS?

Ordinary Least Squares (OLS)

Ordinary Least Squares is the most common method to fit a linear regression line by minimizing prediction errors.

- Chooses the line with the smallest total squared error.
- Residual = actual y – predicted $\hat{y}$.
- Simple linear case has a closed-form solution for b_0 , b_1 .

Smaller squared errors → better fit on the training data.

Sum of Squared Errors

Ordinary Least Squares (OLS)

OLS chooses parameters by minimizing the total squared difference between actual and predicted values.

$$\min \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Sum of squared residuals over all n training points

- Actual value: y_i
- Predicted value: $\hat{y}_i$
- Residual error: $e_i = y_i - \hat{y}_i$

Squared errors make all errors positive, penalize large errors, and provide a clear optimization objective.

Gradient Descent

OLS · Optimization

Loss curve $L(w)$ — minimum at the bottom of the bowl.

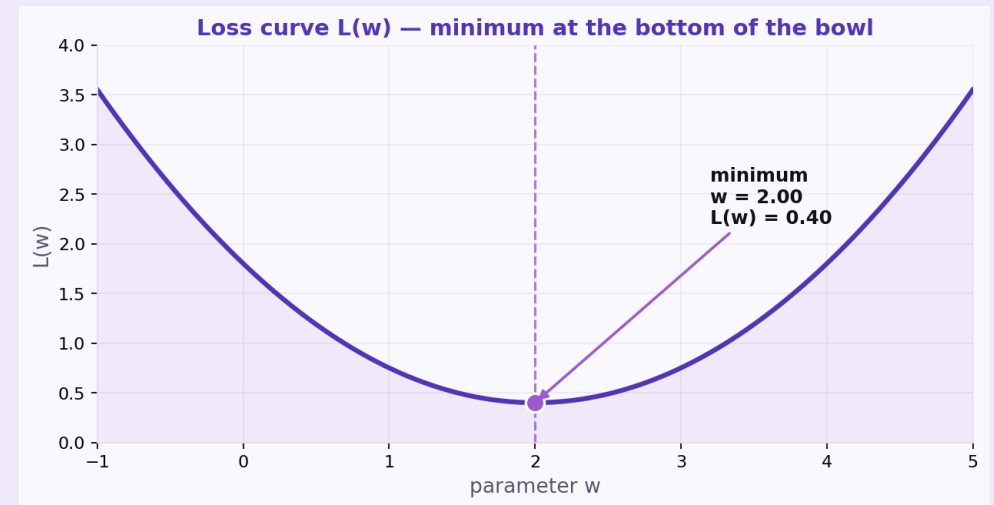
Formula

$$\min_w L(w)$$

We search for the parameter w that makes the loss as small as possible.

- Start with a guess for w , then move downhill.
- Stop when updates become tiny (converged).

Converged: $w = 2.00 \cdot L(w) = 0.40$



R-Squared (R^2)

Goodness of Fit

R^2 measures how much variation in y is explained by the model inputs.

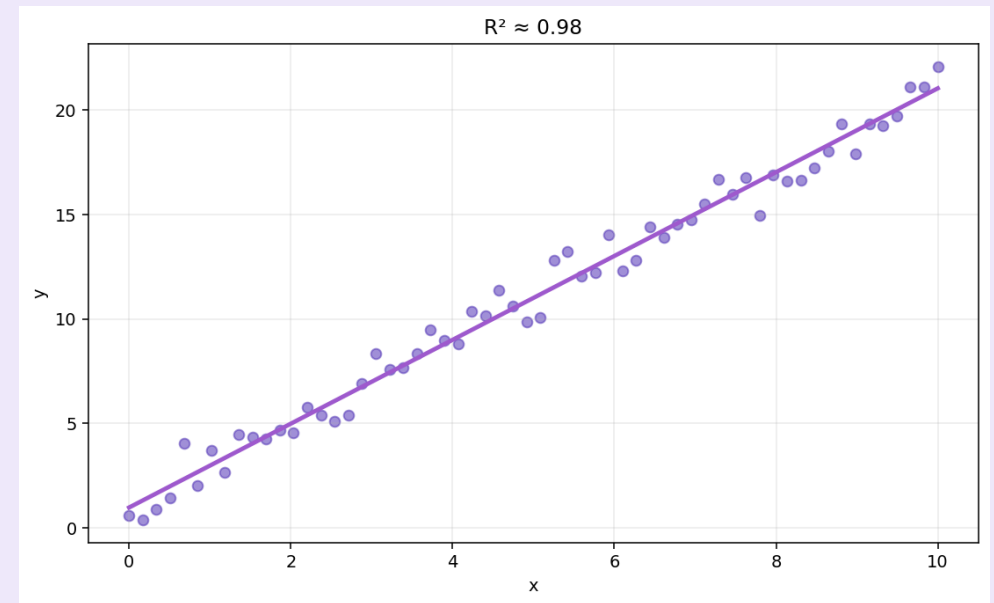
Formula

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

SS_{res} = residual SS · SS_{tot} = total SS

- Near 1 → strong explanatory power.
- Near 0 → little better than predicting the mean.

High R^2 on training data ≠ guaranteed good new-data predictions.

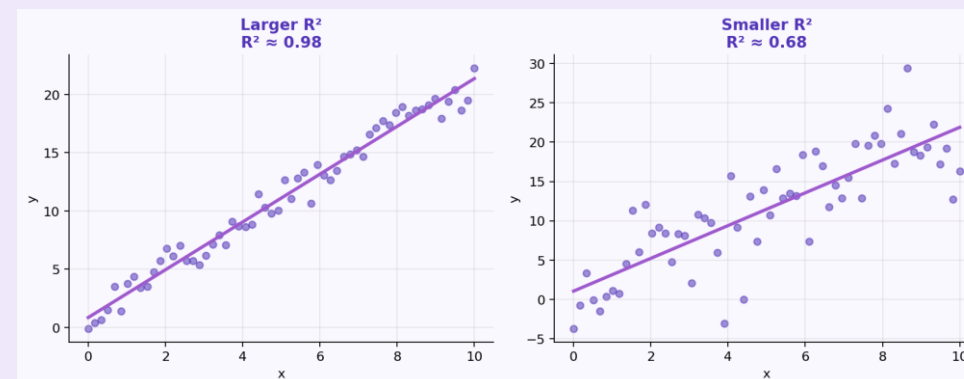


Interpreting R^2

Goodness of Fit

Larger R^2 means more of the variation in y is captured by the model.

- $R^2 = 1 \rightarrow$ perfect fit on that sample.
- $R^2 = 0 \rightarrow$ mean-level baseline performance.



Validate generalization with hold-out metrics, not R^2 alone.

Residual Sum of Squares

R² · Building Blocks

SS_res measures the unexplained error — how far predictions are from the actual values.

$$SS_{res} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Sum of squared residuals (prediction errors)

- Smaller SS_res → predictions closer to the data.
- Appears in the numerator of the R² formula.

Total Sum of Squares

R^2 · Building Blocks

SS_{tot} measures the total variation in y around its mean — the baseline scatter to explain.

$$SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2$$

Sum of squared deviations from the mean $\bar{y}$

- This is the variance (up to a constant factor) of the observed y values.
- R^2 asks: what fraction of SS_{tot} is left unexplained by SS_{res} ?

$$R^2 = 1 - SS_{res} / SS_{tot} \quad \cdot \quad \text{explained variation} = SS_{tot} - SS_{res}$$

What Do These Terms Mean?

R² · Building Blocks

Term	Description
SS_res	Residual Sum of Squares: squared distance between actual values and predictions.
SS_tot	Total Sum of Squares: squared distance between actual values and the mean of y.

R² compares unexplained error (SS_res) with total variation (SS_tot).

Why R^2 Alone Is Not Enough

Adjusted R-Squared

In multiple regression, adding predictors often increases R^2 even when the new variables are not useful.

- Ordinary R^2 never decreases when you add a predictor (it stays the same or goes up).
- That can reward bloated models that fit noise instead of signal.
- We need a metric that balances fit against model complexity.

Next: Adjusted R^2 applies a penalty for extra predictors.

Adjusted R-Squared

Goodness of Fit · Model Comparison

Adjusted R^2 penalizes unnecessary predictors.

$$R_{adj}^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

n = samples · p = predictors

- Better for comparing models with different p .
- Can fall when R^2 rises if the new variable is weak.

Adjusted R² — Quick Example

Adjusted R-Squared

Model	Predictors	R ²	Adjusted R ²	Interpretation
A	X ₁ , X ₂	0.82	0.81	Strong and efficient model
B	X ₁ , X ₂ , X ₃ , X ₄	0.83	0.79	Higher R ² , but worse after penalty

Model B looks better by R², but Adjusted R² shows the extra variables are likely not helpful.

Assumptions of Linear Regression

Overview

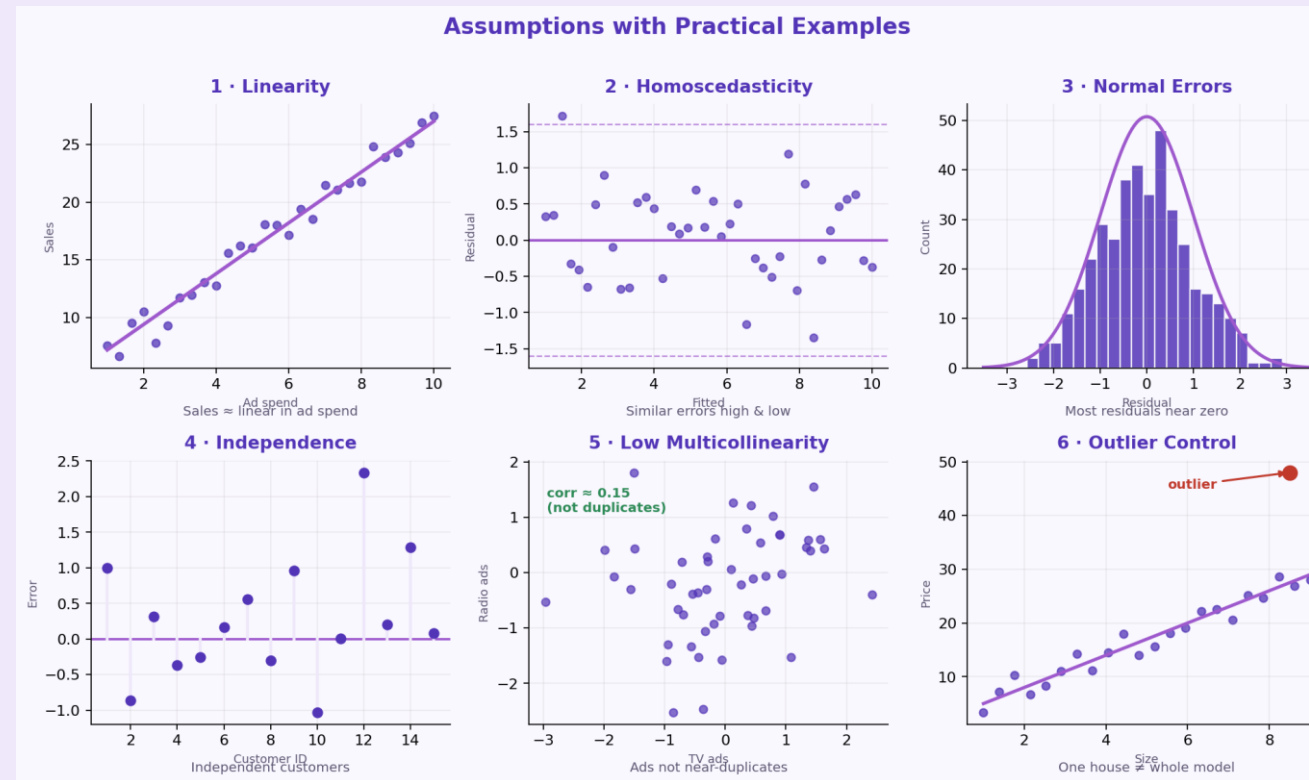
Assumption	Meaning	If Violated
Linearity	y vs each x is roughly linear	Biased predictions
Homoscedasticity	Error variance roughly constant	Unstable SEs
Normal Errors	Residuals $\approx$ normal	Weaker inference
Independence	Errors are independent	Biased tests
Low Multicollinearity	Predictors not highly redundant	Unstable coeffs
Limited Outliers	Extremes do not dominate fit	Distorted line

Next slides: practical checks and residual plots.

Assumptions with Practical Examples

Residual Diagnostics

Each panel shows a practical picture of one assumption — and what “good” looks like.



Checks: scatter/residuals · residuals vs fitted · histogram/Q-Q · design/DW · corr/VIF · Cook/leverage.

Assumptions — Quick Reference

Practical Examples

Assumption	Example	Check
Linearity	Sales $\approx$ linear in ad spend	Scatter / residual plot
Homoscedasticity	Similar errors high & low	Residuals vs fitted
Normal Errors	Most residuals near zero	Histogram / Q-Q
Independence	Independent customers	Design / Durbin-Watson
Low Multicollinearity	Ads not near-duplicates	Corr / VIF
Outlier Control	One house $\neq$ whole model	Cook / leverage

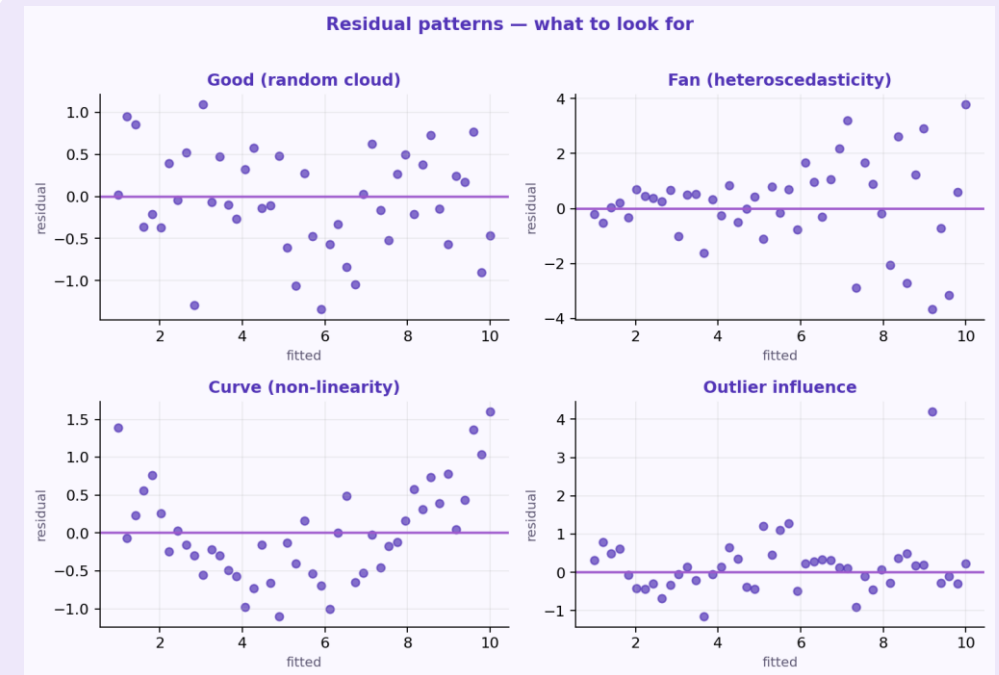
If violated: transforms, robust methods, or another model.

Residual Patterns to Spot

Assumption Checks

Common residual-plot signatures for assumption problems.

- Good: random cloud around zero.
- Fan / curve / outlier → investigate.

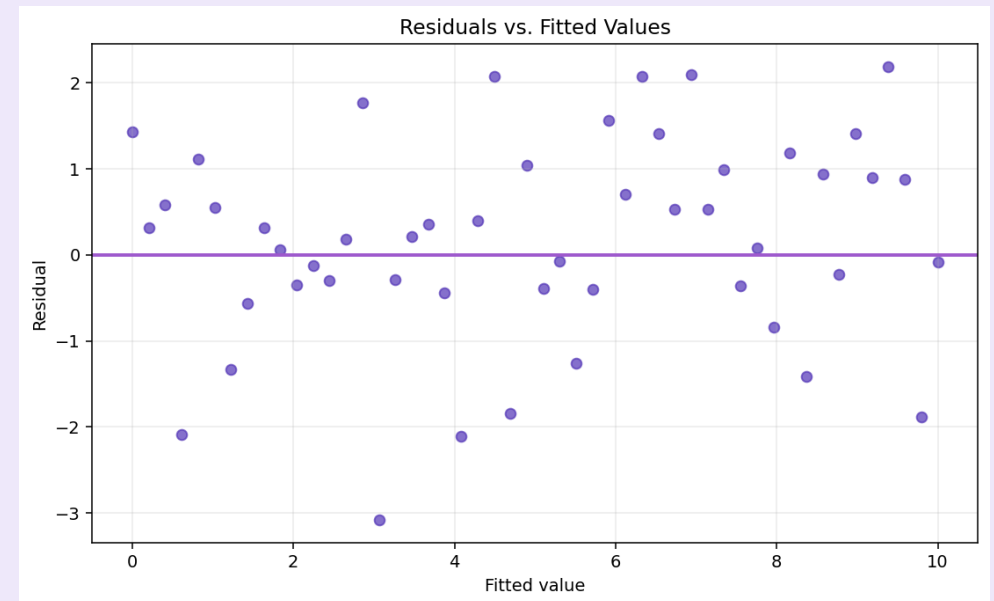


Residuals vs Fitted Values

Practical Check · Homoscedasticity

Look for random scatter around zero with roughly constant spread.

- No curve → linearity looks OK.
- Fan shape → possible heteroscedasticity.



Building a Regression Model

Goal

Method	Core Idea	Best Use Case
All-in	Use all predictors	Strong prior knowledge
Backward	Start full, remove weakest	Many candidates
Forward	Start empty, add strongest	Compact model
Stepwise	Add and remove dynamically	Flexible search

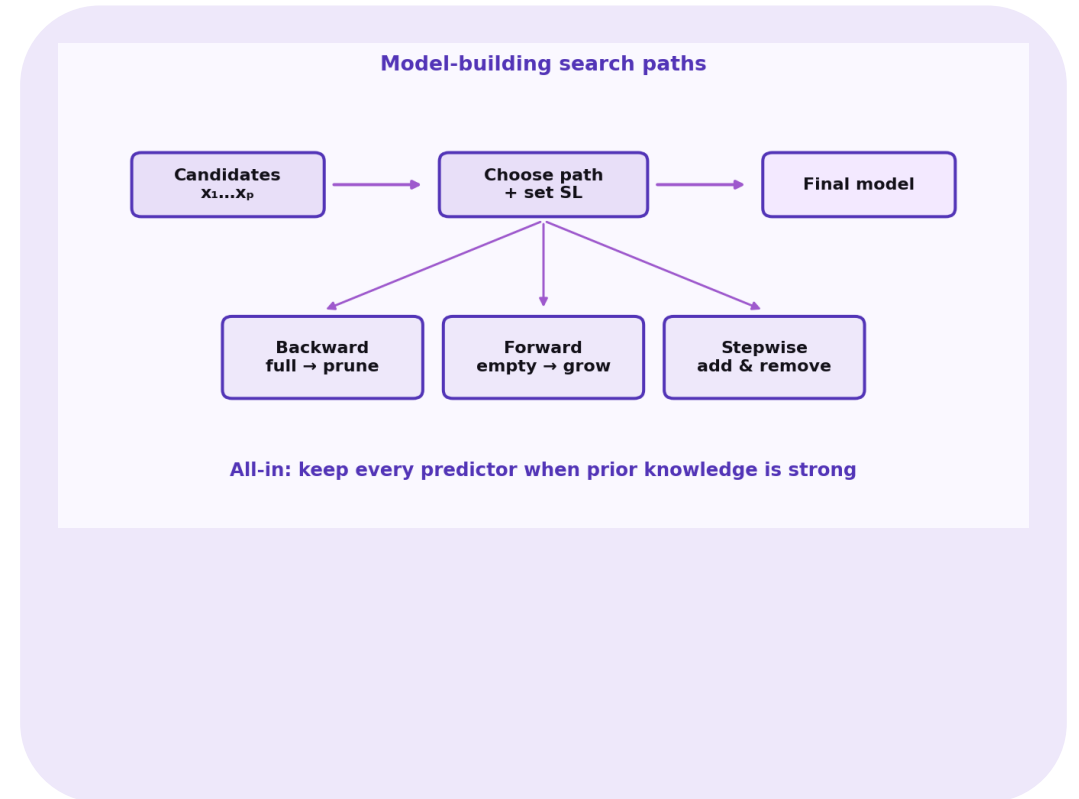
Match the method to candidate count and prior knowledge.

Model-Building Flow

All-in · Backward · Forward · Stepwise

Different search paths for choosing predictors under a significance level.

- Backward: prune a full model.
- Forward: grow from empty.
- Stepwise: add and remove as you go.



Backward Elimination

Model-Building Methods · Steps

Start full, then drop the least useful predictors one at a time.

- Set SL (e.g. 0.05) and fit all predictors.
- Remove highest p if $p > SL$; refit and repeat.

Best with many candidate predictors.

Forward Selection

Model-Building Methods · Steps

Grow the model from scratch by adding the strongest predictors.

- Start empty; add lowest p below SL.
- Stop when no remaining variable qualifies.

Best when you want a compact model from the start.

Bidirectional (Stepwise)

Model-Building Methods · Steps

Add and remove predictors dynamically each iteration.

- Forward step: add significant variables.
- Backward step: drop non-significant ones.

More adaptive than pure forward or backward alone.

Mini Example — Backward Elimination

Model-Building Methods

Iteration	Candidate Predictors	Selected Action
Start	x_1, x_2, x_3, x_4	Full model
1	highest p-value: $x_4 = 0.42$	Remove x_4
2	highest p-value: $x_3 = 0.11$	Remove x_3
Final	x_1, x_2 with $p < 0.05$	Keep final model

With $SL = 0.05$, variables with $p > SL$ are removed until only significant predictors remain.

Significance Level and p-value

Model Selection · Decision Rule

Compare p to alpha to decide if a predictor is statistically significant.

Formula

$$0.03 < 0.05 \Rightarrow \text{Reject } H_0$$

Example: $p = 0.03$ · $\alpha = 0.05$

- $p < \alpha \rightarrow \text{reject } H_0 \text{ (significant)}$.
- $p \geq \alpha \rightarrow \text{fail to reject } H_0$.

Predictor is significant at the 5% level in this example.

Decision rule: compare p-value to alpha



Key Concepts: H_0 , p-value, Alpha

Significance Level and p-value

Concept	Meaning
Null Hypothesis (H_0)	No effect / no relationship
p-value	Probability of observing current result (or more extreme) if H_0 is true
Significance Level (alpha)	Maximum tolerated Type I error probability

In model building, predictors with $p < \alpha$ are kept as statistically significant.

Why Is $SL = 0.05$ Common?

Standard Choice

$SL = 0.05$ balances sensitivity and false-positive risk.

$$SL = 1 - \textit{ConfidenceLevel}$$

95% confidence $\rightarrow SL = 0.05$

- Convention, not a law — adjust for domain risk.
- Report effect size and CIs with p-values.

SL vs Confidence Level

Why Is $SL = 0.05$ Common?

SL	Confidence Level	Strictness
0.10	90%	Less strict
0.05	95%	Standard
0.01	99%	More strict

Lower SL → higher confidence requirement → stricter evidence to keep a predictor.

How to Choose in Practice

Significance Level

Scenario	Suggested SL
High-stakes decisions (medicine, safety, finance)	0.01
General modeling and reporting	0.05
Exploratory analysis	0.10

Start with SL = 0.05, then adjust based on domain risk, dataset size, and model purpose.

Polynomial Regression

Definition

Polynomial Regression extends linear regression to model non-linear relationships by adding polynomial terms.

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots$$

Still linear in the coefficients

- Adds polynomial terms to capture curves.
- Still linear in parameters $b_0, b_1, \dots$

Transform $x \rightarrow x^2, x^3, \dots$ then fit linear regression.

Why Polynomial Regression?

Motivation

Many real-world relationships are curved — a straight line underfits them.

- Population growth · disease progression
- Sales trends · rainfall vs crop yield

How Polynomial Regression Works

Polynomial Regression

Step	Description
1	Start with input feature x
2	Create transformed terms: $x^2, x^3, \dots$
3	Fit linear regression on transformed features
4	Estimate coefficients ($b_0, b_1, b_2, \dots$)
5	Use the fitted equation to predict y

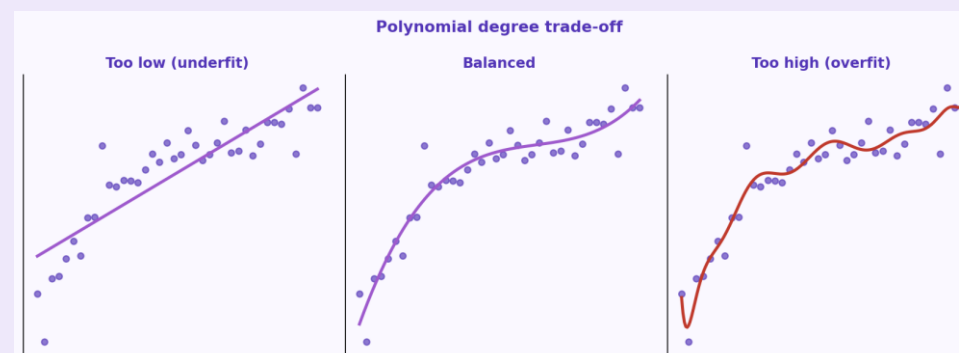
d is the polynomial degree.

Choosing the Polynomial Degree

Model Complexity

Degree controls complexity — too low underfits, too high overfits.

- Pick degree with CV / validation error.
- Also compare adjusted R^2 .



Balanced degree → better bias-variance trade-off.

When to Use Polynomial Regression

Good Use Cases

Data shows clear curved trends · more flexibility than a straight line · you still want interpretable behavior.

Application	Example
Forecasting	Demand and sales trends
Environment	Rainfall vs crop yield
Medicine	Disease / progression patterns
Engineering	Stress-strain and similar curves

Choose polynomial regression when curves matter, but full black-box models are not required yet.

Limitations of Polynomial Regression

Trade-offs

Limitation	Explanation
Overfitting risk	High-degree models may memorize noise
Outlier sensitivity	Extreme points can shift the curve strongly
Extrapolation weakness	Predictions outside training range can be unreliable
Limited flexibility for very complex patterns	Some tasks are better handled by tree-based or neural models

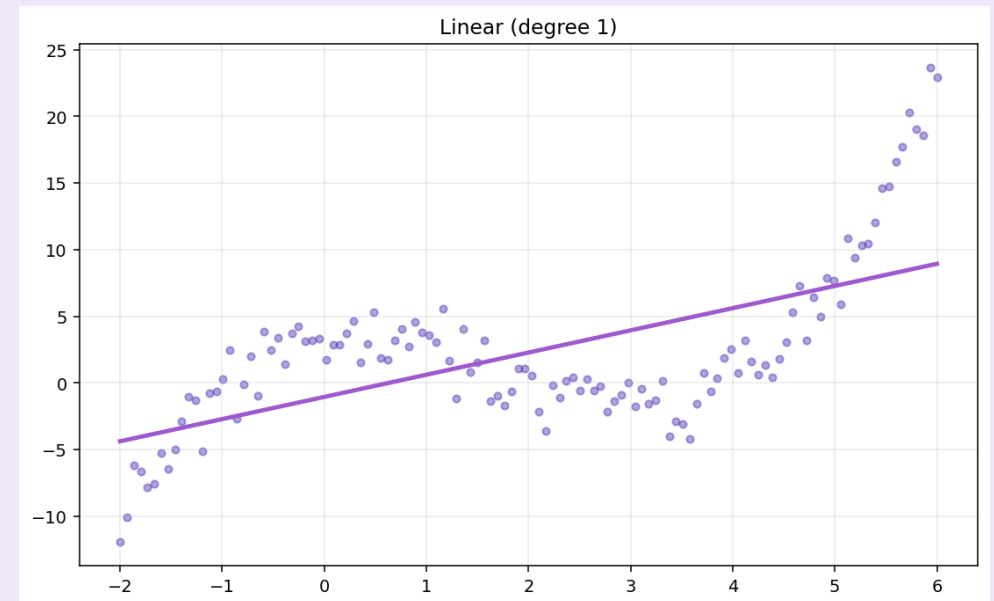
Use polynomial regression as a baseline, then compare with alternatives as complexity grows.

Polynomial Regression Visual Explanation

Linear vs Polynomial Fits

Compare linear vs polynomial fits as degree changes.

- Too low degree → high bias (underfit).
- Too high degree → high variance (overfit).

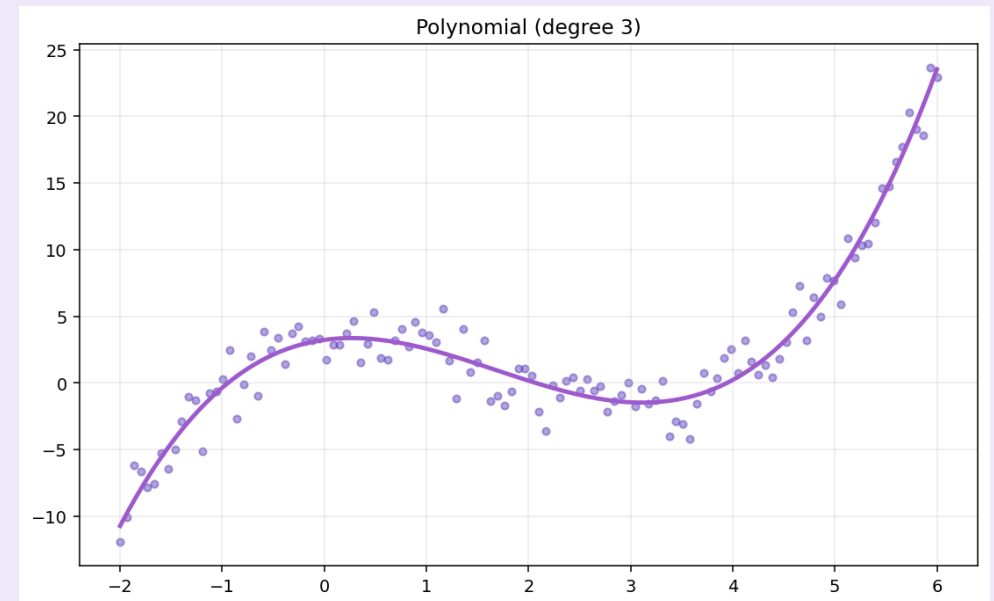


Effect of Polynomial Degree

Visual Explanation

Higher degree = more flexible curve — useful, then noisy.

- Pick degree with validation curves.
- Do not trust training error alone.

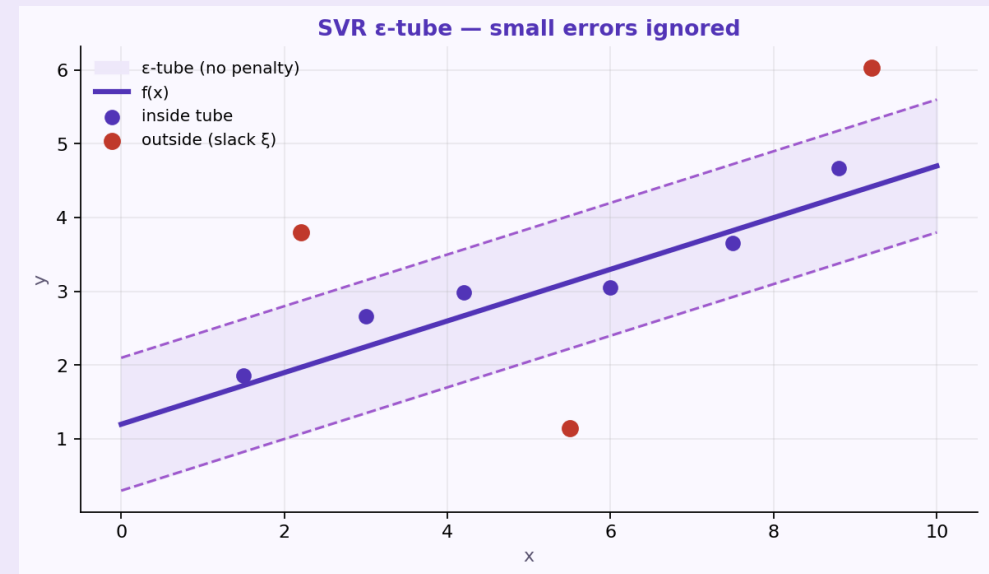


Support Vector Machine (SVM) and SVR

What is SVR?

SVR is the regression version of SVM: a flat function with a tolerance margin (ϵ -tube).

- SVM: widest margin; support vectors define it.
- SVR ignores small errors inside the ϵ -tube.
- Used in forecasting, engineering curves, disease progression.



SVM separates classes; SVR predicts continuous y with an ϵ -insensitive zone.

SVR Formulation and Equations

Model Form

SVR predicts with a (possibly kernelized) linear function in a feature space.

$$f(x) = w^{\top} \phi(x) + b$$

$\phi(x)$ maps input to a (possibly) higher-dimensional space

- $\phi(x)$ maps input to a (possibly) higher-dimensional space.
- With kernels, we avoid computing $\phi(x)$ explicitly.

Soft-Margin SVR Objective

Optimization Objective

Keep the function flat while penalizing points outside the ϵ -tube.

$$\min \frac{1}{2}\|w\|^2 + C \sum (\xi_i + \xi_i^*)$$

C trades flatness vs errors outside the tube

- $\frac{1}{2}\|w\|^2 \rightarrow$ flatter function.
- Larger C $\rightarrow$ tighter fit; smaller C $\rightarrow$ smoother.

SVR Constraints

Soft-Margin SVR

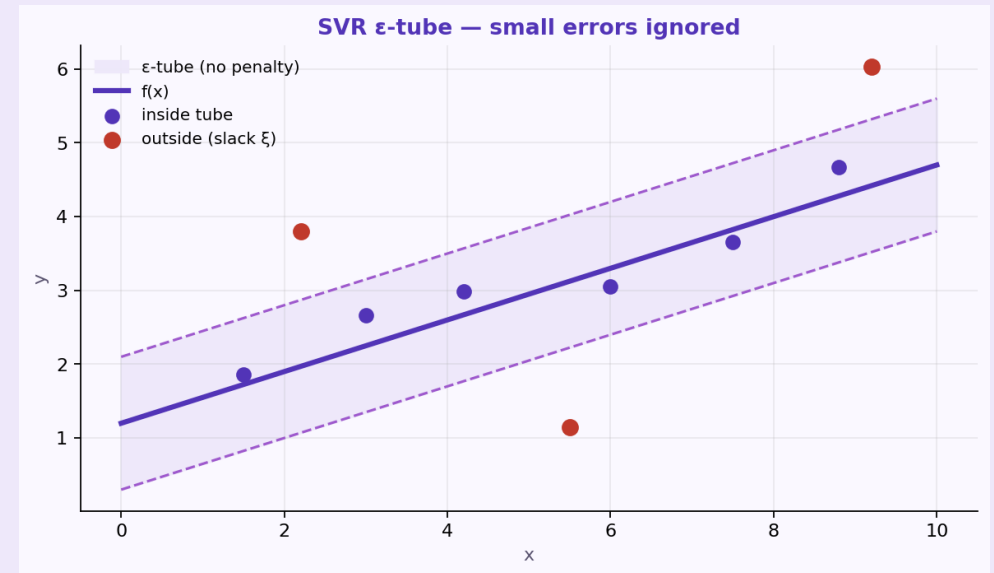
Stay within ε of y_i , or pay with a slack variable.

Formula

$$y_i - (w^\top \phi(x_i) + b) \leq \varepsilon + \xi_i$$

Also: $(w^\top \phi(x_i) + b) - y_i \leq \varepsilon + \xi_i^* \cdot \xi_i, \xi_i^* \geq 0$

- Inside tube $\rightarrow$ no penalty.
- Outside tube $\rightarrow$ slack cost.



SVR Hyperparameters

Tuning Knobs

Hyperparameter	Role
epsilon (ϵ)	Width of the no-penalty tube around prediction
C	Penalty strength for points outside the tube
Kernel	Controls curve shape (linear, RBF, polynomial)

Tune ϵ , C, and the kernel together — changing one often changes how the others behave.

Why SVR?

Compared with Traditional Regression

Aspect	Traditional	SVR
Errors	Minimize all residuals	Ignore inside ϵ -tube
Outliers	Can be sensitive	More robust if tuned
Non-linear	Hand-built features	Kernels directly
Large data	Usually faster	Can get expensive

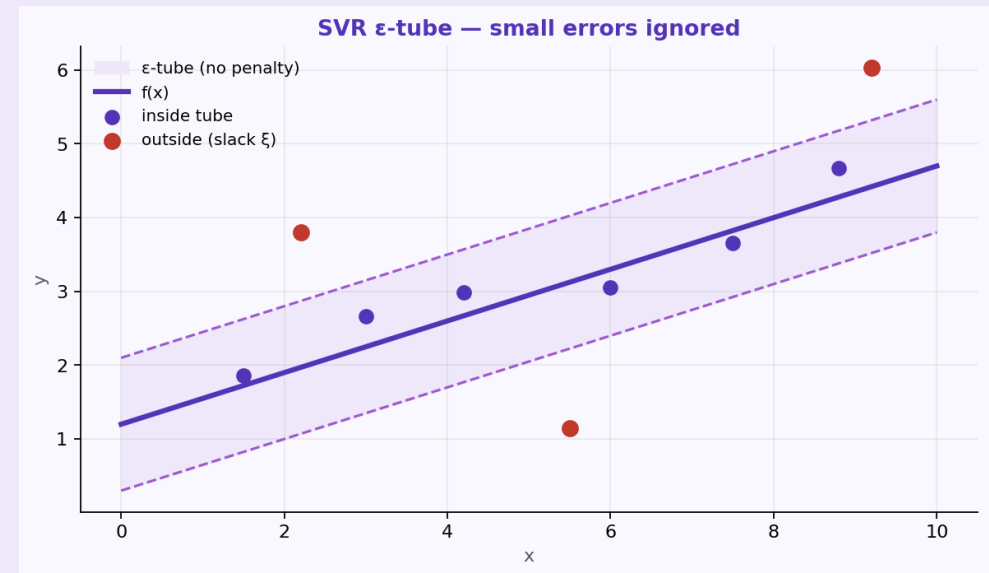
Scale features · start with RBF · tune C, ϵ , kernel params.

SVR Visual Intuition

Why SVR?

ϵ -tube ignores small residuals; larger deviations pull the fit.

- Inside tube $\rightarrow$ no loss.
- Kernels capture curves without hand features.

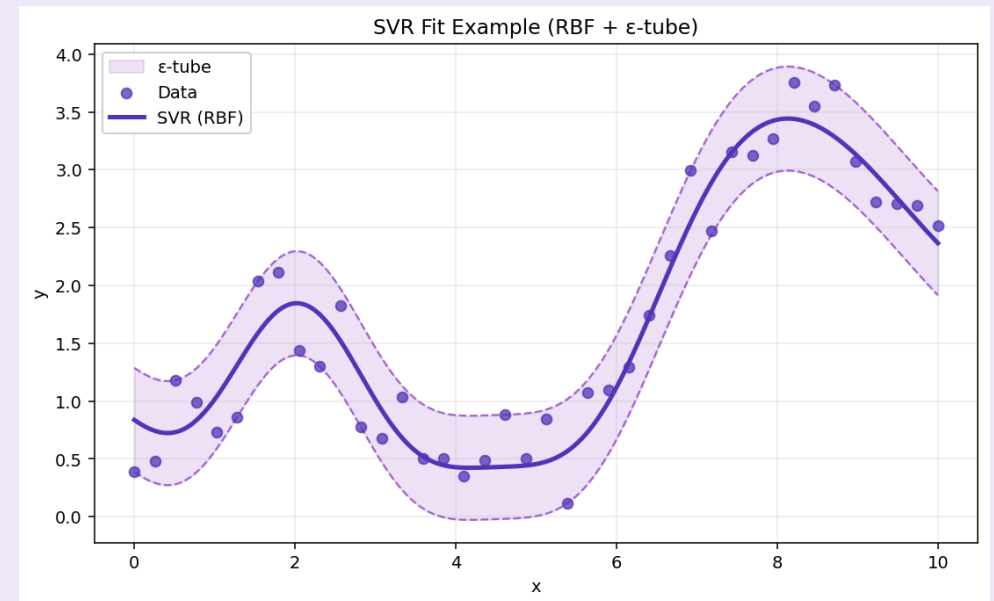


SVR Fit Example

Why SVR?

Tuned tube + kernel follows curves while resisting small noise.

- Scale features first.
- Tune C , ϵ , kernel with CV.

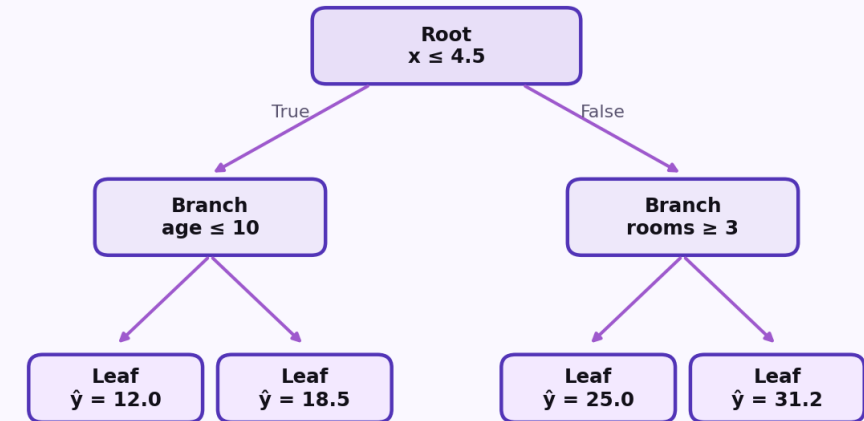


Decision Tree Regression (CART)

What is Decision Tree Regressor?

Decision Tree Regressor predicts continuous values by splitting feature space into rule-based regions and assigning a constant value in each region.

- Captures non-linear relationships.
- Easy to interpret if tree depth is controlled.
- Handles complex feature interactions.



Recursive partitioning by feature threshold · Root → branches → leaf nodes

Decision Tree Split: recursive partitioning by feature threshold · Root → branches → leaf nodes.

Core Concept of Regression Trees

Tree Structure

A regression tree is built from decision rules that partition the input space into regions with constant predictions.

- Internal node: decision rule (e.g., $x \leq 4.5$).
- Branch: outcome of the rule.
- Leaf node: predicted value (usually region mean).

CART Perspective

Classification and Regression Trees

Tree Type	Target Type	Typical Criterion
Classification Tree	Categorical target	Gini / Entropy
Regression Tree	Continuous target	MSE (or MAE)

This section focuses on the Regression Tree part of CART.

How Splitting Happens (MSE Criterion)

Node MSE

CART picks the split that maximally reduces MSE (regression) or Gini/entropy (classification).

$$\text{MSE}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y}_S)^2$$

Average squared distance from the node mean $\bar{y}_S$

- Greedy splits are fast but not globally optimal.
- Depth and min-samples-leaf control overfitting on small datasets.

Split Score

MSE Criterion

A candidate split is scored by the size-weighted average MSE of the left and right child nodes.

$$\text{MSE}_{\text{split}} = (|S_L|/|S|)\text{MSE}(S_L) + (|S_R|/|S|)\text{MSE}(S_R)$$

S_L / S_R = left and right subsets after the candidate split

- Choose the split with the minimum weighted MSE.
- Larger child nodes influence the score more (weighted by $|S_L|$ and $|S_R|$).

Prediction at Leaf

Regression Tree

Once a region is pure enough to stop splitting, the leaf predicts a constant value for every point that lands there.

$$\hat{y}_{leaf} = \frac{1}{|S_{leaf}|} \sum_{i \in S_{leaf}} y_i$$

Usually the mean of the training targets in that leaf region

- Same prediction for all samples that reach the same leaf.
- Deeper trees create smaller regions and more varied leaf values.

Decision Tree Regression Algorithm

Simple Steps

Step	Action
1	Start with all data at the root node.
2	Evaluate candidate splits across features.
3	Compute weighted MSE for each split.
4	Select the split with minimum error.
5	Repeat recursively on child nodes.
6	Stop based on rules (max depth, min samples).
7	Predict using leaf mean value.

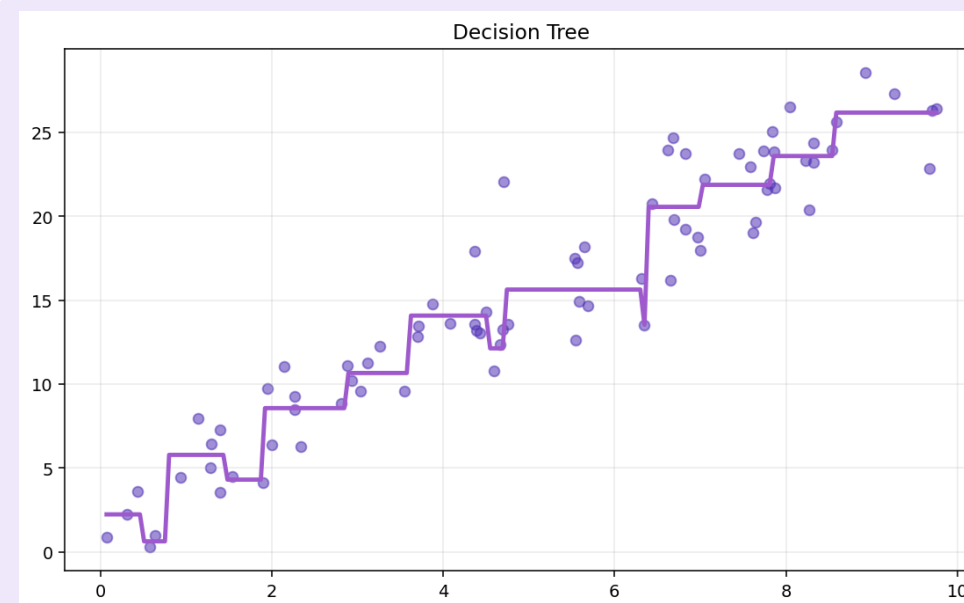
Decision Tree Split: greedy recursive partitioning until a stopping rule fires.

Decision Tree Fit (Visual)

Algorithm · Static Plot

A fitted regression tree partitions x into intervals and predicts a constant in each leaf region.

- Piecewise-constant prediction surface.
- More depth $\rightarrow$ more steps and finer partitions.



Important Hyperparameters

Decision Tree Regression

Hyperparameter	Effect
max_depth	Controls model complexity
min_samples_split	Minimum samples required to split a node
min_samples_leaf	Minimum samples in each leaf
max_leaf_nodes	Limits number of terminal regions

These settings reduce overfitting and improve generalization.

Random Forest Regression

What is Random Forest Regressor?

Ensemble of many trees — average their predictions.

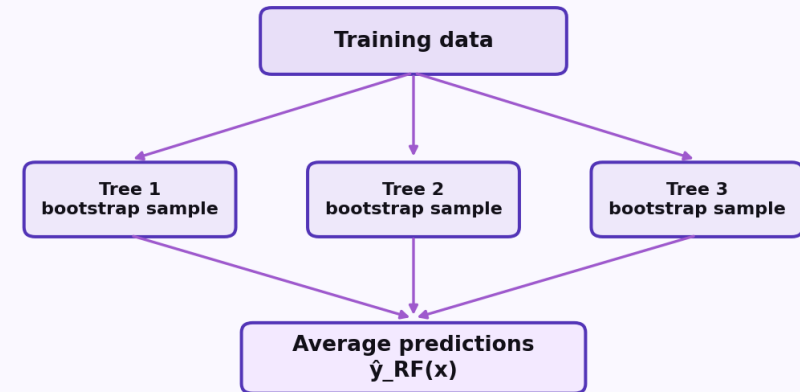
Formula

$$\hat{y}_{RF}(x) = \frac{1}{N_{trees}} \sum_{t=1}^{N_{trees}} \hat{y}_t(x)$$

Average of individual tree predictions

- Bootstrap samples → many trees.
- Averaging reduces variance vs one deep tree.

Also see fitted RF curve: slide-46-3.



Many trees · different samples/features · average = more stable fit

Decision Tree vs Random Forest

Regression Comparison

Aspect	Decision Tree	Random Forest
Model	Single tree	Many trees averaged
Split	MSE per split	MSE trees, then average
Overfit risk	Higher	Lower
Stability	Sensitive	More stable
Prediction	One tree output	Mean of trees
Interpretability	High	Medium

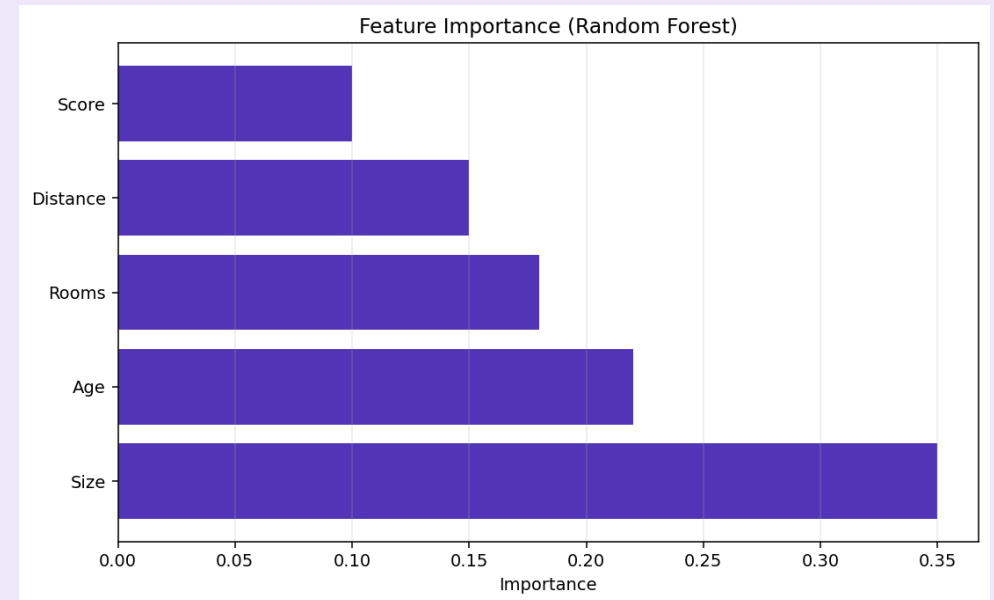
Tree for interpretability · Forest for stronger performance.

Feature Importance (Random Forest)

Decision Tree vs Random Forest

Importance scores show how much each feature helped reduce error.

- Higher bar → more contribution.
- Guide for insight — not causal proof.



Evaluating Regression Models

Key Performance Metrics

Metric	Formula	Tells Us
MAE	$(1/n) \sum y_i - \hat{y}_i $	Average error size
MSE	$(1/n) \sum (y_i - \hat{y}_i)^2$	Penalizes large errors
RMSE	$\sqrt{\text{MSE}}$	Error on y scale
R ²	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Variance explained

Use more than one metric (e.g. RMSE + R²).

Regularization Methods (Why Needed?)

Overfitting Problem

Too flexible → fits noise instead of true patterns.

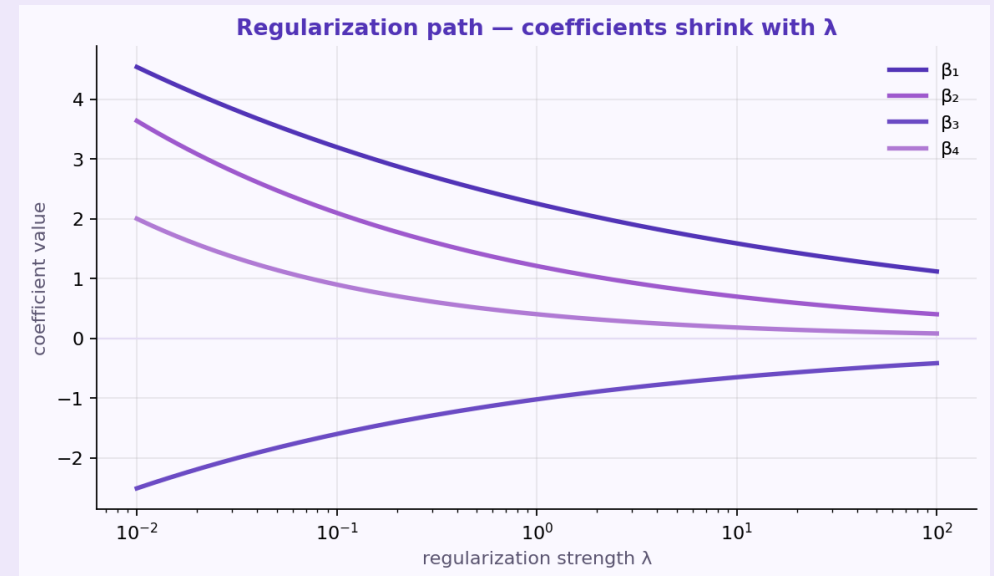
Formula

$$\min_{\beta} \sum (y_i - \hat{y}_i)^2 + \lambda \cdot \Omega(\beta)$$

data loss + $\lambda \times$ penalty

- Low train error, weak test performance.
- High variance / unstable predictions.

Penalize large coefficients to control complexity.



Regularization Objective

Data Loss + Penalty

Trade off fit vs small coefficients.

$$\min_{\beta} \text{data loss} + \lambda \cdot \text{penalty}$$

$$\text{loss} = \sum (y_i - \hat{y}_i)^2 \cdot \text{penalty} = \Omega(\beta)$$

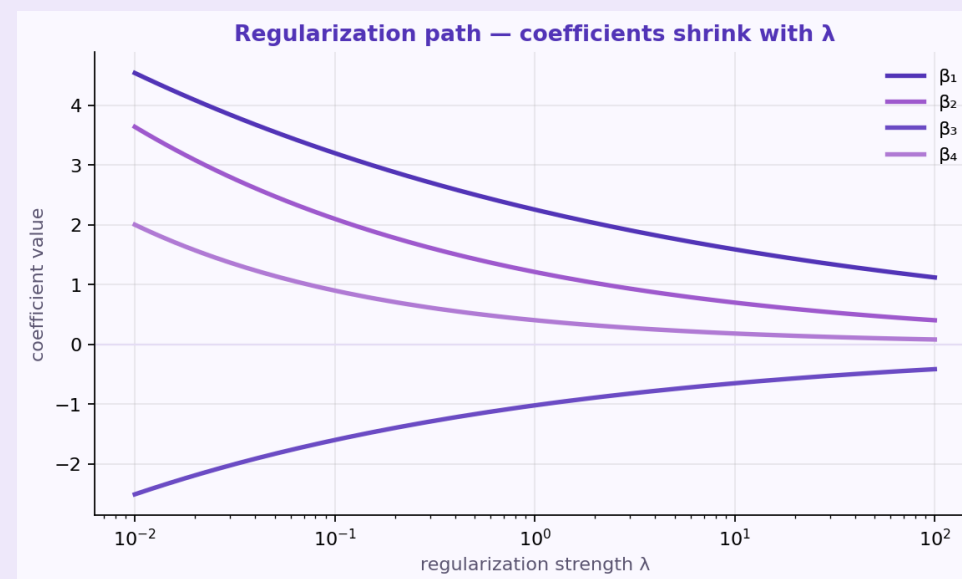
- $\lambda = 0 \rightarrow$ ordinary least squares.
- Larger $\lambda \rightarrow$ stronger shrinkage · tune with CV.

Ridge, Lasso, and Elastic Net

Regularization Families

Different penalties shrink coefficients differently as λ grows.

- Ridge (L2): shrink all — good with correlated features.
- Lasso (L1): can zero coeffs — feature selection.



Elastic Net blends L1 + L2 for correlated sparse settings.

Without Regularization

Ordinary Least Squares

Minimize squared error only — no coefficient penalty.

$$\min_{\beta} \sum (y_i - \hat{y}_i)^2$$

Same as $\lambda = 0$

- No shrinkage.
- Can overfit with many / correlated features.

Ridge Regression (L2)

Penalty: sum of squared coefficients

L2 penalty shrinks coefficients; rarely zeros them.

$$\min_{\beta} \sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2$$

$\lambda \geq 0$ controls shrinkage

- Smooth shrinkage of all coefficients.
- Good when many features correlate.

Lasso Regression (L1)

Penalty: sum of absolute coefficients

L1 penalty can set some coefficients exactly to zero.

$$\min_{\beta} \sum (y_i - \hat{y}_i)^2 + \lambda \sum |\beta_j|$$

Larger $\lambda \rightarrow$ sparser model

- Built-in feature selection.
- Best when few predictors truly matter.

Elastic Net (L1 + L2)

Blend of Lasso and Ridge

Combine L1 + L2 for sparsity with more stable correlation behavior.

$$\min_{\beta} \sum (y_i - \hat{y}_i)^2 + \lambda_1 \sum |\beta_j| + \lambda_2 \sum \beta_j^2$$

$\lambda_1 \rightarrow$ sparsity · $\lambda_2 \rightarrow$ ridge shrinkage

- Good for correlated feature groups.
- Tune λ_1, λ_2 (or mix ratio) with CV.

When to Use Each Method

Ridge · Lasso · Elastic Net

Match the penalty to the feature structure you expect.

- Ridge: many small/medium useful features.
- Lasso: few important features expected.
- Elastic Net: correlated features + need sparsity.

Regularization Comparison Table

Ridge vs Lasso vs Elastic Net

Method	Penalty Type	Main Behavior	Feature Selection
Ridge	L2 (sum β_j^2)	Shrinks coefficients smoothly	No
Lasso	L1 (sum $ \beta_j $)	Shrinks and can set some coefficients to zero	Yes
Elastic Net	L1 + L2	Combines shrinkage + selection	Yes

Scale features before Ridge/Lasso/Elastic Net — penalties depend on coefficient magnitude.

Regression Models — Pros & Cons

Session Summary

Model	Advantages	Disadvantages
Linear	Simple, fast, interpretable	Linear only; outlier-sensitive
Polynomial	Captures curves	Degree sensitive; can overfit
SVR	Kernels; robust if tuned	Needs scaling; slow on big data
Decision Tree	Interpretable; no scaling	Overfits; unstable
Random Forest	Accurate; lower variance	Less interpretable; heavier

Interpretability → Linear/Tree · Performance → Random Forest.

Model Summary (Name + Equation)

Session Summary

Model	Core Equation
Linear Regression	$\hat{y} = b_0 + b_1x$
Polynomial Regression	$\hat{y} = b_0 + b_1x + b_2x^2 + \dots + b_dx^d$
SVR	$f(x) = w^T \varphi(x) + b$
Decision Tree Regression	$\hat{y}_{leaf} = \text{mean}(y \text{ in } S_{leaf})$
Random Forest Regression	$\hat{y}_{RF}(x) = (1/N_{trees}) \sum \hat{y}_t(x)$

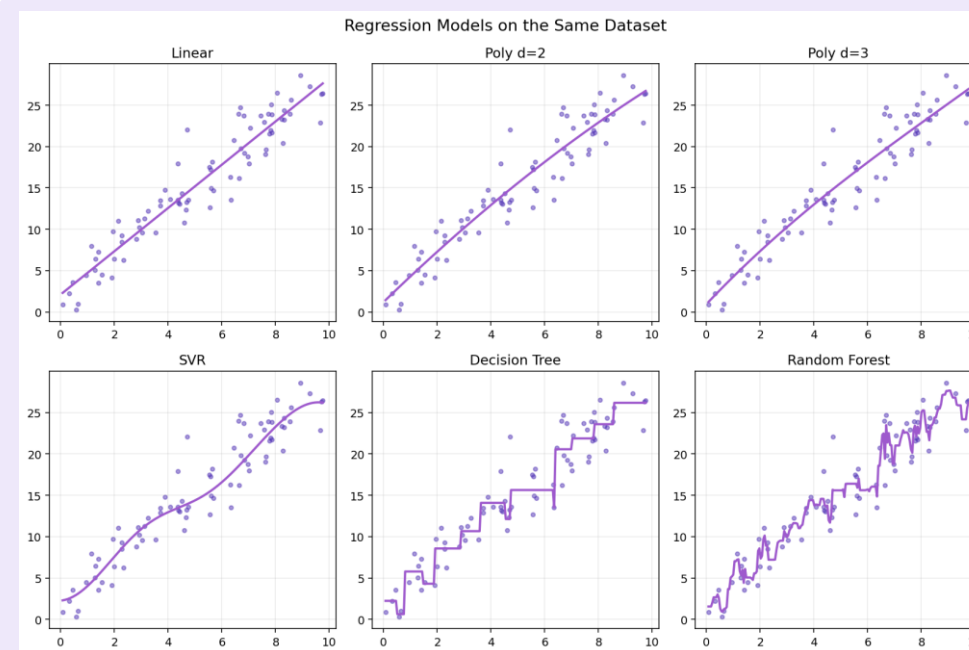
If interpretability is priority: Linear Regression or Decision Tree. If predictive performance is priority: Random Forest.

Visual Comparison of Regression Models

Same Dataset · Different Fits

Same data · five model families side by side.

- Linear/Ridge: fast · watch curves.
- Tree/RF: flexible · weak extrapolation.



Visual output from Python code.

Model Families at a Glance

Session Closing

Model	Strength	Watch Out
Linear / Ridge	Interpretable, fast	Nonlinear patterns
SVR	Margin-based, kernels	Slow on huge data
Tree / RF	Nonlinear, little scaling	Extrapolation

Visual output from Python code is shown on the previous slide.

Week 1 · Session 3

Classical ML

Classification basics — from labels to decision boundaries

Where Are We in the Bootcamp?

After regression — learn models that predict class labels.

Week 1

- S1 Foundations & Preprocessing
- S2 Regression Models
- S3 Classification Basics**
- S4 Naive Bayes & Trees
- S5 SVM & Kernels
- S6 Clustering & PCA

Week 2

- S7 Deep Learning

Week 3

- S8 NLP Fundamentals
- S9 Tokenization
- S10 Text Analysis & NER
- S11 Language Modeling
- S12 Embeddings & RNNs
- S13 Seq2Seq & NMT

Week 4

- S14 Generative AI
- S15 RAG Systems
- S16 MLOps

Week 1 · Session 3

Classical ML

Classification Basics

Classification Basics

Logistic regression

K-NN

Decision boundaries

Threshold tuning

Classification Basics

What we will cover in this session

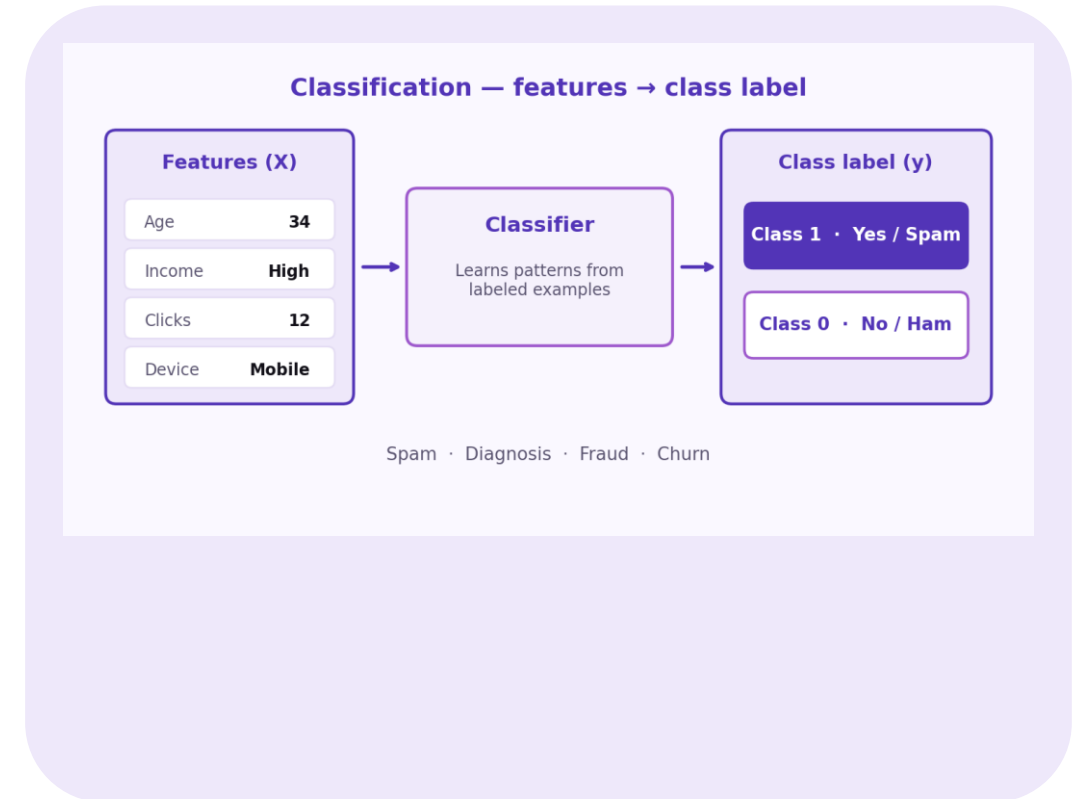
- Classification Basics
- Logistic regression
- K-NN
- Decision boundaries
- Threshold tuning

Classification Overview

Classification Basics

Classification is a supervised learning task used to assign a new observation to a predefined category based on learned patterns from labeled data.

- Output is a class label (e.g., 0/1, Yes/No).
- Used in spam, diagnosis, fraud, and churn prediction.



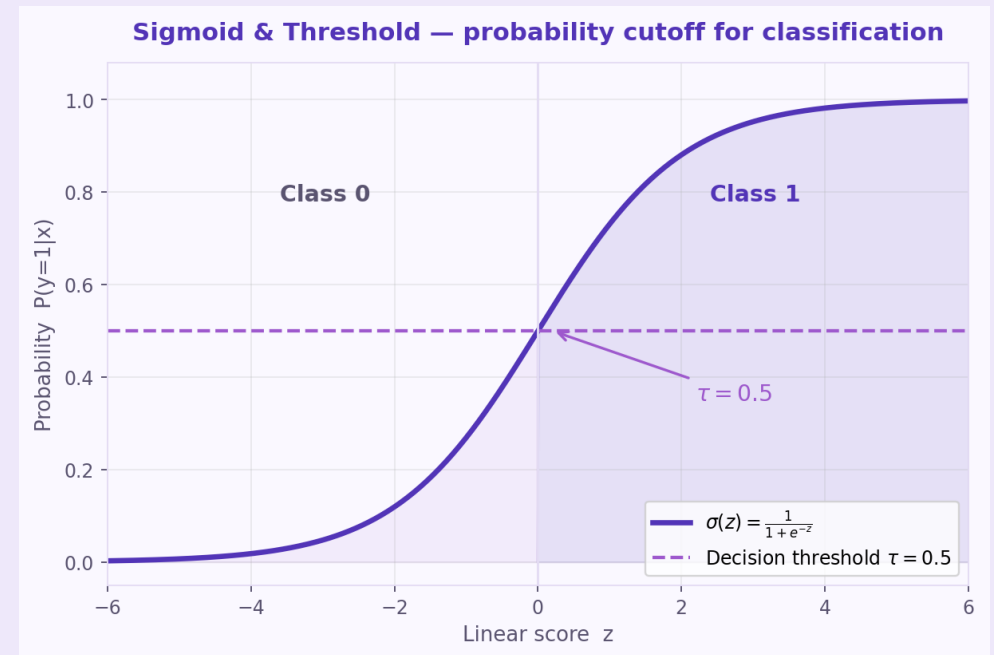
Features in → class label out. Probability models come next.

Logistic Regression: Definition

Binary Classification

Supervised binary classifier that estimates $P(y=1|x)$ from features.

- Outputs probabilities in $[0, 1]$.
- Class label comes from applying a threshold later.
- Models features $\rightarrow$ log-odds of the positive class.



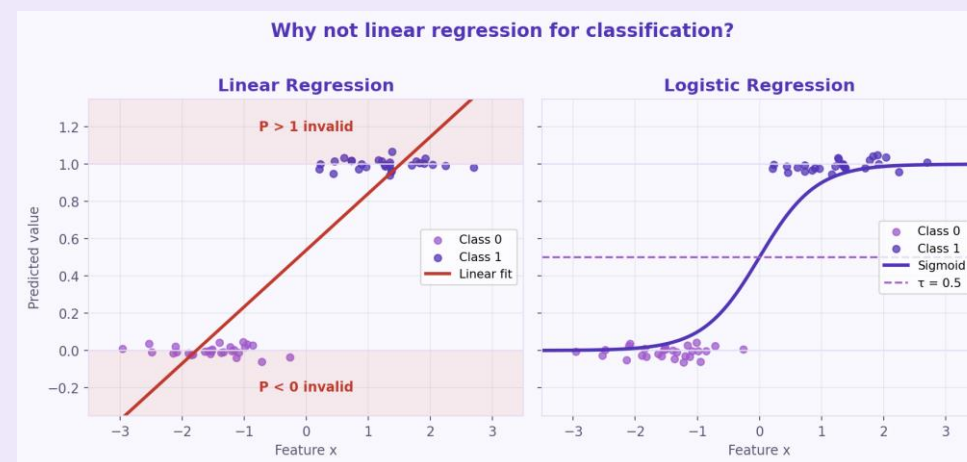
Maps a linear score to a probability via the sigmoid.

Why Not Linear Regression for Classification?

Motivation for Logistic Regression

Linear regression predicts continuous values rather than discrete classes.

- Predictions can fall outside $[0, 1]$ — invalid as probabilities.
- No natural, stable class boundary for labels.



Sigmoid keeps outputs in $[0, 1]$ and supports a clear cutoff.

Sigmoid Mapping and Decision Threshold

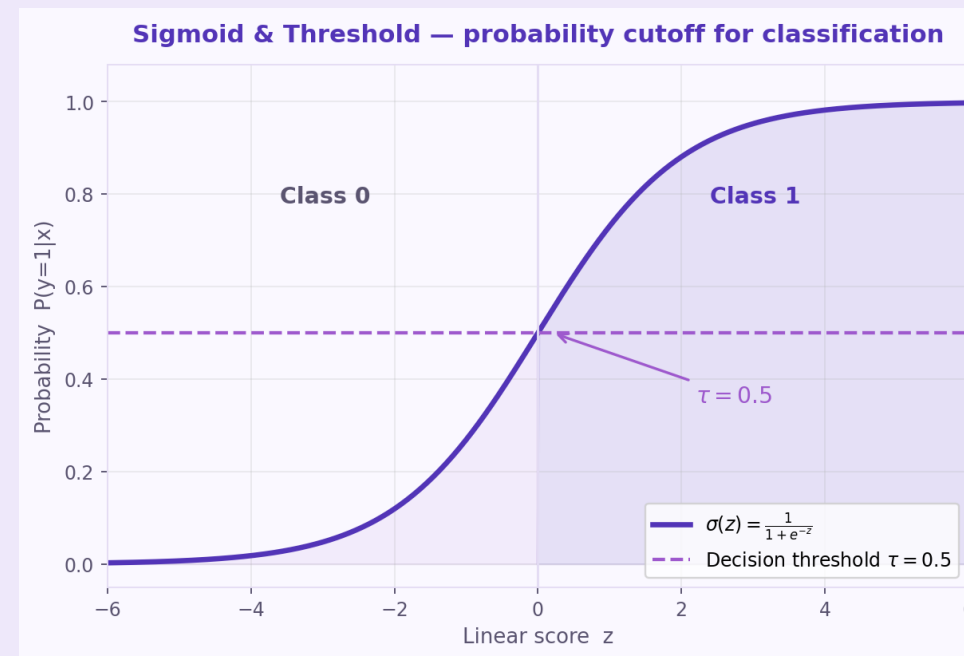
Sigmoid & Threshold

Sigmoid turns the linear score z into a probability.

Formula

$$P(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta^\top x$$

- Default cutoff: $P \geq 0.5 \rightarrow$ class 1, else class 0.
- τ is tunable for cost of false positives vs false negatives.



Tune τ to business/clinical risk — covered in Threshold tuning.

Key Assumptions for Logistic Regression

Model Assumptions

Works on log-odds — check independence, collinearity, and linearity in logit space.

$$\text{Odds} = \frac{p}{1-p}, \quad \text{Logit}(p) = \log\left(\frac{p}{1-p}\right)$$

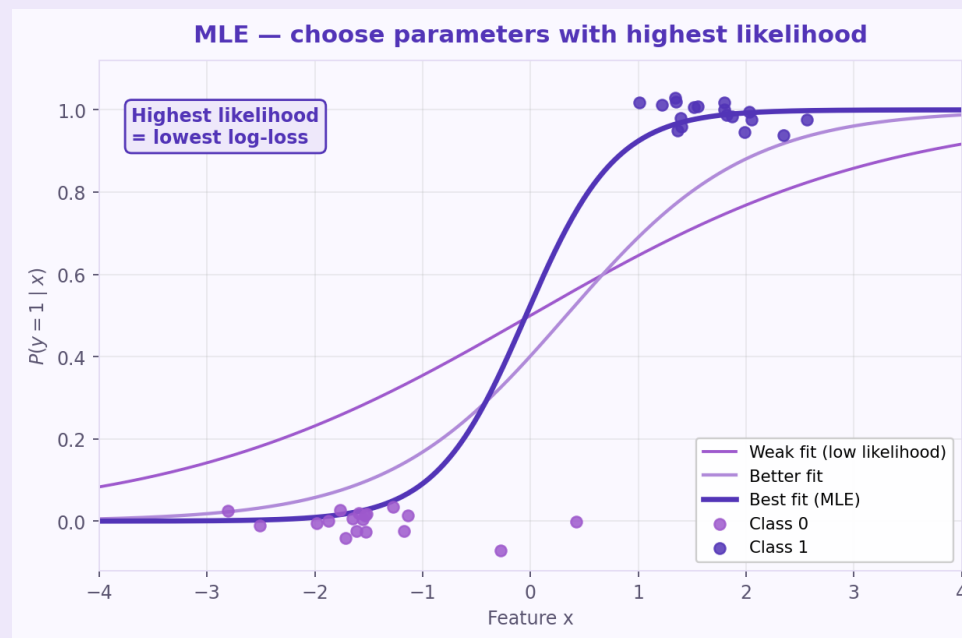
- Observations should be independent.
- Avoid severe multicollinearity among predictors.
- Relationship $\approx$ linear in the log-odds space.

Maximum Likelihood Estimation (MLE)

How Logistic Regression Learns

Parameters β are chosen to maximize the likelihood of the observed labels.

- Each curve is a different β (different likelihood).
- Best fit: highest likelihood / lowest log-loss.



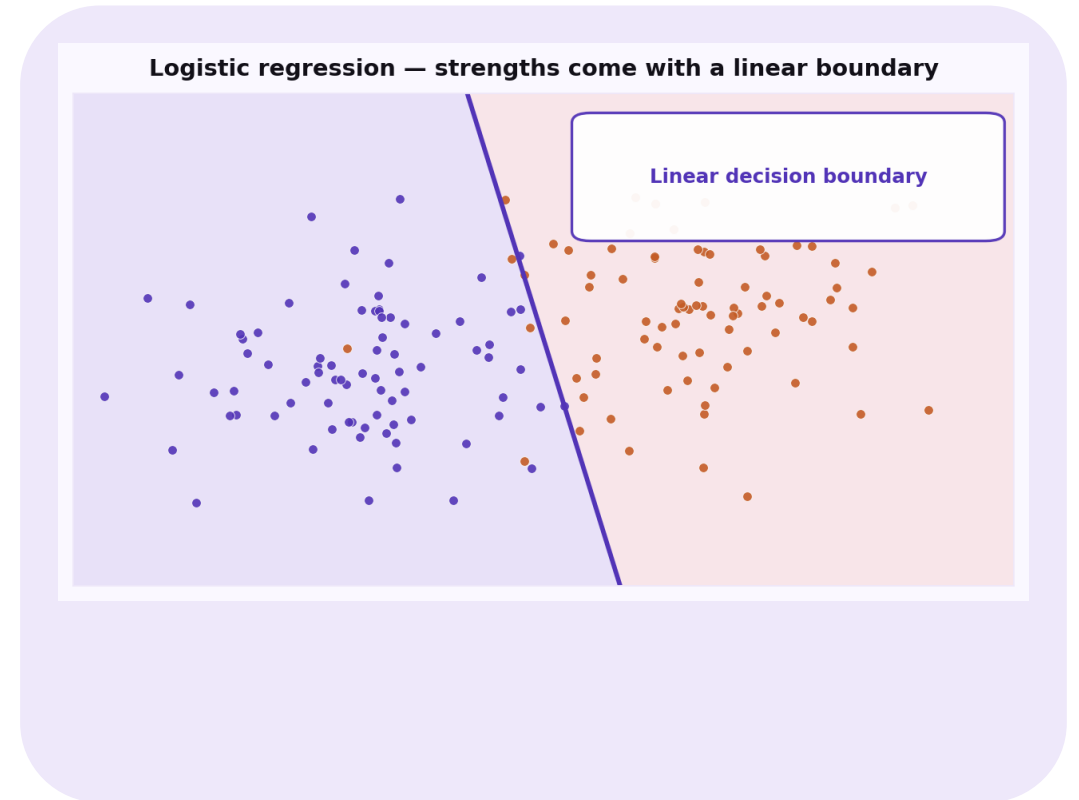
Pick β that makes the observed labels most probable.

Logistic Regression: Strengths and Limits

When to Use It

Strong baseline when classes are roughly linearly separable; weaker on complex shapes.

- Strengths: interpretable odds, fast training, calibrated probabilities.
- Limits: linear boundary; struggles with overlap / non-linear shapes.



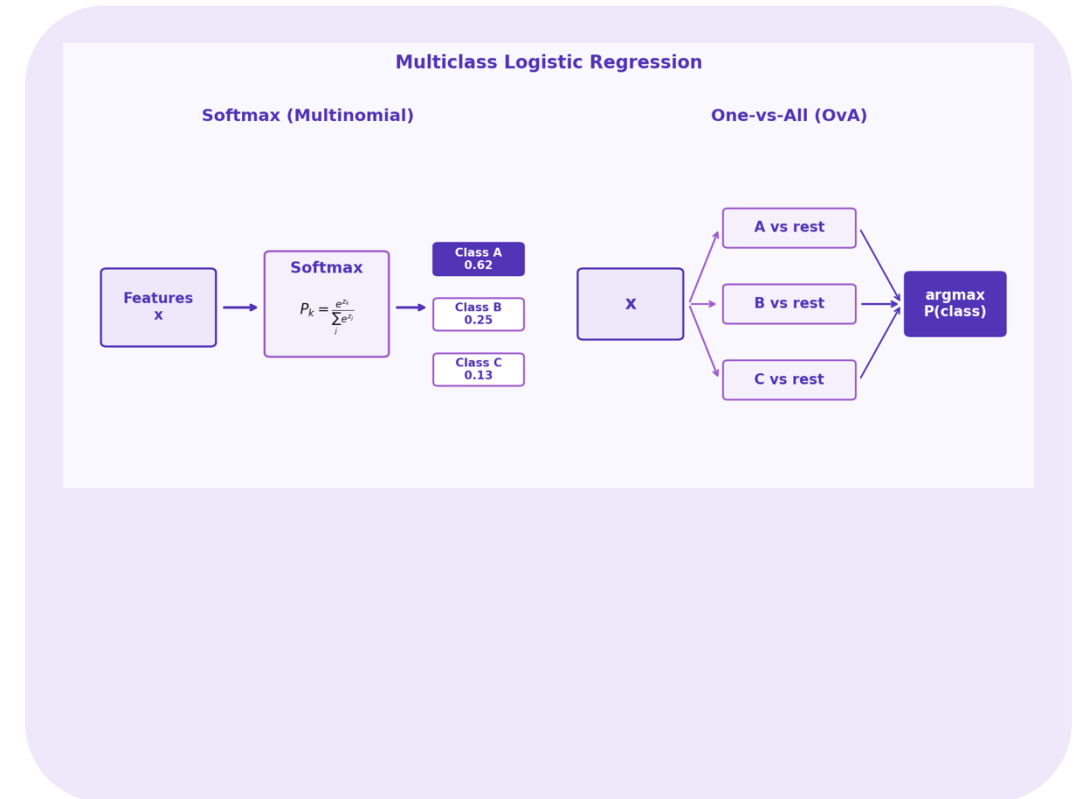
For wiggly boundaries → polynomials, K-NN, or kernels.

Multiclass Extension of Logistic Regression

Beyond Binary

Extend binary logistic regression to 3+ classes with Softmax or One-vs-All.

- Softmax: one multinomial model for all classes.
- One-vs-All: one binary classifier per class; pick max P.



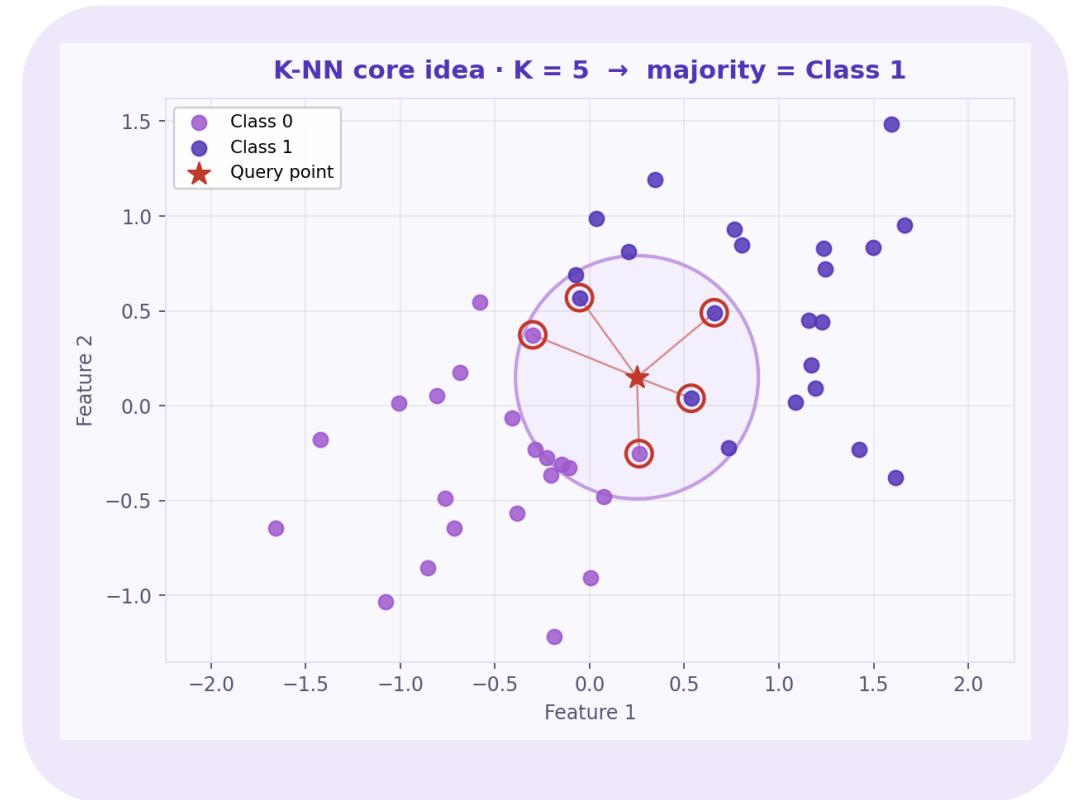
Final label = $\text{argmax}_k P(y=k|x)$.

K-Nearest Neighbors (K-NN): Core Idea

Instance-Based Learning

Non-parametric classifier: label by majority vote among the K nearest training points.

- Choose K and a distance metric.
- Find K nearest neighbors; majority vote wins.



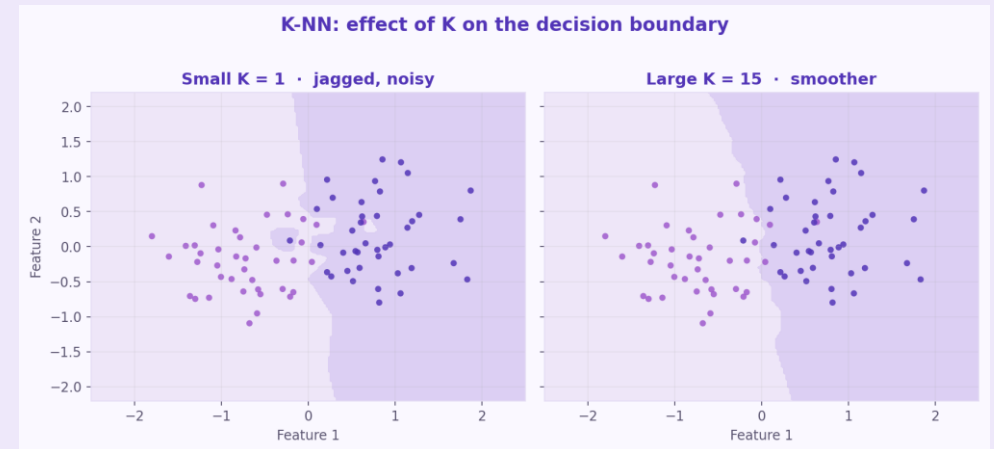
Lazy learner — the training set is the model.

K-NN: Choosing K and Distance Metric

Hyperparameters

K and distance control how smooth the decision boundary is.

- Small K: flexible, noise-sensitive.
- Large K: smoother; may miss local structure.



Pick K with CV · scale features before distance modeling.

K-NN: Strengths and Limitations

When to Use It

Strengths	Limitations
Simple and intuitive	Slow prediction on large datasets
Works well on small, separated data	Sensitive to irrelevant features / imbalance
Native multiclass support	Curse of dimensionality
No training phase (lazy baseline)	Needs feature scaling

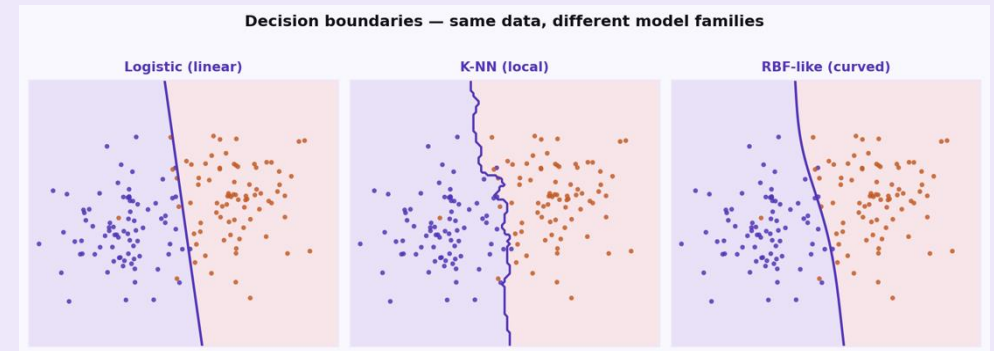
Great baseline; watch cost at inference and high dimensions.

Decision Boundaries Across Models

Geometry of Classification

A decision boundary is the surface where the predicted class flips — its shape depends on the model family.

- Logistic: linear boundary in feature space (unless you add features).
- K-NN: local, piecewise boundary that follows nearest neighbors.
- Kernel / RBF-like models: smooth curved boundaries.



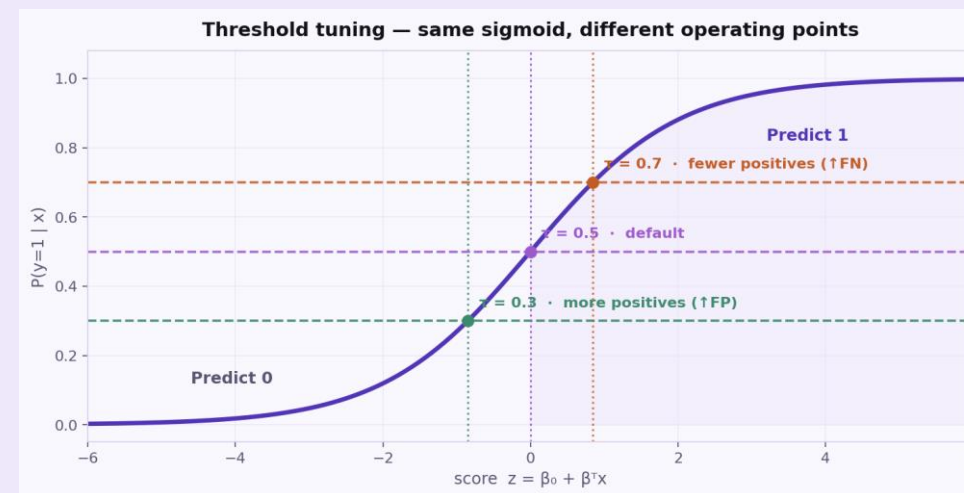
Match model flexibility to how classes are separated in the data.

Threshold Tuning: Trading FP vs FN

Operating Point

The sigmoid gives $P(y=1|x)$; τ turns that probability into a hard class label.

- Lower $\tau \rightarrow$ more positives ($\uparrow$ recall, $\uparrow$ false positives).
- Higher $\tau \rightarrow$ fewer positives ($\uparrow$ precision, $\uparrow$ false negatives).
- Default $\tau = 0.5$ is a starting point, not a law.



Set τ from costs: missed fraud $\neq$ false spam alarm.

Week 1 · Session 4

Classical ML

Naive Bayes, trees, and evaluation metrics

Where Are We in the Bootcamp?

After classification basics — probabilistic models, trees, and metrics.

Week 1

- S1 Foundations & Preprocessing
- S2 Regression Models
- S3 Classification Basics
- S4 Naive Bayes & Trees**
- S5 SVM & Kernels
- S6 Clustering & PCA

Week 2

- S7 Deep Learning

Week 3

- S8 NLP Fundamentals
- S9 Tokenization
- S10 Text Analysis & NER
- S11 Language Modeling
- S12 Embeddings & RNNs
- S13 Seq2Seq & NMT

Week 4

- S14 Generative AI
- S15 RAG Systems
- S16 MLOps

Week 1 · Session 4

Classical ML

Naive Bayes, Trees, Evaluation

Naive Bayes

Decision trees

Random Forest

Precision / recall / F1

Naive Bayes, Trees, Evaluation

What we will cover in this session

- Naive Bayes
- Decision trees
- Random Forest
- Precision / recall / F1

Naive Bayes: Bayes' Theorem

Core Probability Rule

Bayes' theorem updates our belief about A after observing evidence B.

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

Posterior $\propto$ Likelihood $\times$ Prior

- $P(A|B)$: posterior probability after observing evidence B.
- $P(A)$: prior belief before seeing evidence.
- $P(B|A)$: likelihood of evidence if class A is true.

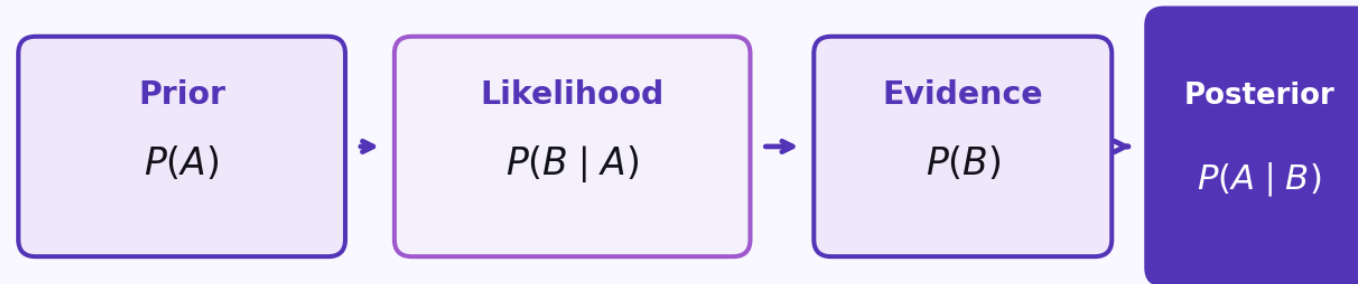
$P(B)$: evidence probability (normalization term). This theorem updates beliefs when new evidence appears.

Bayes' Theorem — Visual Map

Core Probability Rule

Prior × Likelihood → Posterior (normalized by evidence).

Bayes' Theorem — update beliefs with evidence



$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

Posterior $\propto$ Likelihood $\times$ Prior

— Same rule as the formula slide — different view of the update.

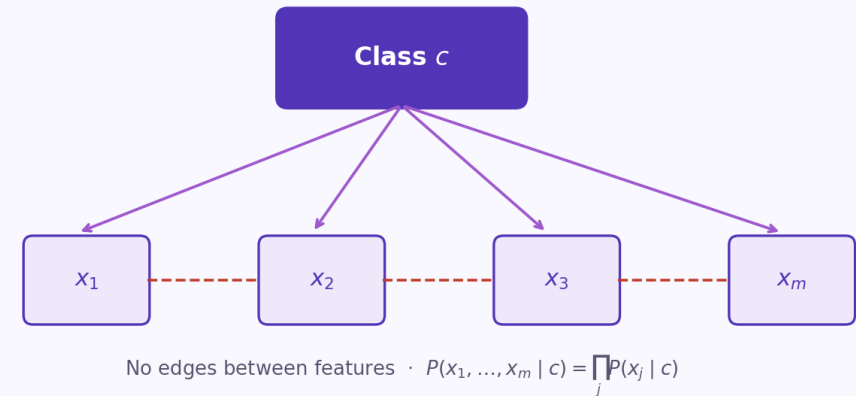
Why Is It Called 'Naive'?

Conditional Independence Assumption

Naive Bayes assumes features are conditionally independent given the class label.

- Simplifies parameter estimation and speeds up training.
- Works well in text classification and high-dimensional sparse data.
- Can still perform well even when independence is not fully true.

Naive assumption: features independent given the class



When comparing classes for the same X , $P(X)$ is constant and can be ignored.

Naive Bayes: Practical Notes

Variants & Implementation

Variant	Typical Data	Key Idea
Gaussian NB	Continuous	Each feature $\sim$ Gaussian given the class
Multinomial NB	Count-based	Common for document word counts
Bernoulli NB	Binary	Presence/absence features

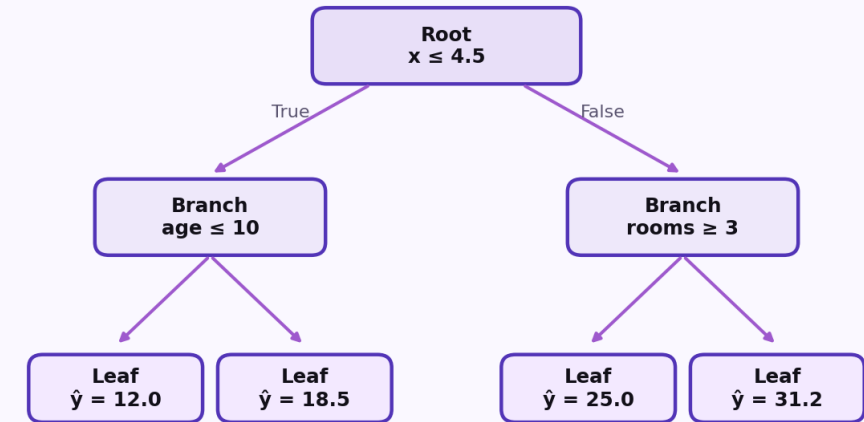
Handle zero-frequency with smoothing (e.g., Laplace). Scale/preprocess by feature type and variant.

Decision Tree Classification (CART)

Recursive Partitioning

Decision Tree Classification predicts class labels by recursively splitting feature space into purer class regions.

- Internal nodes represent feature-based decisions.
- Branches represent decision outcomes.
- Leaf nodes output final class prediction.



Recursive partitioning by feature threshold · Root → branches → leaf nodes

Decision Tree Split · Root → branches → leaf nodes · split by feature threshold.

How Splits Are Chosen in Classification Trees

Gini · Entropy · Information Gain

Splits are chosen greedily to maximize information gain.

$$\text{Gain} = I(\text{parent}) - \sum_k \frac{|S_k|}{|S|} I(S_k)$$

Gini / Entropy measure impurity — lower is better

- Choose the split that best separates classes at each node.
- Stop splitting using rules like max depth or minimum samples.
- Gini impurity and entropy measure node purity — lower is better.

Better splits reduce impurity and increase class purity in child nodes.

Decision Tree: Classification vs Regression

Same Structure · Different Objectives

Aspect	Classification Tree	Regression Tree
Target	Categorical class	Continuous value
Split criterion	Gini / Entropy	MSE / MAE
Leaf output	Class label or class probability	Numeric mean/median

Both use recursive partitioning (Root → branches → leaves) but optimize different objectives.

Gini Impurity and Entropy

Node Purity Measures

Gini (CART default) and entropy both measure node impurity — lower is better.

Node purity measures — lower is better

Gini Impurity

CART default

$$\text{Gini}(S) = 1 - \sum_{k=1}^K p_k^2$$

p_k = share of class k in node S

Entropy

Information Gain

$$\text{Entropy}(S) = - \sum_{k=1}^K p_k \log_2 p_k$$

Used to score how mixed the labels are

— Better splits reduce impurity and increase class purity in child nodes.

Random Forest Classification

Ensemble Learning by Bagging

Train many trees on bootstrap samples, then combine predictions by majority vote.

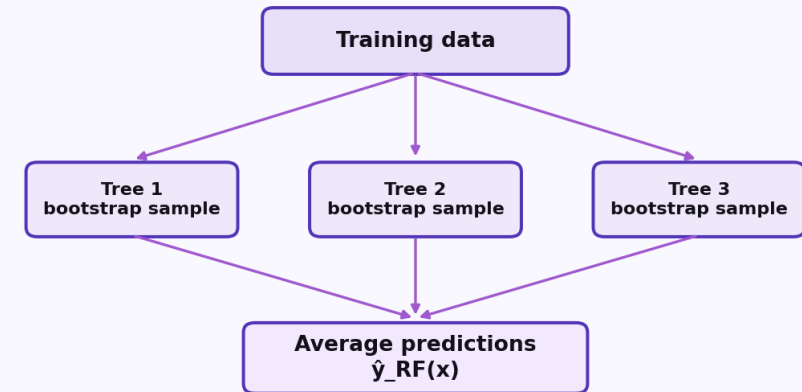
Formula

$$\hat{y}_{RF}(x) = \text{mode} \{ \hat{y}_1(x), \hat{y}_2(x), \dots, \hat{y}_T(x) \}$$

Majority vote across T trees

- Sample bootstrap data and train many trees.
- Each tree predicts a class for the new sample.

Final class = majority vote (mode). Regression RF uses the mean instead.



Many trees · different samples/features · average = more stable fit

Why Random Forest Often Outperforms a Single Tree

Bias–Variance Tradeoff

Property	Single Decision Tree	Random Forest
Variance	High	Lower (averaging effect)
Overfitting risk	Higher	Lower
Interpretability	High	Moderate
Predictive robustness	Sensitive to data noise	More stable

Bagging + feature randomness → less variance without much extra bias.

Classification Errors: FP and FN

Error Types

Not all mistakes are equal — False Positives and False Negatives carry different costs.

- False Positive (Type I): predict positive, actual is negative.
- False Negative (Type II): predict negative, actual is positive.
- Error impact depends on domain costs (healthcare vs spam).

Confusion outcomes — focus on the two error types

		Predicted	
		Positive	Negative
Actual	Positive	TP Correct +	FN Type II error
	Negative	FP Type I error	TN Correct –

FP: false alarm · FN: missed detection

Model evaluation should consider error trade-offs, not accuracy alone.

Confusion Matrix and Accuracy

Actual vs Predicted Counts

Confusion matrix counts TP, FP, FN, TN — Accuracy is overall correctness.

Formula

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Correct predictions ÷ all predictions

- TP / TN: correct positives / negatives.
- FP / FN: false alarms / missed detections.

High accuracy can hide poor minority-class performance when data is imbalanced.

		Predicted	
		Pos	Neg
Actual	Pos	TP	FN
	Neg	FP	TN

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Confusion Matrix Cells

Metric Meanings

Metric	Meaning
TP	Correctly predicted positives
TN	Correctly predicted negatives
FP	Incorrectly predicted positives
FN	Incorrectly predicted negatives

These four counts are the building blocks of precision, recall, and F1.

Precision, Recall, and F1-Score

Beyond Accuracy

Precision and recall trade off · F1 balances both when classes are imbalanced.

Precision · Recall · F1

Precision

Of predicted positives,
how many are truly positive?

Recall

Of actual positives,
how many were detected?

F1-score

Harmonic mean balancing
precision and recall.

Formulas

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

— Use F1 especially when class distribution is imbalanced.

Week 1 · Session 5

Classical ML

SVM and kernel methods for classification

Where Are We in the Bootcamp?

After trees and metrics — maximum-margin classifiers and kernels.

Week 1

- S1 Foundations & Preprocessing
- S2 Regression Models
- S3 Classification Basics
- S4 Naive Bayes & Trees
- S5 SVM & Kernels**
- S6 Clustering & PCA

Week 2

- S7 Deep Learning

Week 3

- S8 NLP Fundamentals
- S9 Tokenization
- S10 Text Analysis & NER
- S11 Language Modeling
- S12 Embeddings & RNNs
- S13 Seq2Seq & NMT

Week 4

- S14 Generative AI
- S15 RAG Systems
- S16 MLOps

Week 1 · Session 5

Classical ML

SVM · Kernel Methods

Margin maximization

Kernel trick

Support vectors

Linear vs RBF

SVM · Kernel Methods

What we will cover in this session

- Margin maximization
- Kernel trick
- Support vectors
- Linear vs RBF

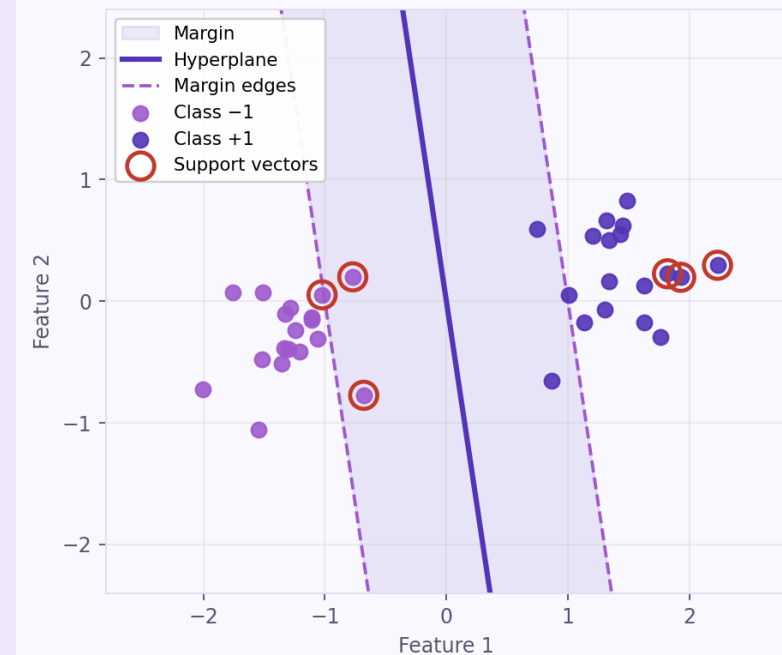
Support Vector Machine (SVM): Core Idea

Maximum Margin Classifier

SVM is a supervised method that finds the best separating boundary (hyperplane) between classes.

- It maximizes the margin between classes.
- Only support vectors (closest points) determine the boundary.
- A larger margin usually improves generalization.

SVM Maximum Margin — widest gap between classes



SVM Maximum Margin · widest gap between classes · support vectors define the margin.

How SVM Classification Works

Decision Function

New points are classified by the side of the hyperplane they fall on.

$$f(x) = w^T x + b, \quad \hat{y} = \text{sign}(f(x))$$

$$f(x) > 0 \rightarrow \text{class } +1 \cdot f(x) < 0 \rightarrow \text{class } -1$$

- Hard margin: no misclassification allowed (strict separation).
- Soft margin: allows violations for better robustness with overlap/noise.

Soft-margin SVM adds penalty C to control the margin–violation tradeoff.

Why SVM Is Powerful

When SVM Shines

Capability	Why It Matters
Maximum-margin principle	Better robustness to small perturbations
Kernel support	Handles non-linear class boundaries
High-dimensional performance	Works well in text and sparse feature spaces

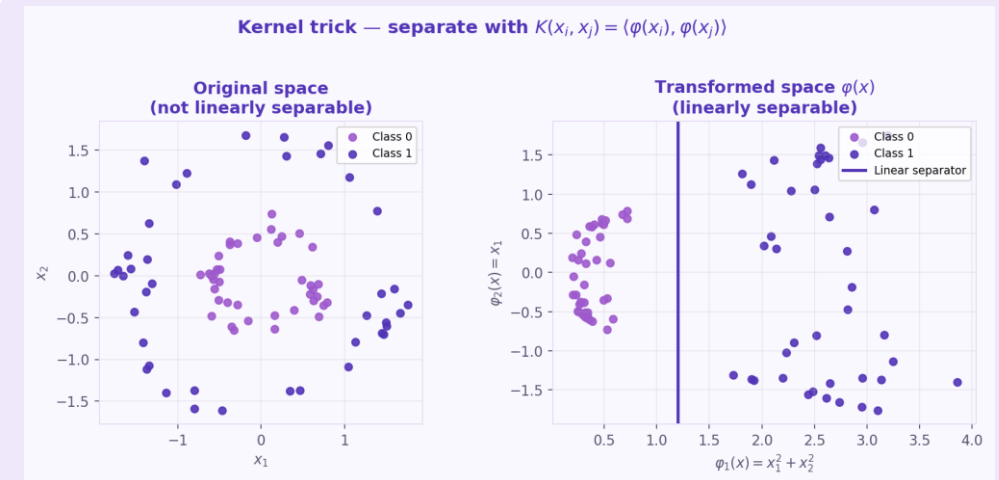
Common in text/image features · often effective on small-to-medium datasets. Support vectors define the margin.

Kernel Trick: From Non-Linear to Linear Separation

Feature Mapping Without Explicit $\phi(x)$

When data is not linearly separable in original space, SVM uses kernel functions to separate it in a transformed feature space.

- Avoids explicit high-dimensional mapping in many cases.
- Computes similarity using kernel function $K(x_i, x_j)$.

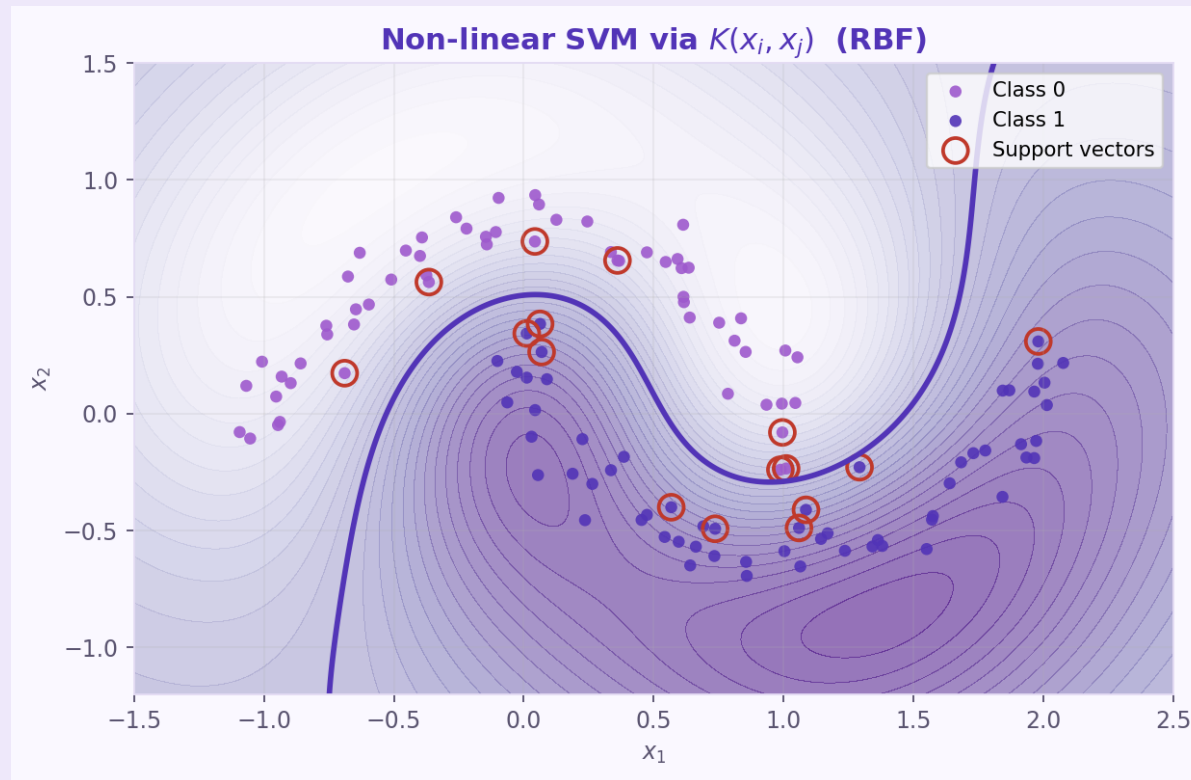


Work with $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$ instead of building $\phi(x)$ by hand.

Kernel Similarity $K(x_i, x_j)$

Kernel Trick

Avoids explicit high-dimensional mapping · computes similarity with $K(x_i, x_j)$.



—SVM decision depends on kernel similarities to support vectors — not on building $\phi(x)$.

RBF Kernel in SVM

Gaussian Similarity

RBF measures how close x is to a landmark l_i — a common default for non-linear SVM.

$$K(x, l_i) = \exp\left(-\frac{\|x - l_i\|^2}{2\sigma^2}\right)$$

Close to $l_i \rightarrow K \approx 1$ · far $\rightarrow K \approx 0$

- If x is close to landmark l_i , similarity is near 1.
- If x is far, similarity approaches 0.
- RBF is a common default for non-linear SVM.

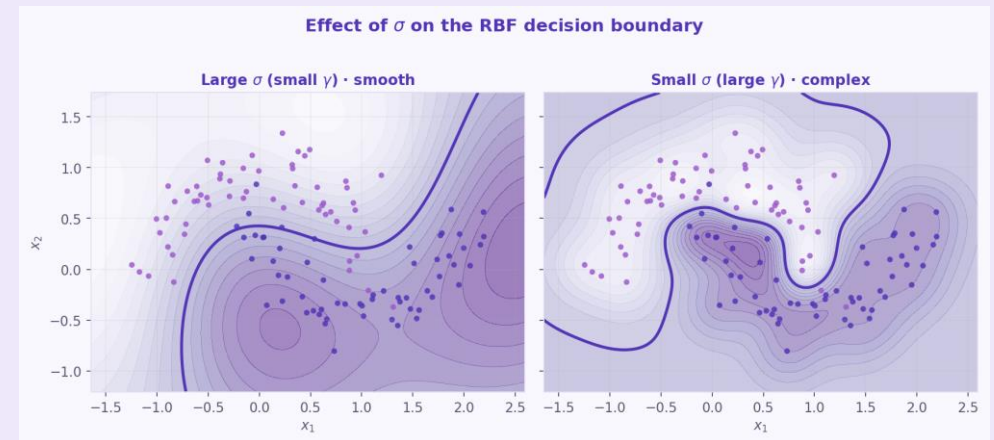
Effect of σ : large $\rightarrow$ smoother, wider boundary (higher bias) · small $\rightarrow$ tighter, complex boundary (higher variance).

Effect of Sigma (σ) in RBF

Bias–Variance of the Kernel Width

σ controls how far a landmark's influence reaches.

- Large sigma: smoother, wider decision boundary (higher bias).
- Small sigma: tighter, complex boundary (higher variance / overfitting risk).



Tune σ (or gamma) with cross-validation together with C.

Common SVM Kernels

Choosing $K(x_i, x_j)$

Kernel	Typical Use	Notes
Linear	High-dimensional sparse data	Fast and interpretable margin
RBF (Gaussian)	General non-linear patterns	Strong baseline in many tasks
Polynomial	Polynomial-like interactions	Degree controls complexity
Sigmoid	Neural-style boundary behavior	Less common in practice

Start with Linear (text/sparse) or RBF (general). Support vectors still define the margin.

When to Use SVM / When Not

Practical Guidance

Use SVM When	Avoid SVM When
Data is high-dimensional or moderately sized	Dataset is very large (training can be expensive)
Classes are reasonably separable	Data is extremely noisy with weak class structure
You need non-linear boundaries via kernels	Kernel matrix becomes too costly (features × samples)

Use: medium data, clear margin, high-dim text · Avoid: massive data, heavy noise without tuning.

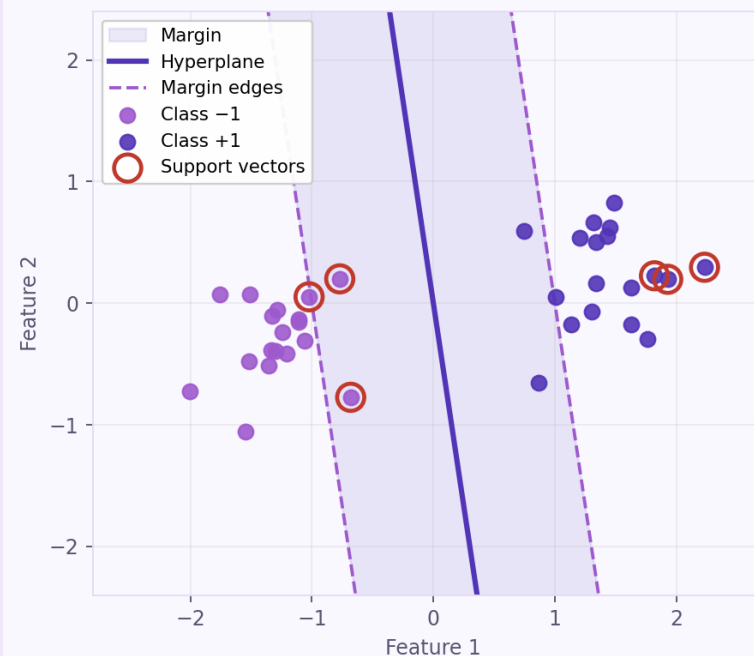
Support Vectors — Why They Matter

Sparse Decision Boundary

Only the points on (or inside) the margin shape the SVM — the rest can be removed without changing the boundary.

- Support vectors define the margin edges.
- Decision depends on a sparse subset of training points.
- Removing non-support points usually leaves the model unchanged.

SVM Maximum Margin — widest gap between classes



Wider margin · fewer influential points · often better generalization.

SVM vs Logistic Regression vs K-NN

Model Comparison

Aspect	SVM	Logistic Regression	K-NN
Primary task	Classification (and SVR)	Classification	Classification
Non-linearity	Yes, with kernels	Limited in linear form	Yes (distance-based)
Compute profile	Can be heavy on large data	Usually efficient	Heavy at prediction
Best for	Complex margins, high-dim spaces	Fast interpretable baseline	Local neighborhood patterns

LogReg: probabilities + $\tau=0.5$ · SVM: max-margin · K-NN: local votes.

SVM Practical Hyperparameters

What to Tune

Parameter	Role	Typical Tuning Direction
C	Penalty for margin violations	Higher C → stricter fit · lower C → smoother margin
Kernel	Similarity function	Start with RBF · compare linear/polynomial
gamma (RBF)	Locality of influence	Higher gamma → tighter boundary
degree (poly)	Polynomial complexity	Increase only when needed

Scale features first · tune with CV · support vectors define the margin.

Week 2 · Session 1

Deep Learning

Neural networks, backpropagation, CNNs, and RNNs

Where Are We in the Bootcamp?

Week 1 classical ML is done — now stacked nonlinear models.

Week 1

- S1 Foundations & Preprocessing
- S2 Regression Models
- S3 Classification Basics
- S4 Naive Bayes & Trees
- S5 SVM & Kernels
- S6 Clustering & PCA

Week 2

- S7 **Deep Learning**

Week 3

- S8 NLP Fundamentals
- S9 Tokenization
- S10 Text Analysis & NER
- S11 Language Modeling
- S12 Embeddings & RNNs
- S13 Seq2Seq & NMT

Week 4

- S14 Generative AI
- S15 RAG Systems
- S16 MLOps

Week 2 · Session 1

Deep Learning

Neural Networks · CNNs · RNNs

Deep Learning

Neural Networks

Perceptron & MLP

Backpropagation

CNNs

RNNs & regularization

Neural Networks · CNNs · RNNs

What we will cover in this session

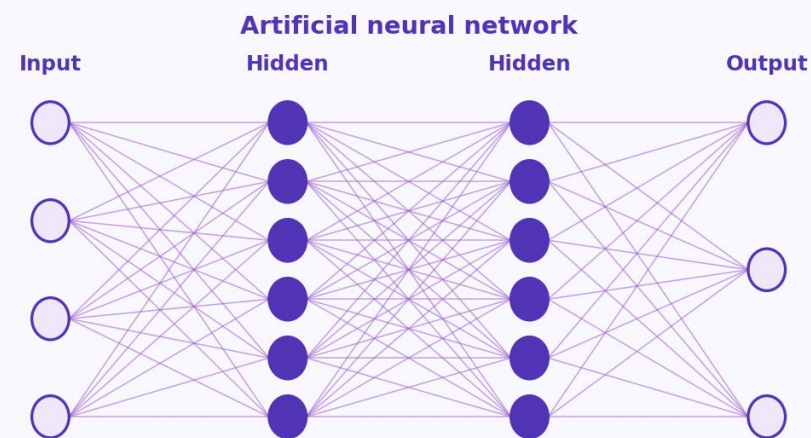
- Deep Learning
- Neural Networks
- Perceptron & MLP
- Backpropagation
- CNNs
- RNNs & regularization

Deep Learning: Introduction

Foundations → Optimization → Architectures → Autoencoders

Each phase answers a different question.

- Four phases: foundations → optimization → architectures → autoencoders.
- Each phase has a lab-style checkpoint — pause at phase boundaries.
- Focus on deep learning basics — phase breaks are intentional.



Phase 1 focus: understand the “why” and the mathematical mechanics of a single neuron before scaling to layers and data.

Deep Learning: Introduction

Four phases, four questions

Foundations → optimization → architectures → autoencoders. Pause at each boundary.

Four phases of this Deep Learning block

1 Foundations

Why depth?
What does a
neuron compute?

2 Optimization

How does training
stay stable and
generalize?

3 Architectures

Images vs
sequences —
which structure?

4 Autoencoders

Generation and
reconstruction

—Phase 1 focus: understand the “why” and the mathematical mechanics of a single neuron before scaling to layers and data.

Phase 1: Foundations & Neural Core

Why depth, neurons, and forward pass

Goal: understand what a neuron computes and why stacking layers enables hierarchical features.

$$z = w \cdot x + b$$

Exit check: explain this — and why nonlinearity is required between layers.

- Topics: perceptron, activations, MLP, forward propagation, and loss functions.
- Stacking layers builds hierarchical features only if a nonlinearity sits between them.

Phase 1 Roadmap

Why depth, neurons, and forward pass

Block	Question it answers
Perceptron & MLP	What does one layer compute?
Forward pass	How does inference flow through the graph?
Loss & gradients	What are we optimizing?

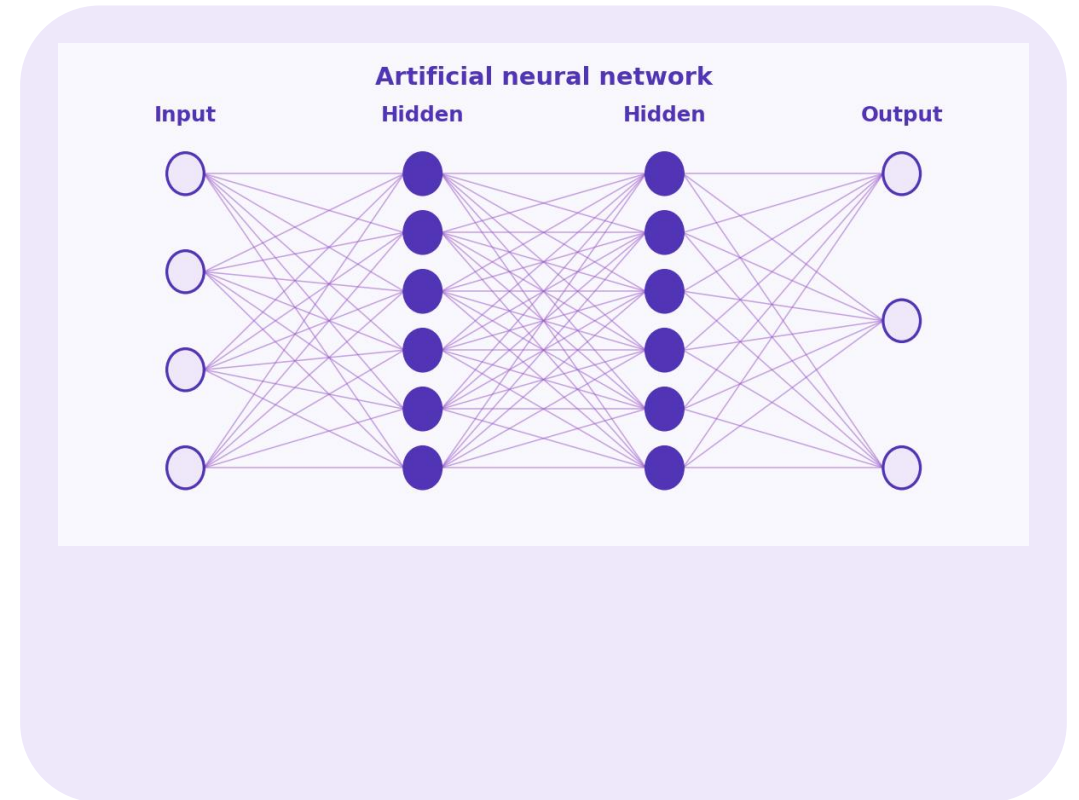
Exit check: explain $z = w \cdot x + b$ and why nonlinearity is required between layers.

What Is Deep Learning?

Definition in One Sentence — Then the Details

Deep learning learns a parameterized mapping from inputs to outputs by stacking nonlinear transformations (layers).

- “Deep” means multiple such transformations — hierarchical features, not one giant linear rule.



Layers rewrite features step by step. Depth is composition, not one giant linear rule.

What Is Deep Learning?

Definition in One Sentence — Then the Details

- Training = pick a loss that measures mistakes, then adjust weights with gradients so average loss drops on data (generalization to unseen data is the real goal).
- Unlike fixed pipelines of hand-crafted features, the network learns representations that are tuned to the task (edges → textures → parts → objects in vision, etc.).

Generalization to unseen data is the real goal — not memorizing the training set.

Three Ingredients You Always Have

Architecture · Loss · Optimizer + data

Ingredient	What it means
Architecture	How signals flow (MLP, CNN, RNN, Transformer, ...)
Loss / objective	What “better” means for your task (MSE, cross-entropy, ...)
Optimizer + data	How you search parameters and what examples you show

You always have these three. Change one, and the model’s behavior changes with it.

Three Ingredients You Always Have

Architecture · Loss · Optimizer + data

Three ingredients of every deep model

1 Architecture

How signals flow
MLP · CNN · RNN

2 Loss

What “better” means
MSE · cross-entropy

3 Optimizer + data

How we search weights
SGD · Adam · batches

—A strong architecture with the wrong loss still learns the wrong thing.

AI, Machine Learning, and Deep Learning

Subset Relationships — Not Interchangeable Labels

- Artificial intelligence (AI): the broad field—rules, search, optimization, learning, planning—anything that aims at intelligent behavior.
- Machine learning (ML): a subfield of AI where behavior improves from data via a learning algorithm (not only hand-written rules).
- Deep learning (DL): a subfield of ML using deep neural networks—multiple learned nonlinear layers; representation learning is central.

AI, Machine Learning, and Deep Learning

Subset Relationships — Not Interchangeable Labels

DL $\subset$ ML $\subset$ AI

AI

Intelligent systems · search · rules · learning

Machine Learning

Learn from data · linear models · trees · kernels

Deep Learning

Stacked nonlinear layers · CNNs · RNNs · Transformers

Containment (conceptual)

Subset Relationships — Not Interchangeable Labels

Set	Contains
AI	Rule systems, classical search, ML, knowledge bases, ...
ML	Linear models, trees, kernel SVMs, shallow nets, deep nets
DL	CNNs, RNNs/GRU/LSTM, Transformers, GANs, VAEs, ...

Saying “we used AI” is vague; “we used supervised DL with a CNN” is precise.

Use-Case Fit: When Deep Learning Shines — and When to Skip It

Match Method to Data, Risk, and Budget

- DL is strongest when: large data (or strong pre-training), high-dimensional raw inputs (images, text, speech), and compositionality you want learned end-to-end.
- Think twice when: tiny tabular datasets, strict interpretability requirements, tight latency on CPU without optimization, or simple baselines already solve the problem.
- Always compare against strong baselines (logistic regression, gradient boosting, k-NN on engineered features)—DL must earn its complexity on validation metrics you care about.

Use-Case Fit: When Deep Learning Shines — and When to Skip It

Match Method to Data, Risk, and Budget

Match method to data, risk, and budget



DL shines

Images · text · speech
Large data or pretraining
Hierarchical composition



Skip or delay DL

Tiny tabular sets
Need legal explanation
Simple baseline already wins

Use-Case Fit: When Deep Learning Shines — and When to Skip It

Match Method to Data, Risk, and Budget

Situation	Typical first choice	Why
Large labeled images / audio / text	Pre-trained DL + fine-tune	Raw inputs; hierarchical features
Small tabular (< few k rows)	Trees / linear + regularization	Lower variance, fast iteration, interpretability
Need causal or legal explanation	Simpler model or hybrid	Auditability and stability
Plenty of GPUs + big data	Larger nets + careful tuning	Capacity + regularization can scale
Edge device, millisecond latency	Small distilled / quantized models	Memory and power limits

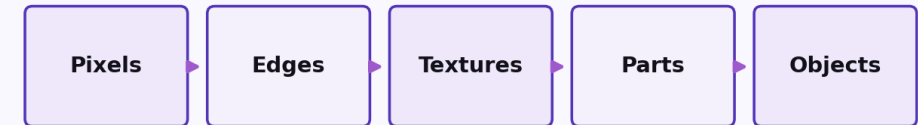
Why Depth Works

Hierarchical Representation Learning

Each layer rewrites the previous layer's features into a more useful space.

- Early layers detect simple patterns (edges, local tokens).
- Middle layers compose those into textures, parts, phrases.
- Late layers assemble task semantics (objects, intent, class scores).

Depth learns hierarchical features



Depth is composition. Width is how many features you keep at each level.

Session Roadmap

What we build today

Block	Question it answers
Perceptron & MLP	What does one layer compute?
Backpropagation	How do we update the weights?
CNNs	How do we exploit spatial structure?
RNNs & regularization	How do we model sequences and generalize?

Exit check: explain $z = w \cdot x + b$ and why nonlinearity is required between layers.

Single Neuron (Perceptron): Math, Weights, and Bias

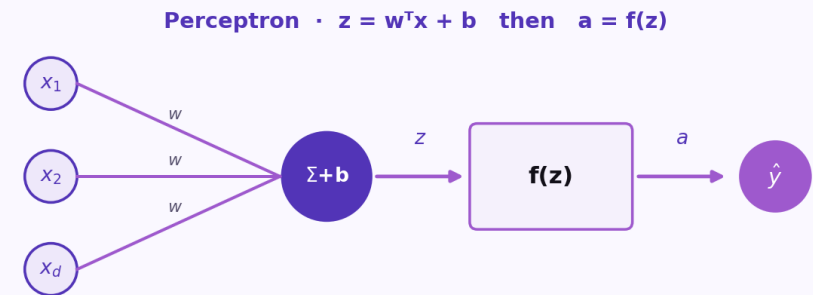
One Affine Map + One Nonlinearity

One weighted sum plus bias, then an activation. The output can be used as-is or passed onward.

Formula

$$z = w^T x + b, \quad a = f(z), \quad \hat{y} = a$$

$\hat{y} = a$ (or further processing)



Single Neuron (Perceptron): Math, Weights, and Bias

One Affine Map + One Nonlinearity

- Affine part (see formula above): z is the weighted sum of inputs plus b —each x_i is multiplied by w_i .
- The bias b shifts the decision boundary (same orientation, normal w); it does not rotate the separating hyperplane.
- Weights encode which patterns excite the neuron; training adjusts w and b so that on labeled data the loss decreases.
- With no hidden layer and a monotonic activation, the decision boundary in input space is linear (a hyperplane) —powerful but limited.

Symbols at a Glance

One Affine Map + One Nonlinearity

Symbol	Shape intuition	Role
x	Feature vector (length d)	One training example's inputs
w	Same length as x	Direction + strength of sensitivity to each feature
b	Scalar	Offset / threshold in logit space
f	Maps $\mathbb{R} \rightarrow \mathbb{R}$ (per neuron)	Introduces nonlinearity (or identity)

Bias shifts the hyperplane; it does not rotate it. Rotation comes from w.

Weight Initialization: Why Zeros Break Training

Symmetry Must Be Broken

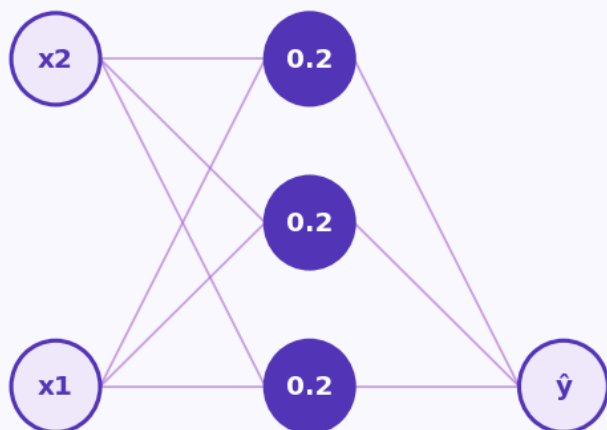
- If all weights in a layer start equal (especially zero), every hidden unit receives the same gradient and stays identical forever—symmetry never breaks.
- Random initialization makes each unit walk a different path in parameter space so they can specialize as feature detectors.
- Frameworks use He (Kaiming) for ReLU and Xavier (Glorot) for Tanh to keep activation scale stable.

Weight Initialization: Why Zeros Break Training

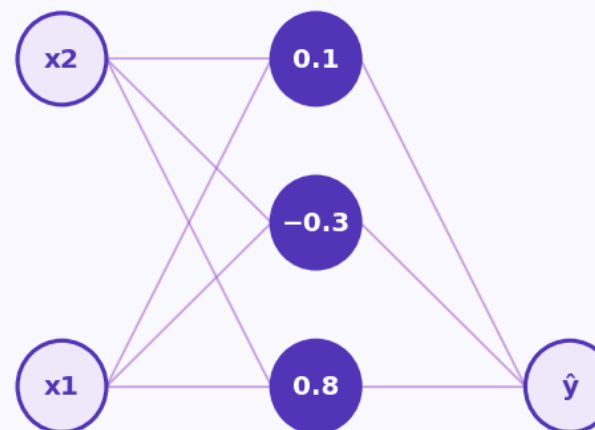
Symmetry Must Be Broken

Random init breaks symmetry

All zeros — units stay identical



He / Xavier — units specialize



Weight Initialization: Why Zeros Break Training

Symmetry Must Be Broken

He (Kaiming) for ReLU: variance = $2 / n_{\text{in}}$. Xavier (Glorot) for Tanh: variance = $2 / (n_{\text{in}} + n_{\text{out}})$.

$$\text{He : Var} = \frac{2}{n_{\text{in}}} \quad \text{Xavier : Var} = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

Keep activation scale stable across layers

- He matches ReLU, which zeros out about half the signals.
- Xavier matches Sigmoid / Tanh, which are symmetric around zero.

Weight Initialization: Why Zeros Break Training

Symmetry Must Be Broken

Init scheme	Problem it avoids	Best suited for
All zeros	Hidden units stay identical; no representation diversity	None (always avoid for weights)
Xavier (Glorot)	Vanishing or exploding variance across layers	Sigmoid / Tanh activations
He (Kaiming)	Variance drop due to ReLU zeroing out half the signals	ReLU / Leaky ReLU activations

Activation Functions: Sigmoid, Tanh, and ReLU

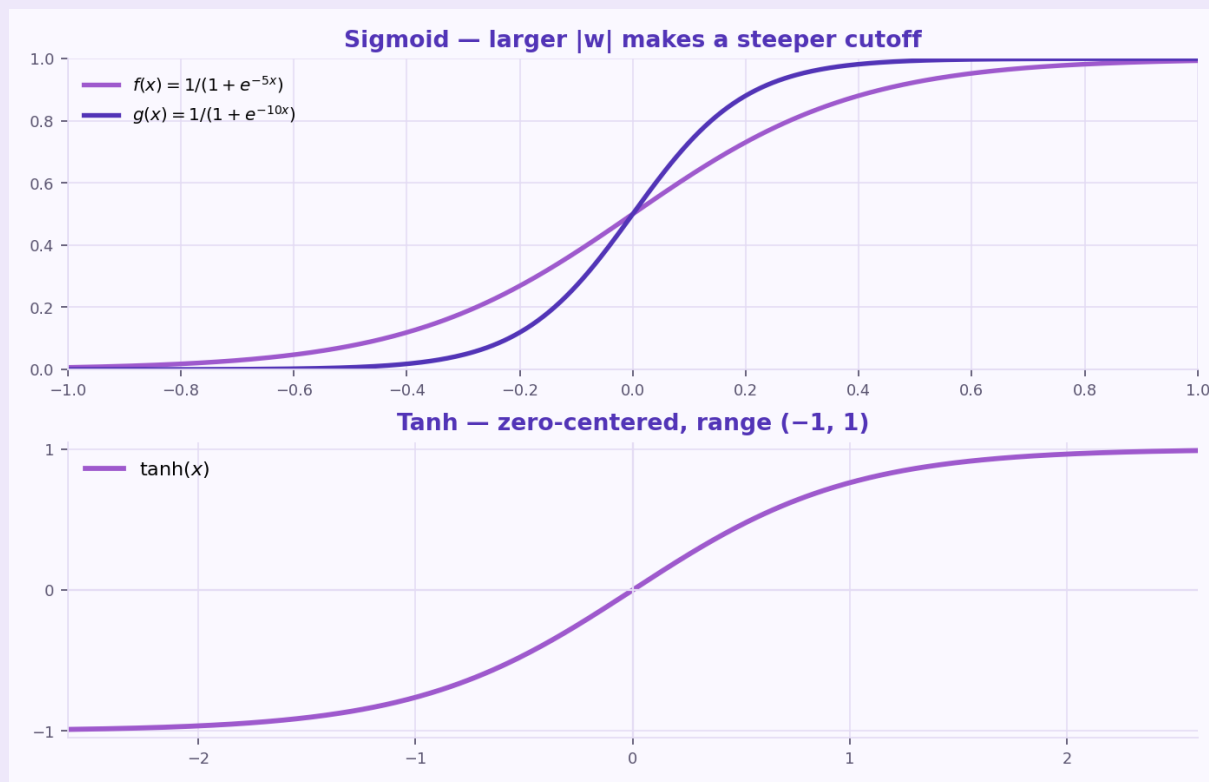
Nonlinearity is the Key to Network Depth

Stacked linear layers collapse to one linear map. Activations prevent that collapse.

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \text{ReLU}(z) = \max(0, z)$$

Activation Functions: Sigmoid, Tanh, and ReLU

Nonlinearity is the Key to Network Depth



$f(x)=1/(1+e^{-5x})$ vs $g(x)=1/(1+e^{-10x})$ — $\tanh(x)$ is zero-centered, range (-1, 1)

Activation Functions: Sigmoid, Tanh, and ReLU

Nonlinearity is the Key to Network Depth

- Sigmoid: maps inputs to $(0, 1)$ —useful for probabilities, but suffers from vanishing gradients when z is very large or very small (saturation).
- Tanh: zero-centered, maps inputs to $(-1, 1)$ —often nicer than sigmoid for hidden layers but still saturates.
- ReLU: modern default for deep stacks—cheap to compute, avoids vanishing gradient on positive domain; leaky ReLU / ELU are variants.

Activation Functions: Sigmoid, Tanh, and ReLU

Nonlinearity is the Key to Network Depth

Function	Range	Primary use-case
Sigmoid	$(0, 1)$	Binary classification output layer, gating networks
Tanh	$(-1, 1)$	Hidden layers (recurrent networks)
ReLU	$[0, \infty)$	Default choice for hidden layers in MLPs and CNNs

Sigmoid & Threshold

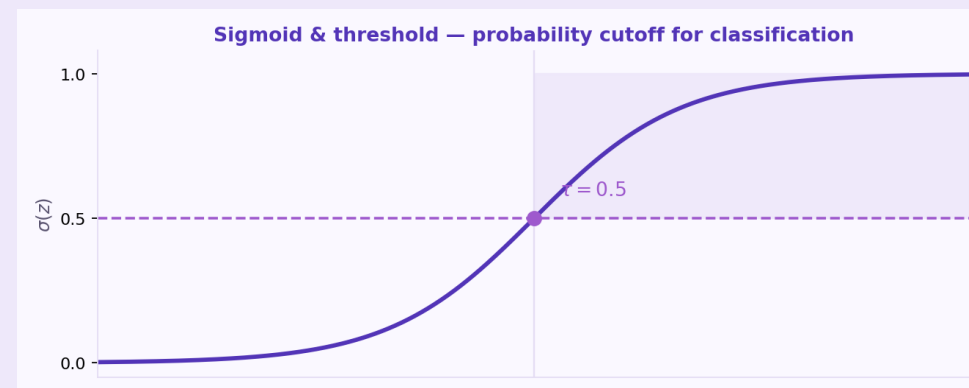
Probability cutoff for classification

Decision threshold $\tau = 0.5$ splits class 0 from class 1 on the sigmoid probability.

Formula

$$\tau = 0.5$$

Predict class 1 when $\sigma(z) \geq \tau$

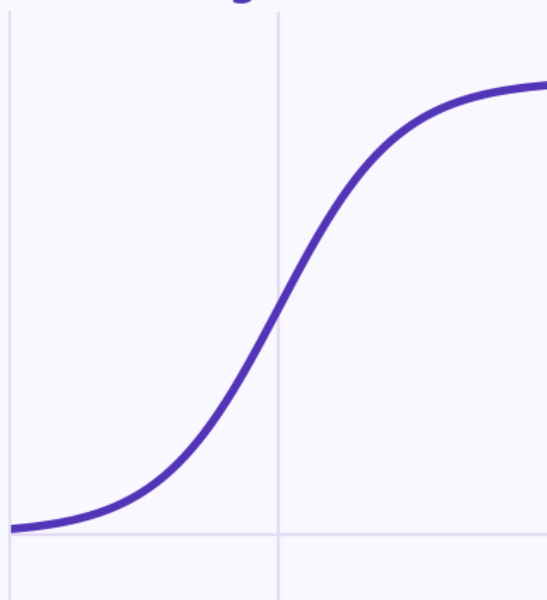


Activation Curves

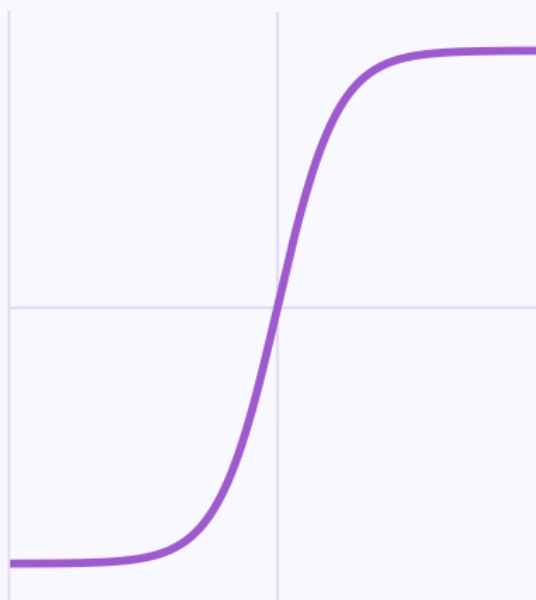
Nonlinearity is the Key to Network Depth

Activation functions introduce nonlinearity

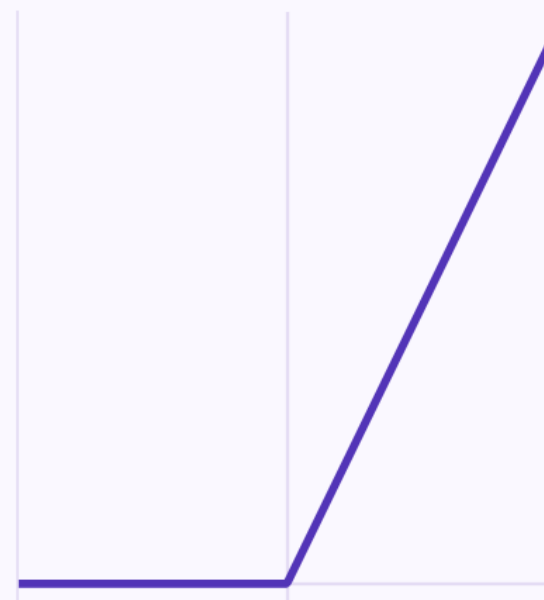
Sigmoid



Tanh



ReLU



— Flat tails (sigmoid / tanh) → tiny gradients. ReLU is linear on the positive side.

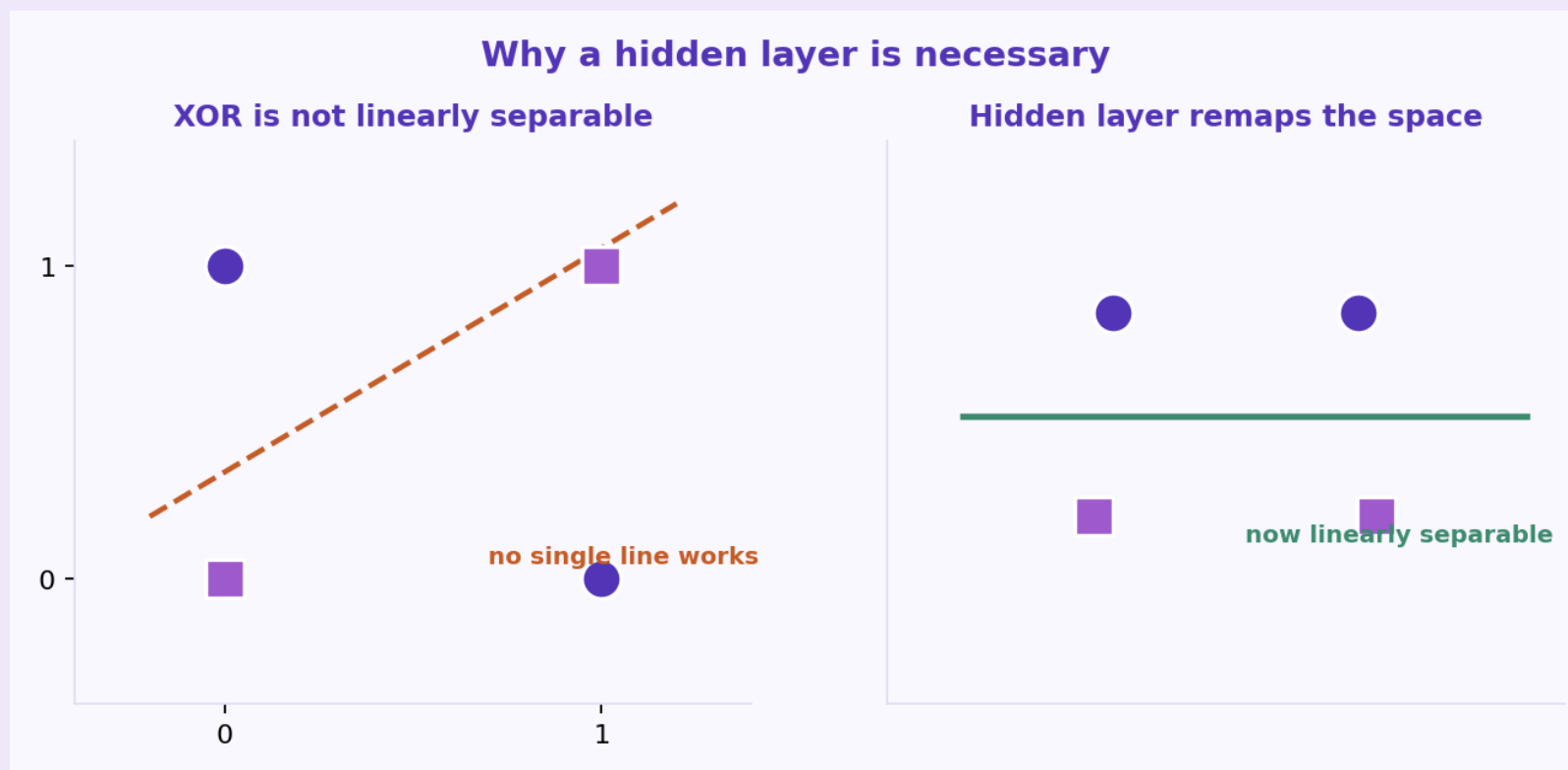
The XOR Problem: Why Hidden Layers Are Logically Necessary

Why a single line cannot solve XOR

- XOR is not linearly separable in the 2D input plane: no single line separates class 1 from class 0 on the four corners.
- A single perceptron computes a half-space; XOR requires nonlinear mixing of inputs—exactly what a hidden layer provides.
- Hidden units reparameterize inputs into a space where the task becomes linearly separable—this is the core intuition for representation learning.

The XOR Problem: Why Hidden Layers Are Logically Necessary

Why a single line cannot solve XOR



A hidden layer remaps the inputs so the output layer can cut with a line.

XOR Truth Table

Four Points, One Nonlinear Rule

x ₁	x ₂	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Hidden units reparameterize the inputs so XOR becomes linearly separable.

From Perceptron to MLP (Multi-Layer Perceptron)

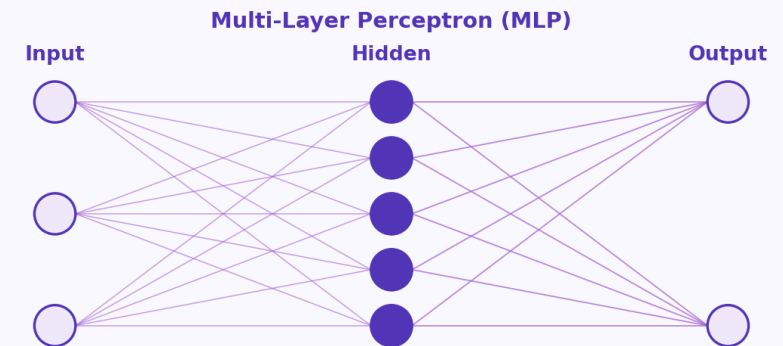
Depth = Composed Nonlinear Features

Hidden layer: affine map + nonlinearity $\rightarrow$ features h . Output: another affine map + τ .

Formula

$$h = \phi(W_1x + b_1), \quad \hat{y} = \tau(W_2h + b_2)$$

ϕ = hidden activation · τ = output activation



From Perceptron to MLP (Multi-Layer Perceptron)

Depth = Composed Nonlinear Features

- Each hidden layer applies an affine map followed by a nonlinearity, producing new features h as functions of the original inputs.
- Width (units per layer) and depth (number of layers) trade off capacity, data needs, compute, and optimization difficulty.
- Universal approximation (broad strokes): one sufficiently wide hidden layer can approximate many continuous functions on compact domains—in practice depth + inductive bias wins.

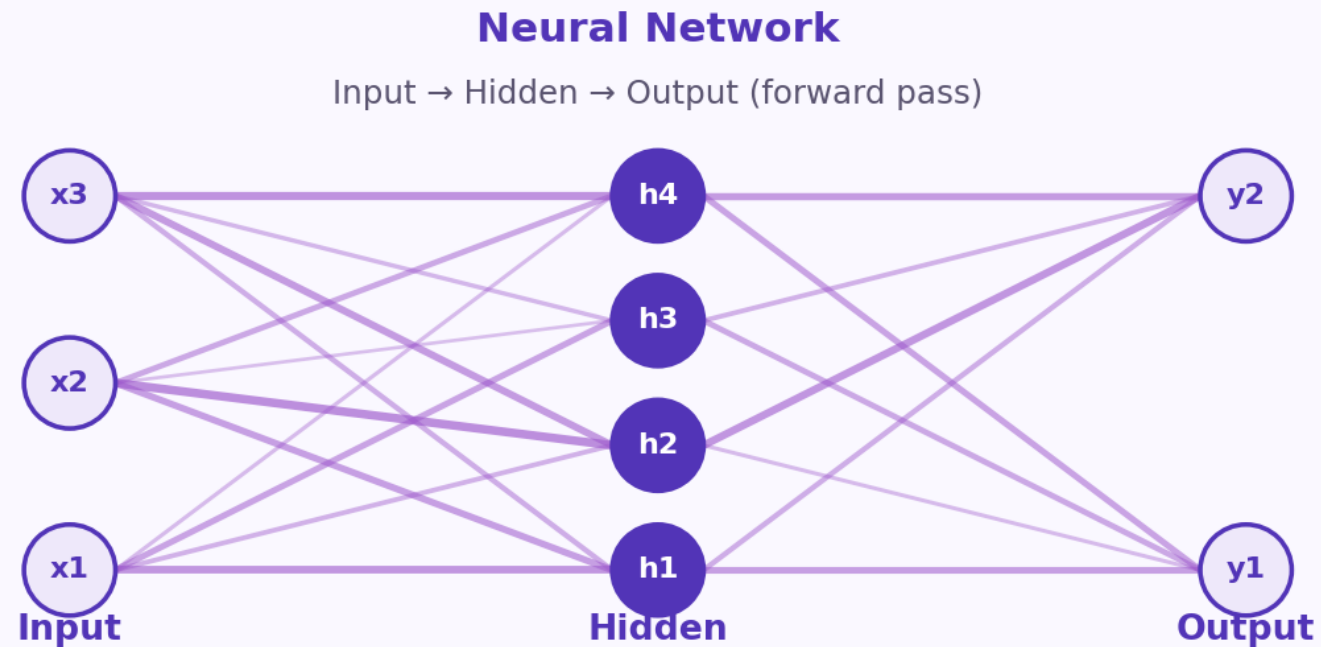
From Perceptron to MLP (Multi-Layer Perceptron)

Depth = Composed Nonlinear Features

Hyperparameter	What it controls
Hidden width	How many independent nonlinear features per layer
Depth	How many levels of composition (edges→textures→...)
Skip / residual	Ease of optimization in very deep nets

Forward Propagation: The Engine of Inference

Neural Network



—Edge thickness reflects weight magnitude.

Forward Propagation: The Engine of Inference

The Engine of Inference

- Forward pass: feed input x through each layer in order—compute logits or scores at the end (then softmax for class probabilities if needed).
- Implementation-wise this is a computational graph: each op (matmul, ReLU, ...) knows how to propagate values forward.
- Training repeats forward passes on mini-batches for efficiency and gradient noise that can help generalization.

Forward Propagation: The Engine of Inference

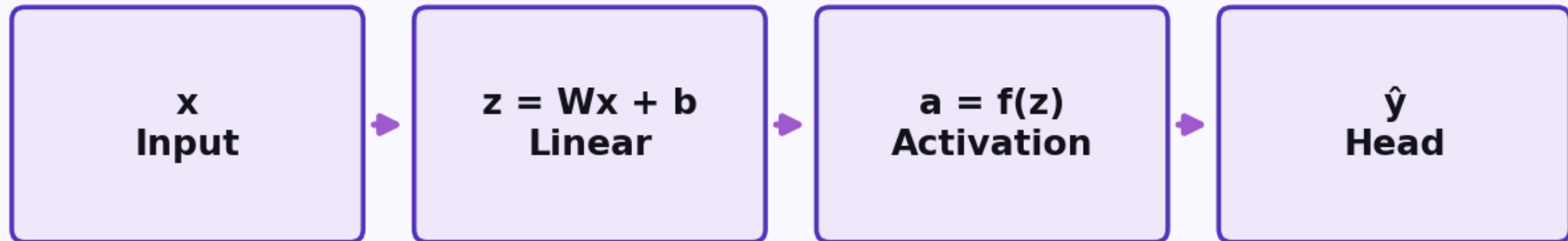
The Engine of Inference

Step	Operation	Output carries...
1	Linear: $z = Wx + b$	Weighted sum at each unit
2	Activation: $a = f(z)$	Nonlinear feature for next layer
3	Repeat / head	Class logits or regression value

Forward Propagation: The Engine of Inference

Linear → Activation → Head

Forward pass — inference through the graph



— Training repeats this on mini-batches. The graph also knows how to send gradients backward.

Loss Functions: MSE vs Cross-Entropy

Match the Loss to the Output and Task

Regression — Mean Squared Error (MSE)

$$\mathcal{L} = \frac{1}{N} \sum_i \|\hat{y}_i - y_i\|^2$$

- Penalizes large errors heavily (quadratic); differentiable everywhere—classic for continuous targets.
- Assumes Gaussian-like noise in a probabilistic story; outliers can dominate—Huber loss is a robust alternative.

Loss Functions: MSE vs Cross-Entropy

Match the Loss to the Output and Task

Classification — Cross-entropy

$$\mathcal{L} = -\frac{1}{N} \sum_i \sum_c y_{ic} \log \hat{p}_{ic}$$

Often with logits + log-softmax for stability

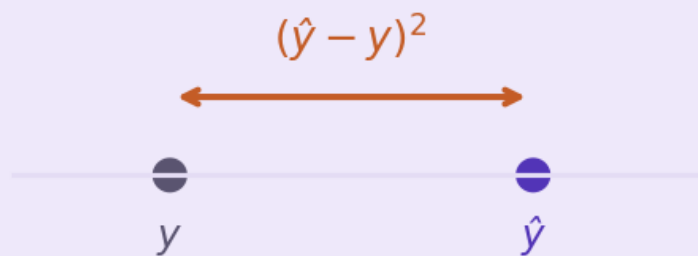
- Compares predicted probabilities $\hat{p}$ to the true distribution (one-hot or soft labels).
- Pairs naturally with softmax at the output; gradients are well-behaved when implemented in log-space.

Loss Functions: MSE vs Cross-Entropy

Match the Loss to the Output and Task

Loss matches the task head

MSE — regression



Cross-entropy — classes



$-\log \hat{p}_y$ on the true class

Loss Functions: MSE vs Cross-Entropy

Match the Loss to the Output and Task

Task	Common head + loss
Regression	Linear output + MSE / Huber
Binary classification	Sigmoid + binary cross-entropy
Multi-class	Softmax + cross-entropy

Often with logits + log-softmax for numerical stability.

Backpropagation: Chain Rule on the Computational Graph

Chain along paths

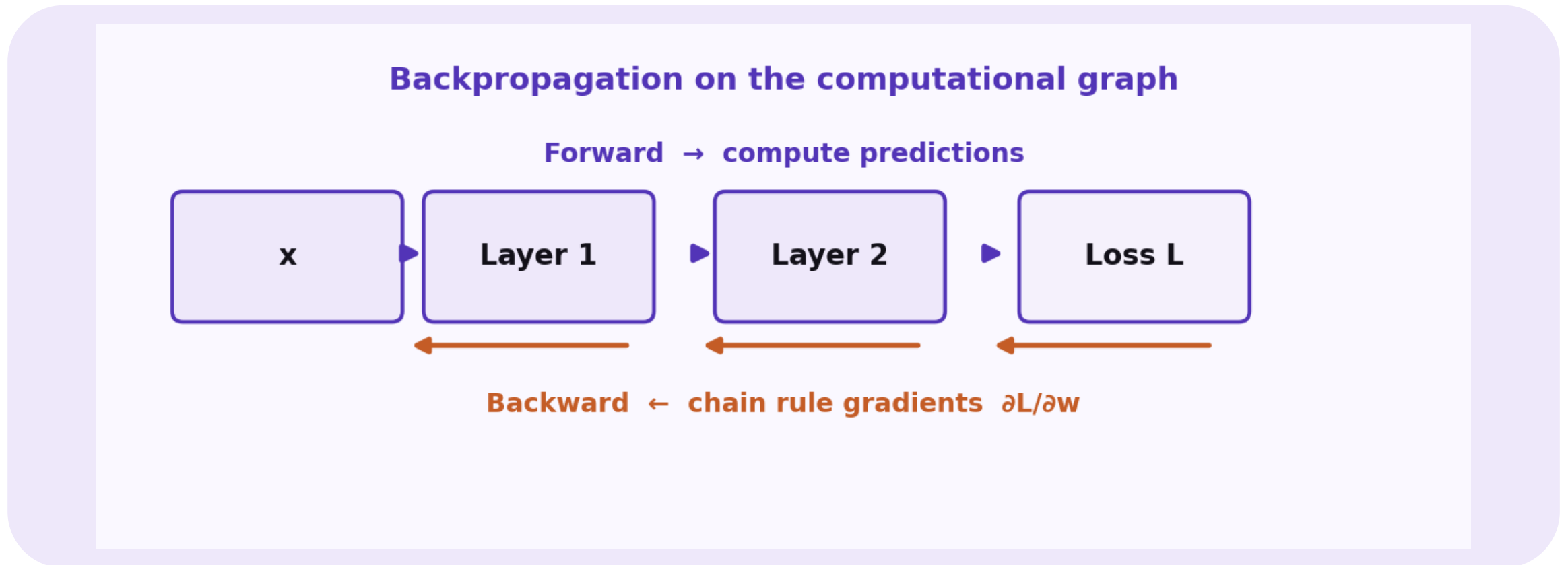
$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial Z} \frac{\partial Z}{\partial W}$$

Chain along paths

- Backprop applies the chain rule systematically: compute adjoints $\partial \mathcal{L} / \partial$ each intermediate tensor from outputs backward.

Backpropagation: Chain Rule on the Computational Graph

Chain along paths



Forward computes predictions. Backward carries $\partial L / \partial w$ along every path.

Backpropagation: Chain Rule on the Computational Graph

Chain along paths

- Reverse-mode autodiff (backprop) is efficient when many parameters feed one scalar loss—exactly the neural net case.
- Modern frameworks (PyTorch, JAX, TensorFlow) build the graph (explicit or traced) and implement this reliably—you still debug shapes, numerical stability, and vanishing gradients.

Understanding backprop = understanding which paths carry gradient and which activations saturate.

Backpropagation: Chain Rule on the Computational Graph

Chain along paths

Idea	Meaning
Shared subexpressions	Same forward values reused; backward visits each edge once
Topological order	Visit nodes after their successors—dynamic programming on the graph

Understanding backprop = understanding which paths carry gradient and which activations saturate.

Phase 2: Optimization & Training Strategy

Stable Optimization + Generalization

- Phase 2 is about the art of training: diagnosing fit, controlling gradients, picking optimizers, normalizing, and regularizing.
- Good models are not only “big”—they match capacity to data, monitor validation faithfully, and fail visibly when something is wrong.

Phase 2: Optimization & Training Strategy

Stable Optimization + Generalization

Four phases of this Deep Learning block

1 Foundations

Why depth?
What does a
neuron compute?

2 Optimization

How does training
stay stable and
generalize?

3 Architectures

Images vs
sequences —
which structure?

4 Autoencoders

Generation and
reconstruction

Phase 2: Optimization & Training Strategy

What you will control in practice

Layer	Examples
Optimization	SGD, Adam, gradient clipping
Stabilization	Batch Norm, residual paths, initialization
Generalization	Dropout, weight decay, early stopping

Training Diagnostics: Underfitting, Good Fit, and Overfitting

Read Train vs Validation Behavior

- Underfitting: both train and validation error stay high—the model is too simple, features are insufficient, or optimization is stuck (LR too low, wrong loss).
- Good fit: train and validation errors are both low and track each other; small gap is normal if train is slightly better.
- Overfitting: train error keeps dropping while validation worsens or plateaus badly—the model memorizes idiosyncrasies (noise, augment leakage).

Training Diagnostics: Underfitting, Good Fit, and Overfitting

Read Train vs Validation Behavior



A small train-val gap is normal. A widening gap is overfitting.

Training Diagnostics: Underfitting, Good Fit, and Overfitting

Read Train vs Validation Behavior

Regime	Train loss	Val loss / metric	Typical response
Underfitting	High	High	More capacity, better features, longer train, higher LR (carefully)
Good fit	Low	Low, close to train	Keep regularization; ship or iterate on data
Overfitting	Very low	Worse / diverging gap	More data, dropout, wd, simpler model, early stop

Vanishing and Exploding Gradients

Depth Multiplies Jacobians Layer by Layer

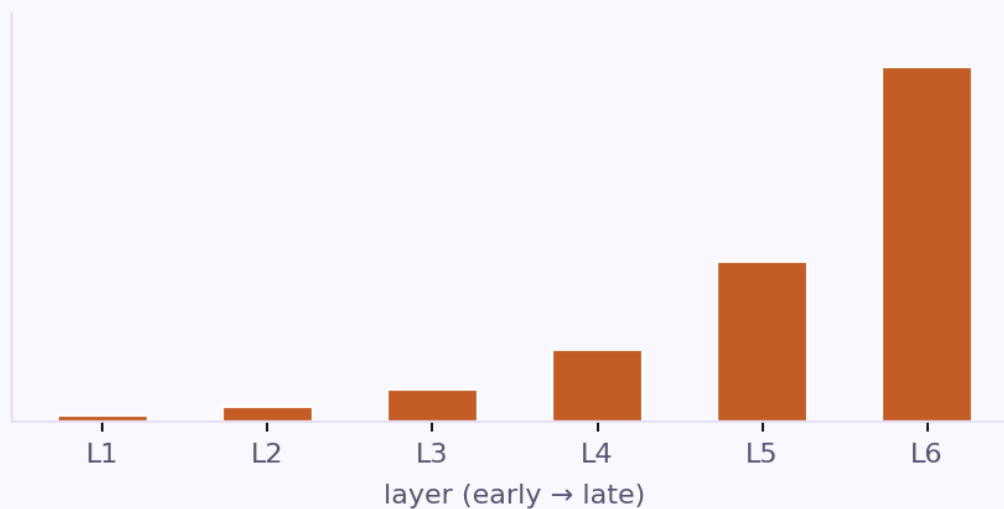
- Vanishing: backprop multiplies many factors < 1 (saturated sigmoid/tanh, poor init)—early layers get tiny updates and learn slowly.
- Exploding: repeated factors > 1 or large weights produce huge parameter updates—loss spikes, NaNs.
- Fixes are layered: architecture (residual paths), activations (ReLU), initialization (He/Xavier), normalization (BN/LN), and optimization (clipping).

Vanishing and Exploding Gradients

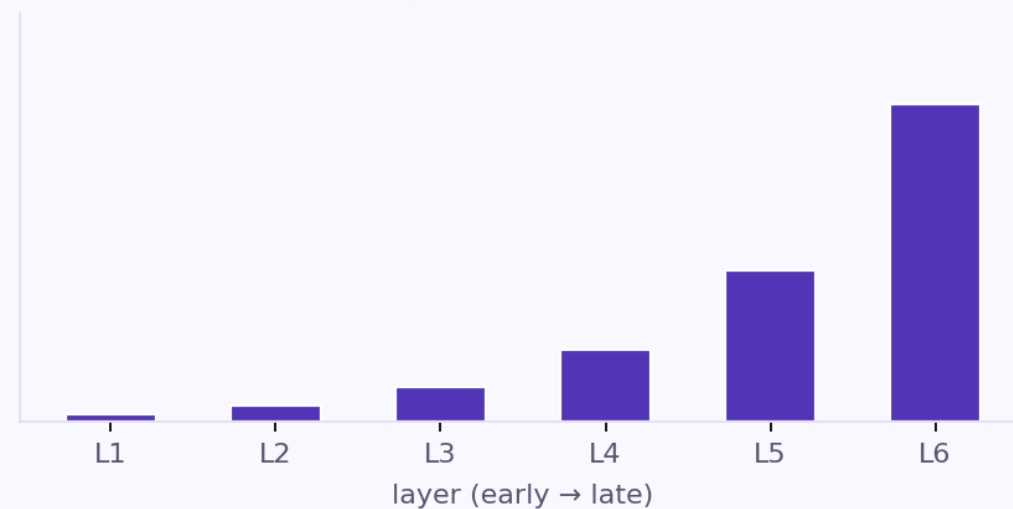
Depth Multiplies Jacobians Layer by Layer

|gradient| **through depth**

Vanishing — signal dies



Exploding — updates blow up



Vanishing and Exploding Gradients

Depth Multiplies Jacobians Layer by Layer

Symptom	Likely cause	Mitigation
Early layers barely move	Vanishing chain through saturated units	ReLU, He init, residuals, norm
Loss → NaN quickly	Exploding activations or LR too high	Clip grads, lower LR, check loss scaling
Stable then sudden blow-up	Rare but real—mixed precision, bad data batch	Finite checks, gradient clipping

Optimizers: SGD and Adam

Learning rate, Momentum, and Adaptive Steps

- SGD + momentum: accumulates updates in a velocity vector to dampen oscillations in ravines and accelerate along consistent directions.
- Adam: tracks both first-moment (momentum) and second-moment (recent squared gradients) of gradients to adapt learning rates per parameter.
- SGD is often preferred in computer vision for generalization, while Adam is the default starting point for Transformers and general MLPs.

Learning rate (LR) is the single most important hyperparameter to tune (typically searched on a log grid, e.g., $1e-4$, $1e-3$, $1e-2$).

Optimizers: SGD and Adam

Learning rate, Momentum, and Adaptive Steps



Optimizers: SGD and Adam

Learning rate, Momentum, and Adaptive Steps

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

η = learning rate · typical log grid: 1e-4, 1e-3, 1e-2

Optimizers: SGD and Adam

Learning rate, Momentum, and Adaptive Steps

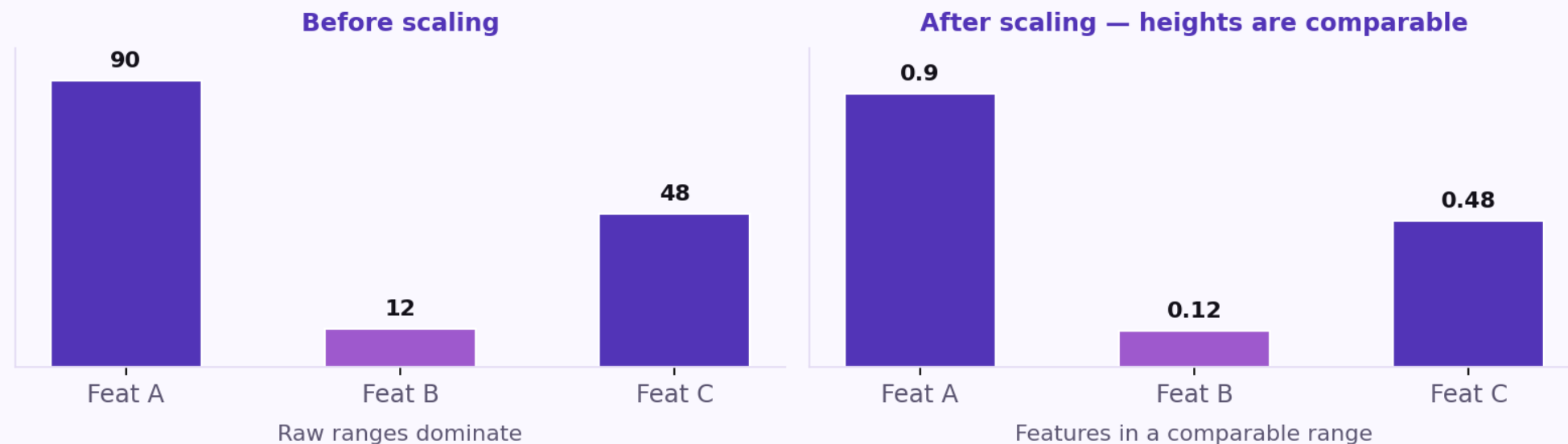
Optimizer	Core mechanism	When to use
Vanilla SGD	Step in negative gradient direction	Rarely used alone today
SGD + Momentum	Adds fraction of previous step direction	Standard for CV models
Adam	Adapts learning rate per parameter based on gradients	Robust default for text/tabular/MLPs

Learning rate (LR) is the single most important hyperparameter to tune (typically searched on a log grid, e.g., 1e-4, 1e-3, 1e-2).

Normalization: Batch Normalization

Stabilize Activations and Speed Up Deep Training

Feature Scaling



—Feature scaling: features brought to a comparable range.

Normalization: Batch Normalization

Stabilize Activations and Speed Up Deep Training

Batch Norm (BN): normalizes activations across the mini-batch for each channel, then learns scale γ and shift β .

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

- It stabilizes internal activations, allowing higher learning rates and reducing dependency on precise initialization.
- Alternative is Layer Norm (LN) which normalizes across features per example—standard in Transformers and sequence models.

Normalization: Batch Normalization

Stabilize Activations and Speed Up Deep Training

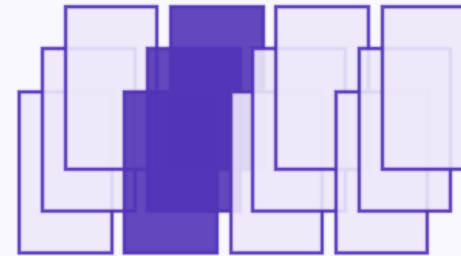
Which axis you normalize changes the model class

Batch Norm



across batch (and space)

Layer Norm



across features, one example

Normalization: Batch Normalization

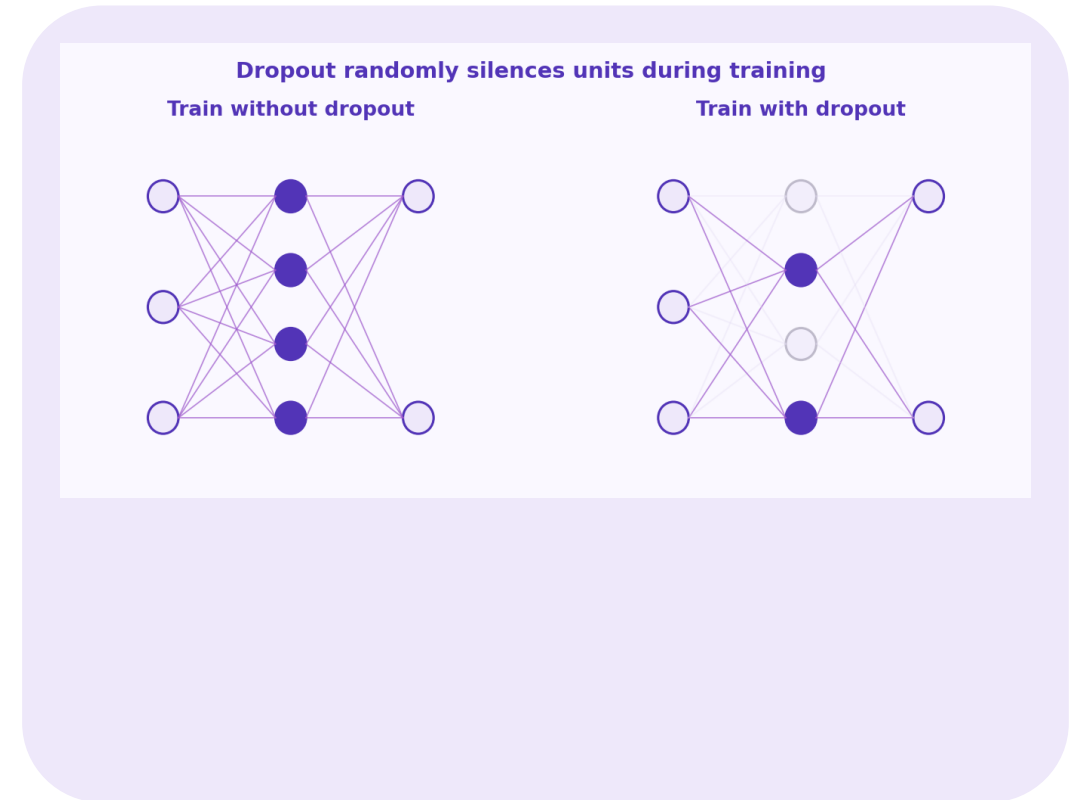
Stabilize Activations and Speed Up Deep Training

Method	Normalization dimension	Typical use-case
Batch Norm	Across batch and spatial dimensions	CNN classifiers, ResNets
Layer Norm	Across features for a single token/example	Transformers, RNNs

Regularization: Dropout, Weight Decay, and Early Stopping

Constrain Capacity to Prevent Overfitting

- Randomly zero out hidden units during training with probability p (e.g., 0.2-0.5) so neurons cannot co-adapt.



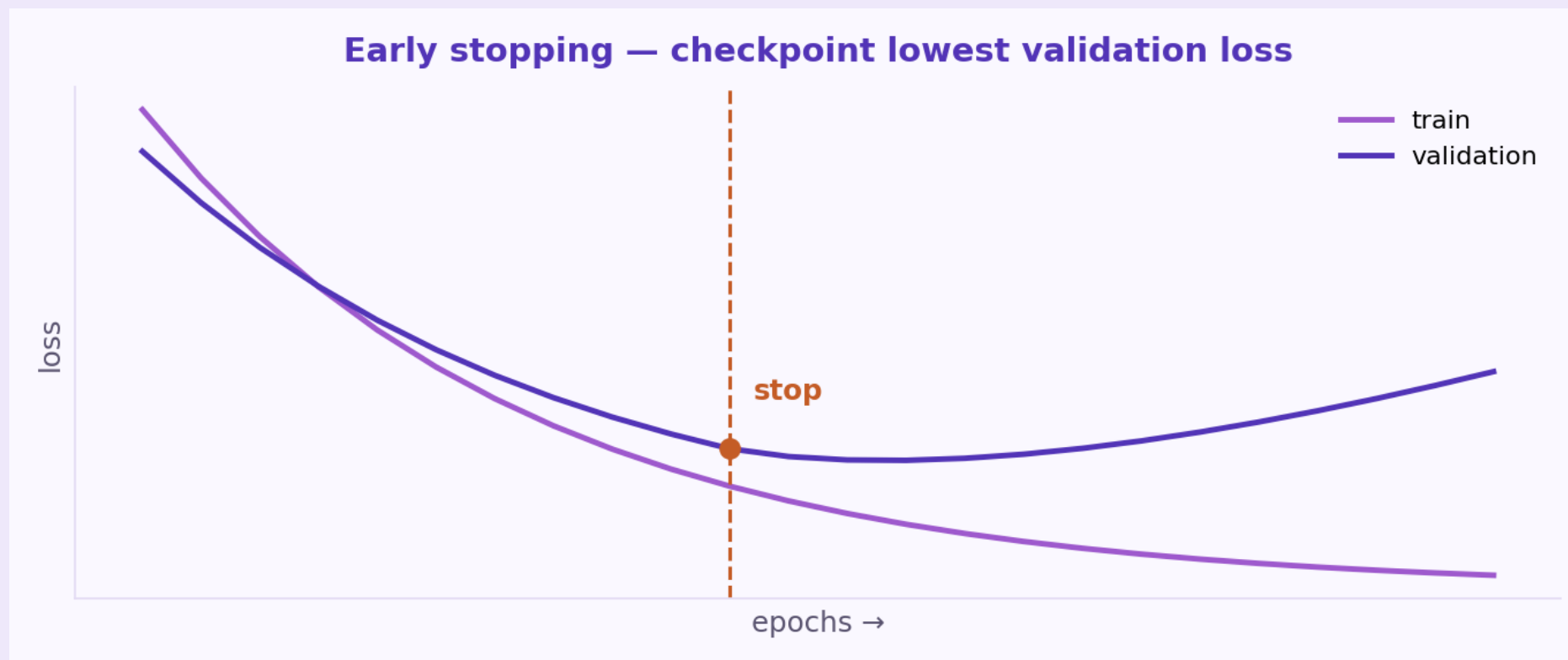
Regularization: Dropout, Weight Decay, and Early Stopping

Constrain Capacity to Prevent Overfitting

- Dropout acts as a powerful implicit ensemble method; disabled during inference (evaluation mode).
- Weight Decay (L2 regularization): adds a penalty to the loss proportional to the squared sum of weights to keep weights small and smooth.
- Early stopping: monitor validation loss during training and stop when it ceases to improve for a certain number of epochs (patience).

Regularization: Dropout, Weight Decay, and Early Stopping

Constrain Capacity to Prevent Overfitting



Regularization: Dropout, Weight Decay, and Early Stopping

Constrain Capacity to Prevent Overfitting

Technique	Primary action	Inference behavior
Dropout	Randomly drops connections	Turned OFF (weights scaled)
L2 Weight Decay	Shrinks weights toward zero	Turned ON (weights are static)
Early Stopping	Halts training at lowest val loss	N/A (model is checkpointed)

Phase 3: Specialized Architectures (Vision & Sequences)

Images vs Ordered Sequences

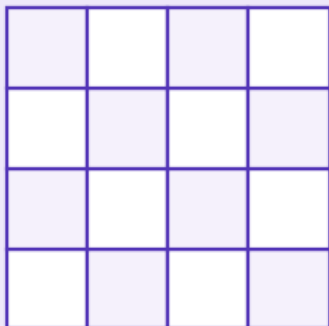
- Images have 2D locality and translation symmetry—convolutions exploit shared weights across space.
- Sequences have order—recurrence, causal convolutions, or attention carry context across time or position.
- Choosing the right inductive bias beats blindly scaling the wrong architecture.

Phase 3: Specialized Architectures (Vision & Sequences)

Images vs Ordered Sequences

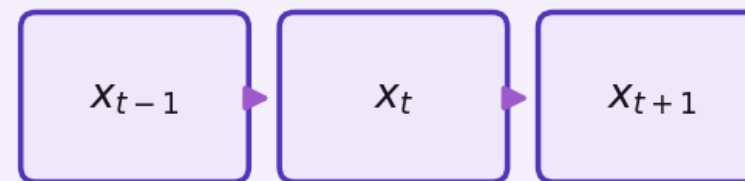
Inductive bias follows the data's structure

Images — spatial grid



Conv2D · pooling · residual

Sequences — order



RNN / GRU / LSTM · attention

Phase 3: Specialized Architectures (Vision & Sequences)

Images vs Ordered Sequences

Modality	Core structure	Representative layers
Image / video frames	Spatial grids	Conv2D, pooling, residual blocks
Text / speech / sensors	Ordered tokens or time steps	RNN/GRU/LSTM, causal conv, self-attention

Image Data: How Computers “See” Pixels

Tensors, Channels, and Local Structure

- A color image is usually a tensor of shape $H \times W \times 3$ (RGB); each entry is an intensity discretized into 8 bits (0–255) before normalization.
- Meaning lives in local neighborhoods: small patches reveal edges; larger contexts reveal objects—hierarchical composition.
- Augmentation (crop, flip, color jitter) teaches invariance; normalization (e.g., ImageNet mean/std) stabilizes optimization.

Image Data: How Computers “See” Pixels

Tensors, Channels, and Local Structure

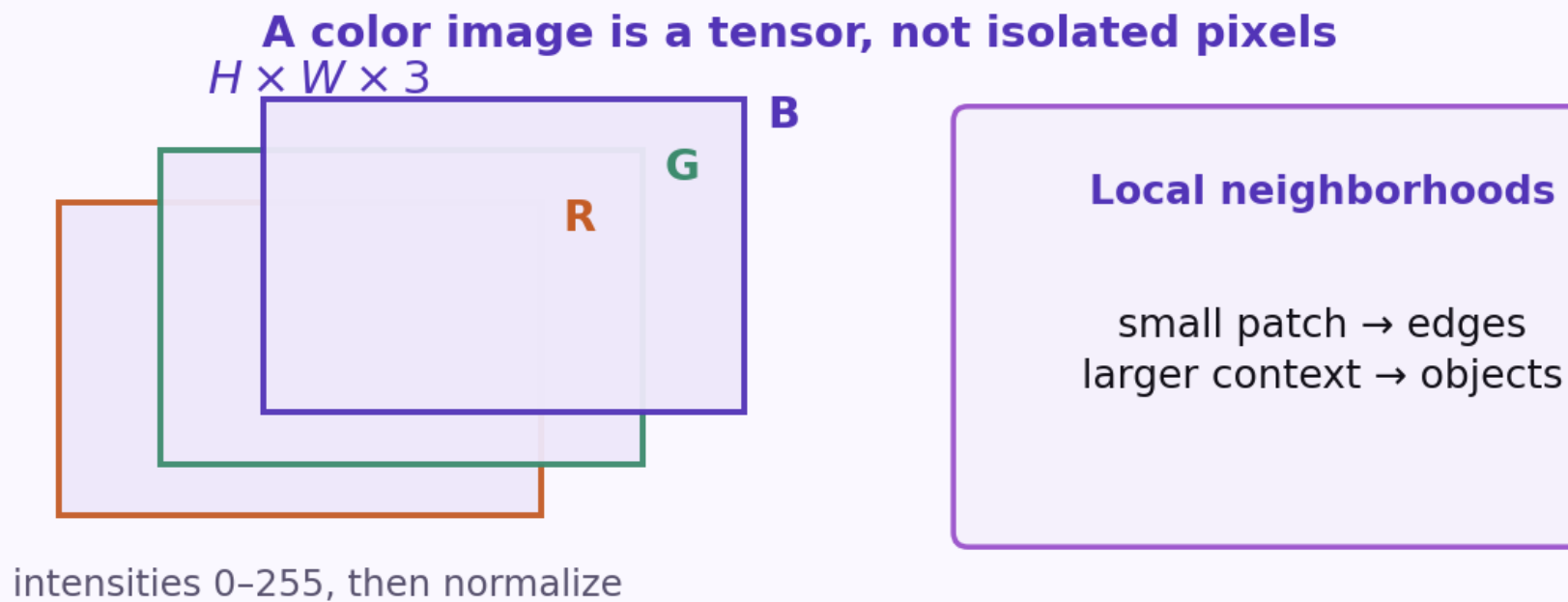


Image Data: How Computers “See” Pixels

Tensors, Channels, and Local Structure

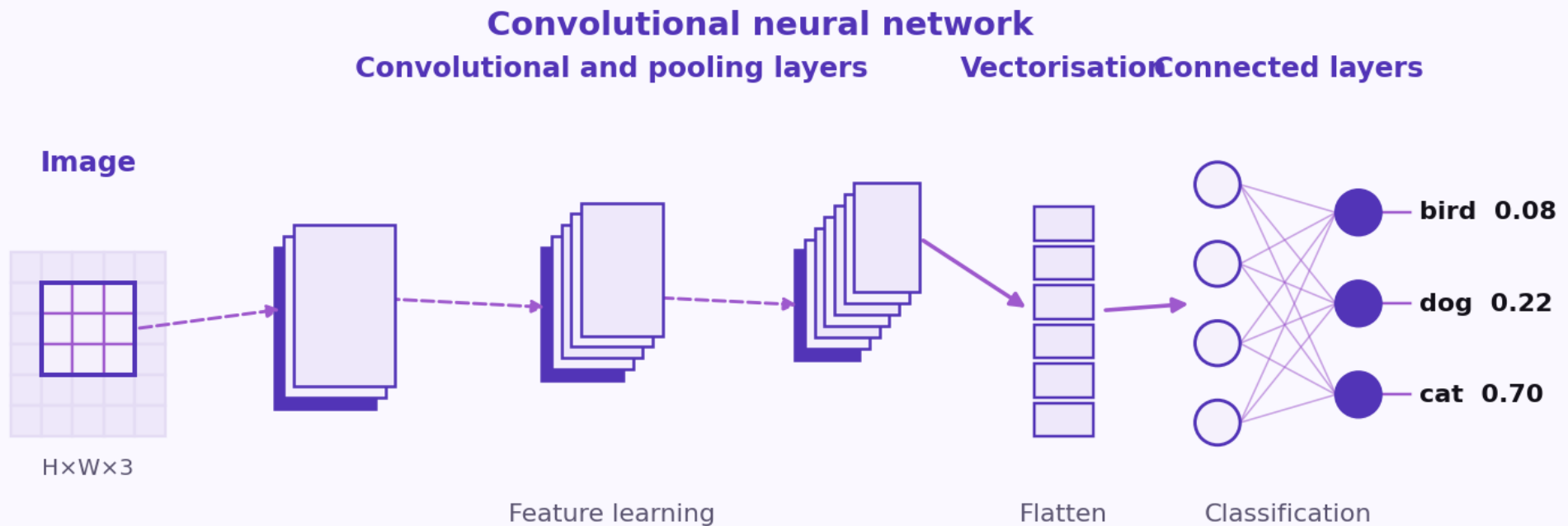


Image Data: How Computers “See” Pixels

Tensors, Channels, and Local Structure

Stage	What happens
Raw sensor	Quantized intensities per channel per pixel
Preprocess	Resize, crop, normalize, augment
Model input	Batch × C × H × W tensor fed to stem convolutions

Convolution: Kernels, Edge Filters, and Feature Maps

Sliding Linear Filters + Nonlinearity

- A kernel is a small learnable matrix (e.g., 3×3) that responds to local patterns; multiple kernels produce a stack of feature maps.
- Classical edge detectors (Sobel, shown) are fixed kernels; CNNs learn task-specific filters from data.
- Deep stacks widen the receptive field: early layers $\approx$ edges/textures; late layers $\approx$ object parts and semantics.

Convolution: Kernels, Edge Filters, and Feature Maps

Sliding Linear Filters + Nonlinearity

Convolution — slide a kernel, write a feature map

Image patch

7	1	2	2	2
7	7	5	1	1
3	4	5	4	3
2	6	6	1	1
4	4	8	5	4



**3×3
kernel**



Feature map

4	5	5
3	6	6
7	6	3

Convolution: Kernels, Edge Filters, and Feature Maps

Sliding Linear Filters + Nonlinearity

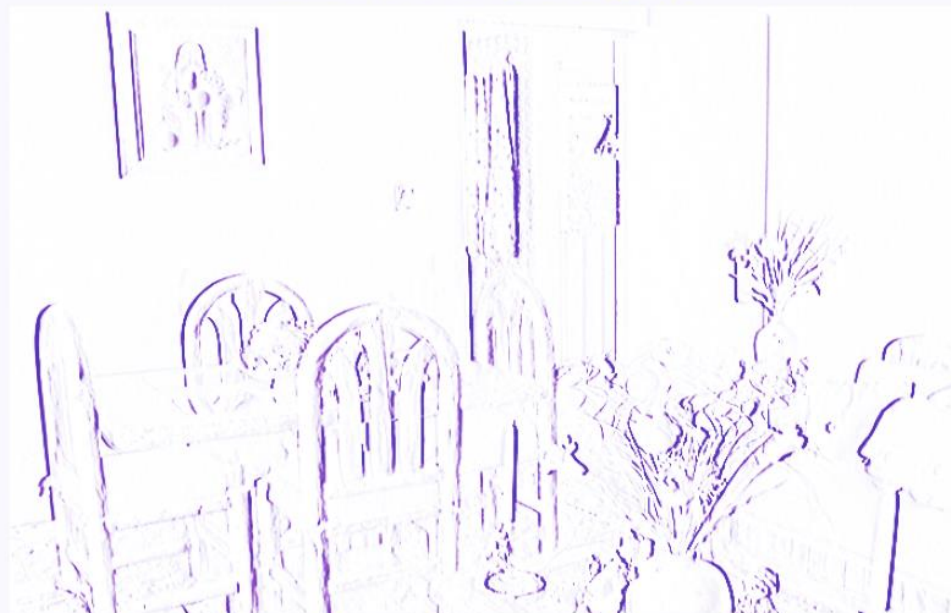
Classical edge detector — a kernel that is not learned

Sobel Y on a photograph

Sobel Y kernel

-1	-2	-1
0	0	0
+1	+2	+1

Fixed 3×3 filter

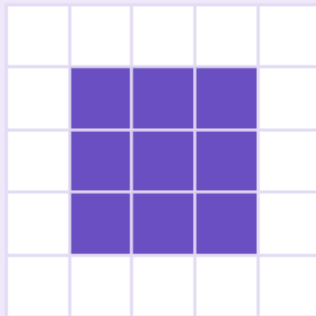


Convolution: Kernels, Edge Filters, and Feature Maps

Sliding Linear Filters + Nonlinearity

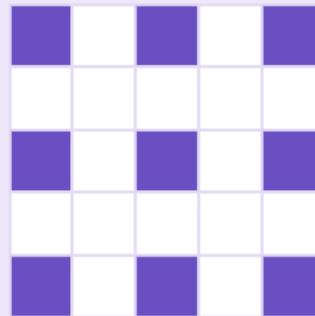
Convolution hyperparameters

Kernel 3×3



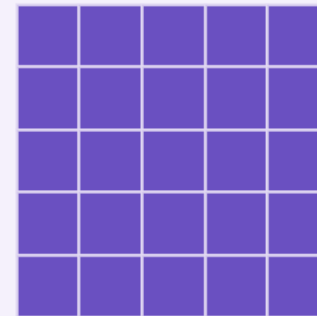
neighborhood size

Stride 2



downsample when >1

Same padding



keep spatial size

Convolution: Kernels, Edge Filters, and Feature Maps

Sliding Linear Filters + Nonlinearity

Hyperparameter	Effect
Kernel size	Immediate neighborhood size (3×3 common)
Stride	Downsample spatially when >1
Padding	Keep spatial size (same padding) or control border

CNN Blocks: From Conv-Pool to ResNet

Depth, Regularization, and Residual Learning

$$y = \mathcal{F}(x) + x$$

(residual block learns $\mathcal{F}$, identity carries gradient)

CNN Blocks: From Conv-Pool to ResNet

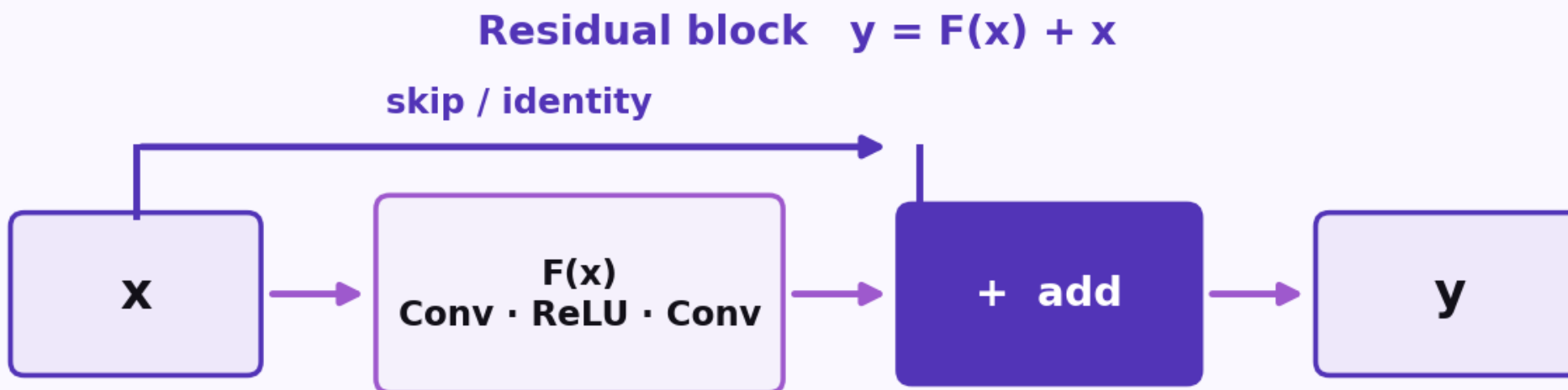
Depth, Regularization, and Residual Learning

Typical CNN stack



CNN Blocks: From Conv-Pool to ResNet

Depth, Regularization, and Residual Learning



CNN Blocks: From Conv-Pool to ResNet

Depth, Regularization, and Residual Learning

- Standard CNN: Alternates Conv2D, Activation (ReLU), and Pooling (Max Pool) layers to extract spatial features and downsample.
- AlexNet (2012): First large-scale CNN to win ImageNet; used ReLU, dropout, and multi-GPU training.
- ResNet (2015): Introduced skip (shortcut) connections, allowing gradients to flow back easily through identity mappings. Enables training networks with 100+ layers.

CNN Blocks: From Conv-Pool to ResNet

Landmark ideas (simplified)

Model / block	Idea you should remember
Conv + MaxPool	Feature extraction + spatial downsampling
AlexNet	Scale + ReLU + data aug on GPUs
ResNet	Residual mapping $F(x)+x$; skip paths help gradients

Transfer Learning with Pretrained Models

Reuse Features, Adapt the Head

- Pretraining on millions of labeled images (e.g. ImageNet) learns general low/mid-level filters (edges, textures) transferable to new domains.
- Typical recipe: replace the final classifier head; freeze the pretrained backbone for small datasets, or fine-tune with a very small learning rate for larger datasets.
- Transfer learning dramatically reduces training time and required labeled samples for a new vision task.

Transfer Learning with Pretrained Models

Reuse Features, Adapt the Head

Transfer learning — reuse features, adapt the head

Frozen backbone

ImageNet features
edges → textures → parts

small or zero learning rate



New head

Train classifier
on your labels

few classes · less data

Transfer Learning with Pretrained Models

Reuse Features, Adapt the Head

Target Dataset Size	Strategy
Small target dataset	Freeze backbone, train head only
Medium dataset	Fine-tune top blocks with a smaller learning rate
Large target dataset	Full fine-tune of all layers with a small learning rate

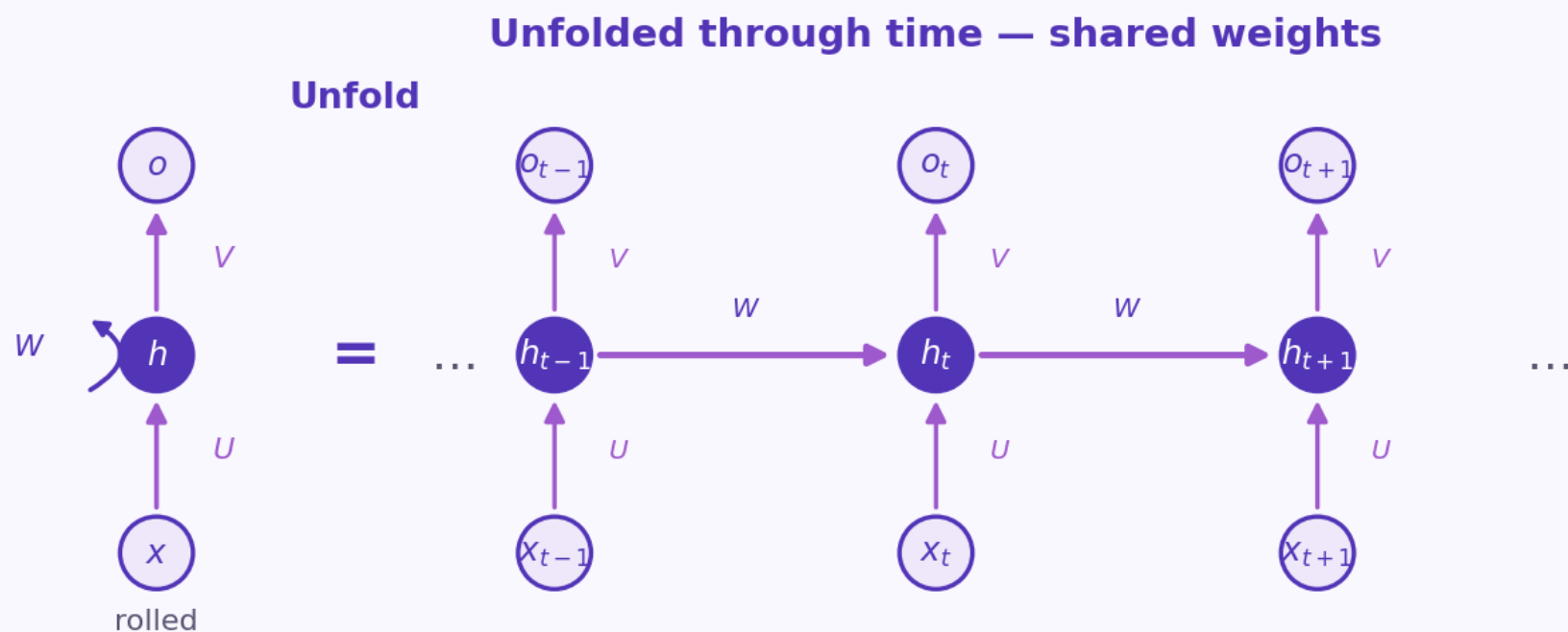
Sequential Data & Vanilla RNNs

One Transition, Unfolded Through Time

$$h_t = \varphi(W_{hh}h_{t-1} + W_{xh}x_t + b)$$

Sequential Data & Vanilla RNNs

One Transition, Unfolded Through Time



Sequential Data & Vanilla RNNs

One Transition, Unfolded Through Time

- Sequence modeling: text, audio, and sensor streams have order; shuffling them breaks their semantic meaning.
- Recurrent Neural Networks (RNN): maintain a recurrent state vector h_t that acts as a memory summary of the past.
- The same weights W_{hh} and W_{xh} are reused across all timesteps (parameter efficiency), but long-range dependencies suffer from vanishing/exploding gradients.

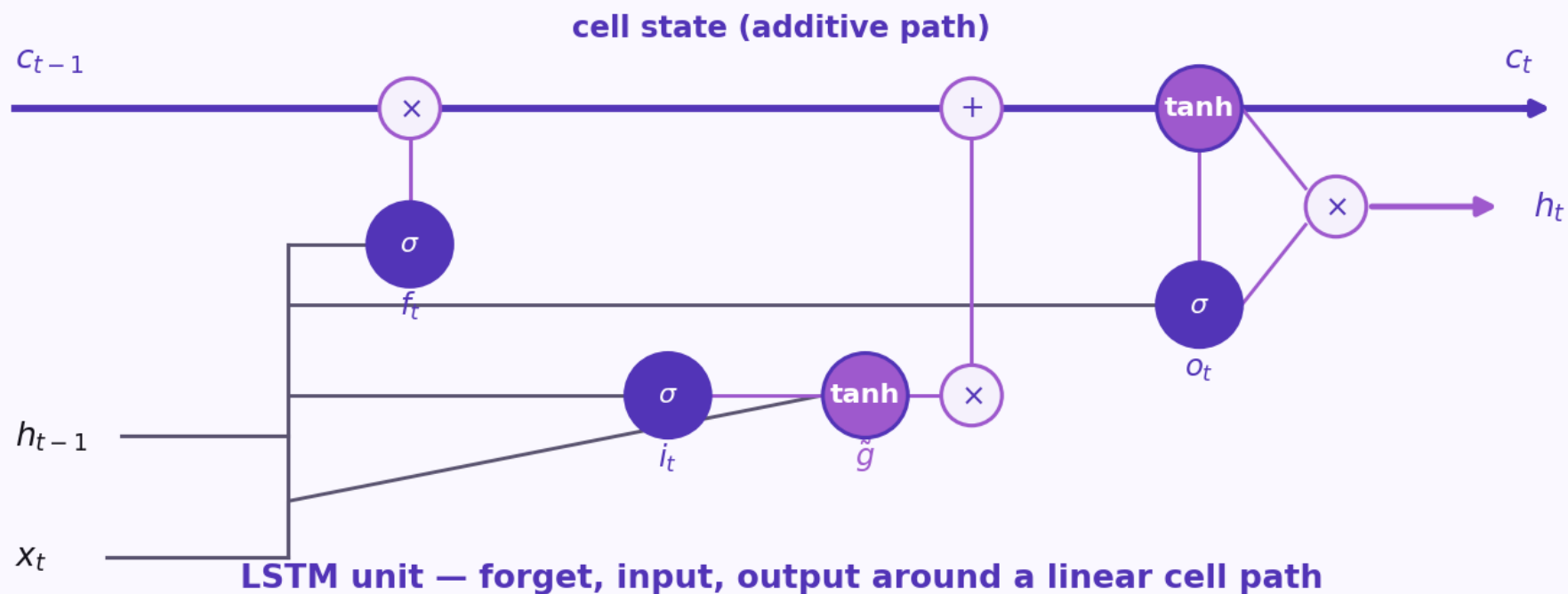
Sequential Data & Vanilla RNNs

One Transition, Unfolded Through Time

Feature	Limitation
Reuses parameters	Sequential execution (cannot parallelize steps)
Tracks state h_t	Cannot hold long-term memories in practice

LSTM: Gates for Long-Term Memory

Forget, Input, Output — Cell State Flows Linearly



LSTM: Gates for Long-Term Memory

Forget, Input, Output — Cell State Flows Linearly

- Cell state c_t can accumulate information with an additive path—mitigates vanishing signal compared with plain tanh RNNs.
- Forget gate f_t decides how much past cell content to erase; input gate i_t and candidate $\tilde{g}$ control new information written.
- Output gate o_t filters what becomes hidden state h_t exposed to the next layer or timestep.

LSTM: Gates for Long-Term Memory

Forget, Input, Output — Cell State Flows Linearly

Gate	Role (intuition)
f_t	Erase irrelevant history from the cell
i_t	Allow new candidate information in
o_t	Expose part of the cell as h_t

Phase 4: Generative Models & Reconstruction

Generative Models and Representation Learning

- Phase 4 explores unsupervised representation learning and basic generative frameworks.
- Autoencoders are the foundation here: they learn to compress inputs to a low-dimensional bottleneck (latent space) and reconstruct them.

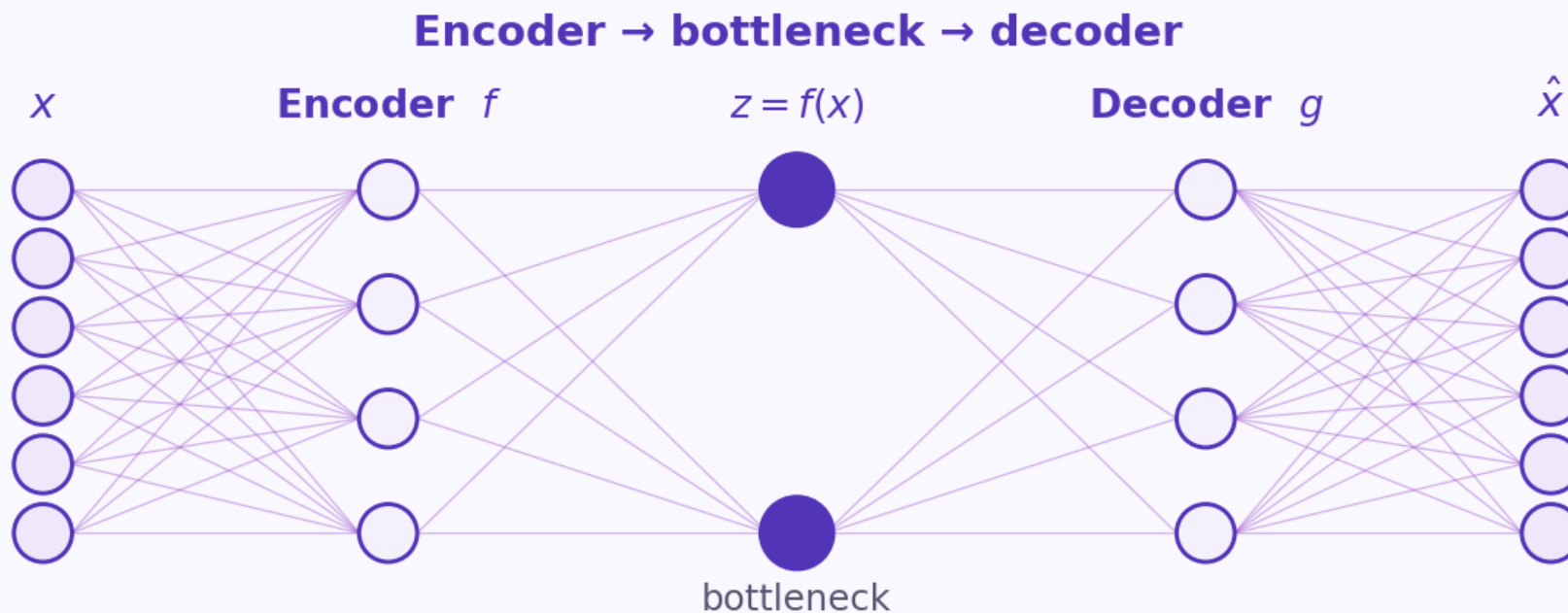
Phase 4: Generative Models & Reconstruction

Generative Models and Representation Learning

Concept	Input	Goal
Autoencoder	Unlabeled data x	Compress and reconstruct $\hat{x} \approx x$

Autoencoders (AE): Compression and Reconstruction

Encoder → Bottleneck → Decoder



Autoencoders (AE): Compression and Reconstruction

Encoder → Bottleneck → Decoder

- Encoder f maps input x to a low-dimensional code $z = f(x)$; decoder g maps z back to $\hat{x} \approx x$.
- The bottleneck forces a compressed representation—useful for denoising (train on noisy→clean), anomaly detection (high recon error = abnormal).
- Linear AE with MSE relates to PCA when constraints align—nonlinear AE learns curved manifolds.

Autoencoders (AE): Compression and Reconstruction

Encoder → Bottleneck → Decoder

Variant	Idea
Denoising AE	Corrupt input, reconstruct clean—robust features
Sparse AE	Penalty on activations—encourages informative sparse codes
Contractive AE	Penalize sensitivity of code to small input changes—smoother latent map

3 Ideas to Keep

Leave with these

- 1 A neuron is $z = w^T x + b$ then $a = f(z)$. Depth needs nonlinearity between layers.
- 2 Backprop is the chain rule on a graph. Watch vanishing gradients, LR, and train vs val.
- 3 CNNs share filters across space; RNNs share weights across time; regularize so they generalize.

Thank you

Labs: code/15- Deep Learning

Week 3 · Session 1

NLP

NLP Workshop: Foundations to Seq2Seq

Where Are We in the Bootcamp?

Week 3 Session 1 — NLP from raw text to Seq2Seq.

Week 1

- S1 Foundations & Preprocessing
- S2 Regression Models
- S3 Classification Basics
- S4 Naive Bayes & Trees
- S5 SVM & Kernels
- S6 Clustering & PCA

Week 2

- S7 Deep Learning

Week 3

- S8 **NLP Fundamentals**
- S9 Tokenization
- S10 Text Analysis & NER
- S11 Language Modeling
- S12 Embeddings & RNNs
- S13 Seq2Seq & NMT

Week 4

- S14 Generative AI
- S15 RAG Systems
- S16 MLOps

Week 3 · Session 1

NLP

Foundations · Tokenization · Seq2Seq

NLP Fundamentals

Regex & Cleaning

Tokenization

POS & NER

Language Modeling

Embeddings & RNNs

Seq2Seq & Attention

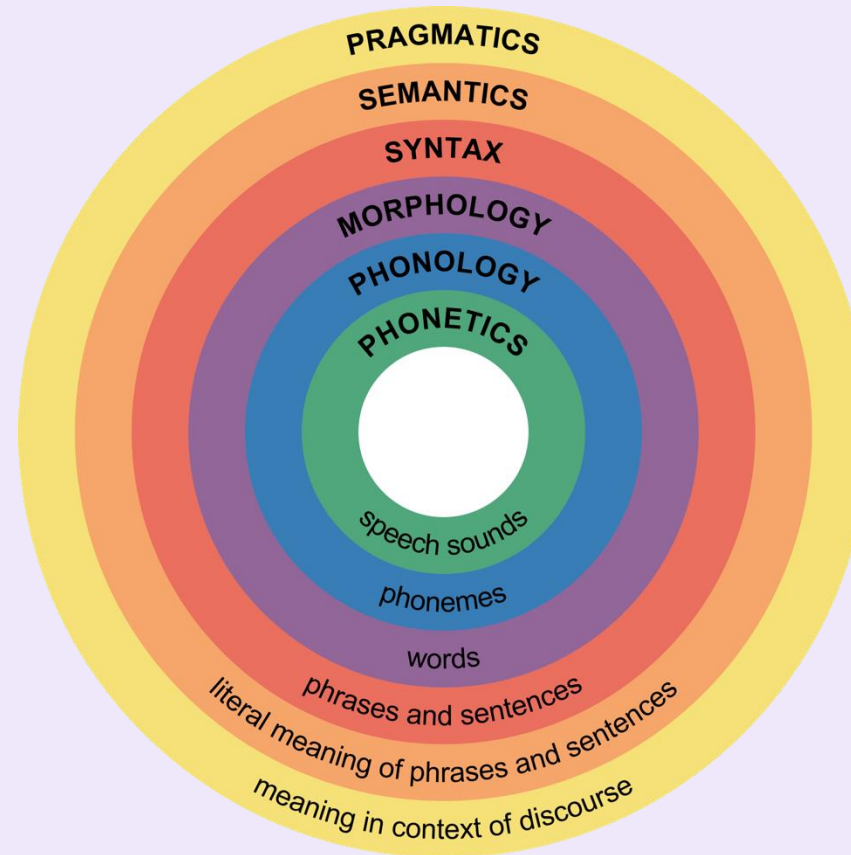
Foundations · Tokenization · Seq2Seq

What we will cover in this session

- NLP Fundamentals
- Regex & Cleaning
- Tokenization
- POS & NER
- Language Modeling
- Embeddings & RNNs
- Seq2Seq & Attention

NLP Fundamentals and Challenges

Why Natural Language Is Difficult for Machines



NLP Fundamentals and Challenges

Why Natural Language Is Difficult for Machines

- Natural language is ambiguous, context-dependent, and full of idioms.
- Meaning often depends on pragmatics, domain, and cultural background.
- The same sentence can map to multiple valid interpretations.

NLP systems must model form, meaning, and context simultaneously.

Ambiguity in Language: Practical Examples

One string, two valid interpretations

"The bank is closed"



A · financial institution

B · river bank

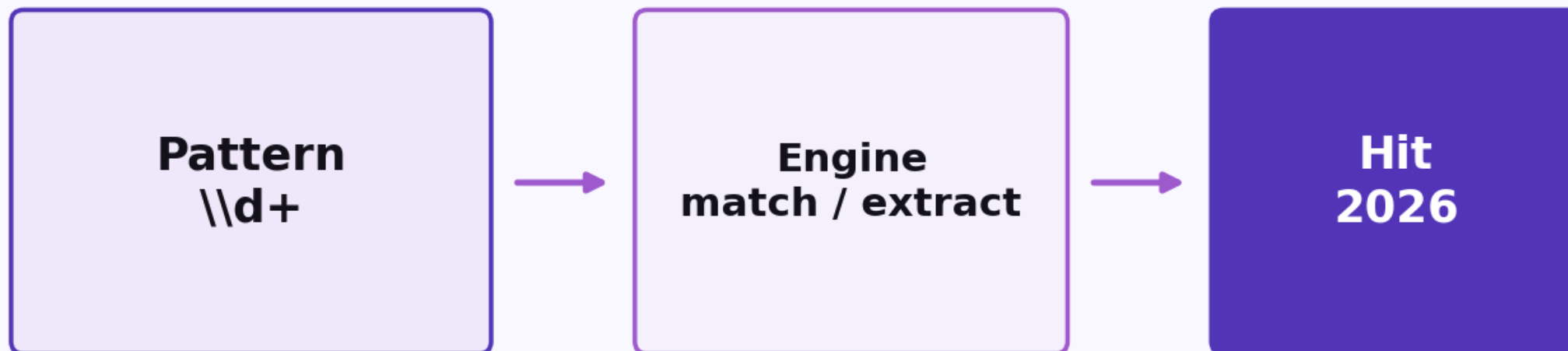
Ambiguity in Language: Practical Examples

Expression	Possible Interpretation A	Possible Interpretation B
The bank is closed	Financial institution is closed	River bank area is inaccessible
It's cold	Low temperature	Emotionally distant behavior
The chicken is ready to eat	Food is ready to be eaten	Animal is ready to eat food

Robust NLP requires disambiguation using surrounding textual and situational context.

Regular Expressions (Regex): Core Idea

Regex is a compact pattern language for text



Regular Expressions (Regex): Core Idea

- Regex is a compact pattern language for matching and transforming text.
- Typical uses include validation, extraction, and rule-based cleanup.
- Useful for emails, phone numbers, logs, and normalization pipelines.

Regular Expressions (Regex): Core Idea

Pattern	Meaning	Example Match
\d+	one or more digits	2026
\w+	one or more word chars	nlp_model1
^[A-Z]	starts with uppercase letter	Hello
\S+	one or more non-space chars	token123

Text Cleaning — Conceptual Steps & Python Code

Conceptual stages matched with Python regex implementations

Cleaning pipeline — raw web text → tokenizer-ready string



Each stage is usually one regex (or encode) step

Text Cleaning — Conceptual Steps & Python Code

Conceptual stages matched with Python regex implementations

Stage	Python Implementation	Goal
1. Decode & Clean HTML	<code>html.unescape(text) & re.sub(r'<[^>]+>', '', text)</code>	Keep readable text only
2. Remove Contacts/URLs	<code>re.sub(r'https?://\S+ www\.\S+', '', text)</code>	Drop URL metadata
3. Remove Mentions/Socials	<code>re.sub(r'@\w+ #\w+', '', text)</code>	Remove handles & hashtags
4. Drop Non-ASCII & Emoji	<code>text.encode('ascii', 'ignore').decode('ascii')</code>	Drop emoji/special symbols
5. Normalize Whitespace	<code>re.sub(r'\s+', ' ', text).strip()</code>	Normalize extra spaces

A regex-based cleaning function is usually the first stage in an NLP preprocessing pipeline.

Text Cleaning — Worked Example

Trace of raw input transitioning to clean text

Stage	Operation performed	Text snapshot
0. Raw Input	Original raw string	John: "AI is amazing!!!! 🤖🔥" Visit: https://ai.com @john #AI
1. Clean HTML	Decode entities & remove tags	John: "AI is amazing!!!! 🤖🔥" Visit: https://ai.com @john #AI
2. Remove URLs	Remove web addresses	John: "AI is amazing!!!! 🤖🔥" Visit: @john #AI
3. Remove Socials	Remove @mentions and #hashtags	John: "AI is amazing!!!! 🤖🔥" Visit:
4. Drop Emoji/Symbols	ASCII-only encoding & cleanup	John AI is amazing!!!! Visit

Text cleaning produces standard, noise-free input for tokenizer components.

Tokenization Granularities (Word, Char, Subword)

Granularity	Strength	Tradeoff	Example output (for 'running')
Word / Space-based	Human-readable tokens	Weak on rare/compound words (OOV)	['running']
Character-based	No unknown words (no OOV)	Very long sequences, loss of meaning	['r', 'u', 'n', 'n', 'i', 'n', 'g']
Subword-based	Handles rare words, compact vocab	Requires tokenizer training/tuning	['run', '##ning']

Subword tokenization is the industry standard for modern LLMs and neural NLP models.

Tokenization Views (Code Examples)

Same sentence, word / char / subword splits side by side

Tokenizer Type	Python Implementation	Result on "I'm running to the store!"
Word	<code>word_tokenize(text)</code>	<code>['I', "'m", 'running', 'to', 'the', 'store', '!']</code>
Character	<code>list(text)</code>	<code>['I', "'", 'm', ' ', 'r', 'u', 'n', 'n', 'i', 'n', 'g', ' ', 't', 'o', ' ', 't', 'h', 'e', ' ', 's', 't', 'o', 'r', 'e', '!', '!']</code>
Subword	<code>model_tokenizer.tokenize(text)</code>	<code>['I', "'m", 'run', '##ning', 'to', 'the', 'store', '!']</code>
Sentence	<code>sent_tokenize(text)</code>	<code>["I'm running to the store!"]</code>

Subword Tokenization (LLMs) — Why It Wins

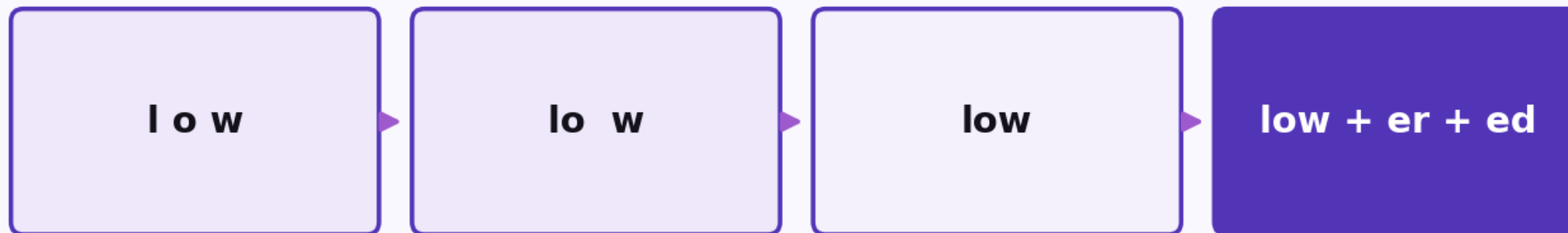
How subwords solve core vocabulary challenges

Word Tokenizer Issue	Subword Solution	Example Worked Out
Huge vocabulary size	Reuse frequent morphemes/pieces	unbelievably → un + ##believe + ##ably
Out-of-Vocabulary (OOV) words	Break unknown forms into known units	ChatGPT → ['Chat', 'G', 'PT']
Morphological variants	Shared roots reduce feature sparsity	running/runs/runner share 'run'

BPE Walkthrough & Inference Example

How Byte Pair Encoding learns merges and splits unseen words

BPE: merge frequent pairs, then split unseen words



lowered → ['low', 'er', 'ed']

BPE Walkthrough & Inference Example

How Byte Pair Encoding learns merges and splits unseen words

- Walkthrough: start with chars (l, o, w, e, s, t, r) -> merge l+o (lo) -> merge lo+w (low) -> compact vocabulary.
- Inference on 'lowered the lowest curtain': splits rare words to ['low', 'er', 'ed'] and ['low', 'est'] while keeping 'the' and 'curtain' whole.

BPE Walkthrough & Inference Example

How Byte Pair Encoding learns merges and splits unseen words

Word	Subword Output	Explanation
lowered	['low', 'er', 'ed']	Decomposes into learned root and suffix subwords
lowest	['low', 'est']	Decomposes into base 'low' and suffix 'est'

BPE repeatedly merges frequent adjacent units until target vocabulary size is reached.

Stop Words: Purpose & Tradeoffs

Filtering low-information tokens from texts

- Function words (the, is, and) dominate word counts but carry minimal topic signal.
- Removing stop words reduces dimensionality for TF-IDF & Bag-of-Words.
- Critical Rule: NEVER remove stop words for language models (GPT/BERT), translation, or sentiment analysis where negation ('not') and syntax carry the core meaning.

Stop Words: Purpose & Tradeoffs

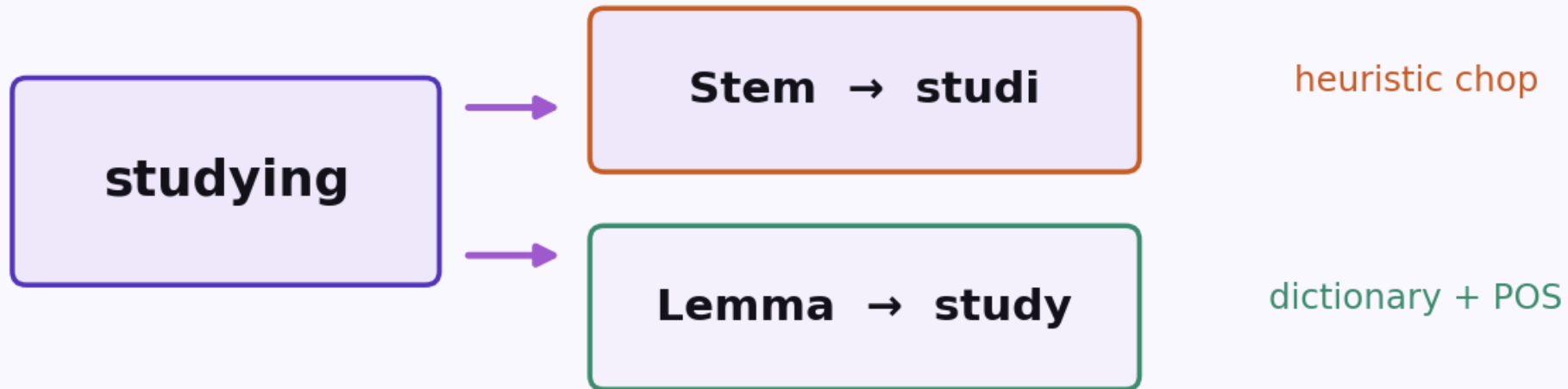
Filtering low-information tokens from texts

Task Category	Stop Word Policy	Reasoning
Search & Topic Modeling	Remove	Improves speed and focus on key words
Large Language Models	Keep	Syntactic and semantic flow is vital
Sentiment Analysis	Keep	Negations like 'not' reverse polarity

Stemming vs Lemmatization

Normalizing words to their base forms

Stemming is fast; lemmatization keeps real words



Stemming vs Lemmatization

Normalizing words to their base forms

Aspect	Stemming	Lemmatization
Mechanism	Rule-based suffix chopping (heuristic)	Dictionary lookup + morphological analysis
Speed	Very fast, computationally cheap	Slower, requires resource lookup
Output	Can be non-words (e.g., study -> studi)	Always dictionary words (e.g., study -> study)
POS Awareness	No	Yes (e.g., saw -> see or saw depending on POS)

Lemmatization preserves semantics better than aggressive stemming.

Part-of-Speech (POS) Tagging

Grammar Labels per Token

POS tagging — a grammar label on every token

Tesla

PROPN

is

AUX

building

VERB

cars

NOUN

quickly

ADV

Part-of-Speech (POS) Tagging

Grammar Labels per Token

- Universal categories include NOUN, VERB, ADJ, ADV, DET, ADP, PROPN, NUM, ...
- POS feeds lemmatization quality and downstream relation extraction.
- SpaCy exposes coarse pos_ and fine-grained tag_ plus dependency dep_.

Part-of-Speech (POS) Tagging

Grammar Labels per Token

POS	Role	Quick examples
NOUN / PROPN	Things and names	city, Tesla
VERB / AUX	Actions and helpers	run, is
ADJ / ADV	Modifiers	quick, quickly
DET / ADP	Structure words	the, in

Named Entity Recognition (NER)

Typed Spans over Text

NER — typed spans over text, not one tag per word

Tim Cook
PERSON

Apple
ORG

\$430B
MONEY

Named Entity Recognition (NER)

Typed Spans over Text

- Labels include PERSON, ORG, GPE, DATE, MONEY, PERCENT, PRODUCT, EVENT, ...
- Useful for indexing, compliance redaction, financial news graphs, and search facets.
- Small models err on edge cases — always spot-check domain text.

Named Entity Recognition (NER)

Typed Spans over Text

Label	Examples
PERSON	Tim Cook, Jensen Huang
ORG / GPE	Apple, Texas
MONEY / PERCENT	\$430 billion, 3.5%

Full SpaCy Pipeline Functionally

One SpaCy doc carries tokens, lemmas, POS, and entities

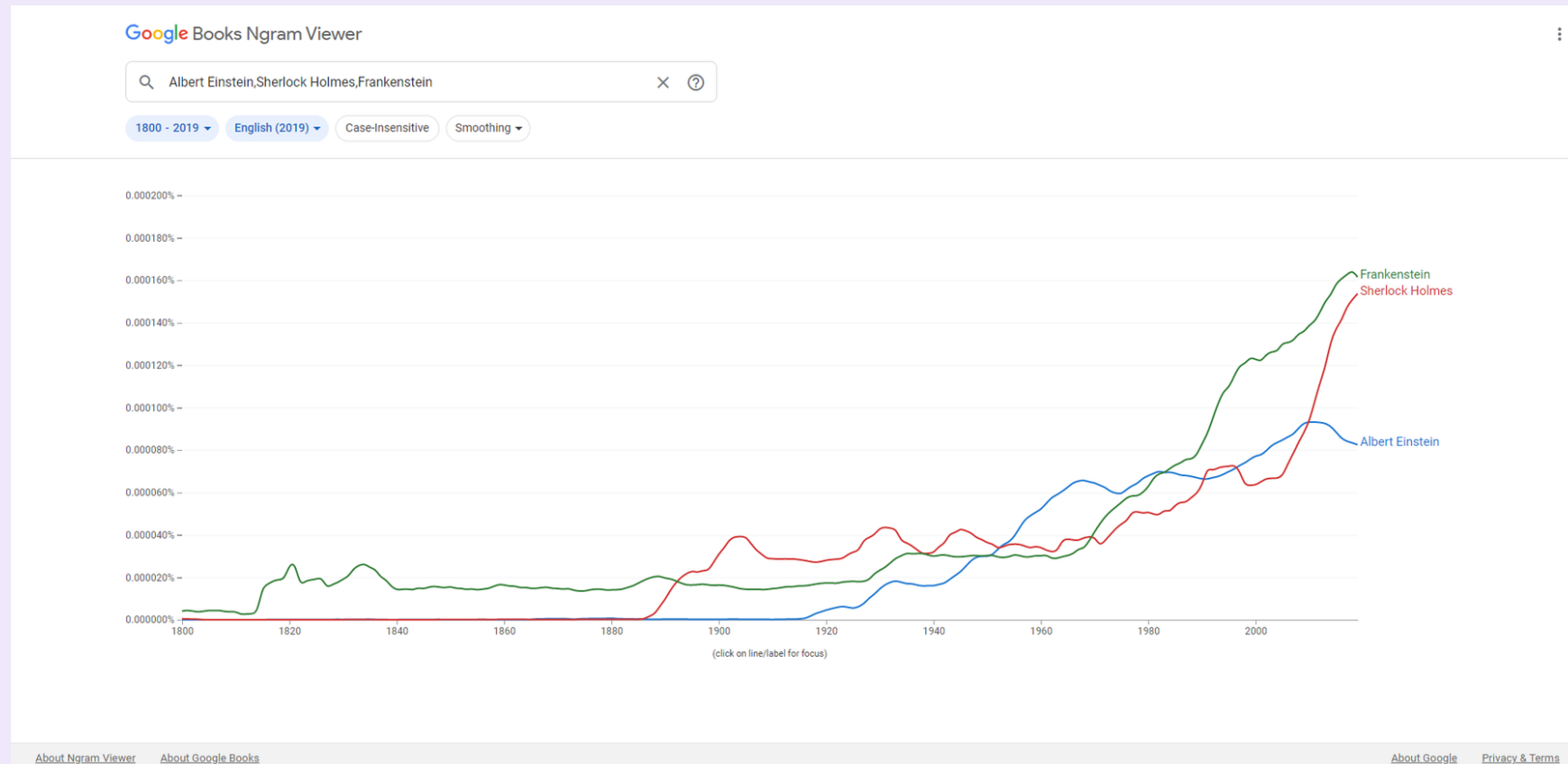


Full SpaCy Pipeline Functionally

- One doc object: tokens, lemmas, POS, entities, noun_chunks, sents.
- Typical export: clean token list, lemma bag, entity list, POS histogram.
- Use length-based heuristics (e.g. longest sentence) only as weak importance cues.

NLP Language Modeling with N-grams

From Count-Based Prediction to Evaluation



NLP Language Modeling with N-grams

From Count-Based Prediction to Evaluation

- Language models estimate probabilities over word sequences.
- N-gram models predict the next token from a limited context window.
- This module connects N-gram intuition to modern LLM generation.

Formal Language Modeling Objective

N-gram LM: predict the next token from a short window



Formal Language Modeling Objective

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

- A language model can score full sentences or predict the next word.
- Exact estimation over full histories is intractable for real corpora.
- Approximation strategies include N-grams, smoothing, and neural LMs.

Worked Bigram Example

Given counts: $C(\text{"want to"}) = 608$, $C(\text{"want"}) = 927$

$$P(\text{to} \mid \text{want}) = \frac{608}{927} \approx 0.656$$

- Interpretation: in this corpus, "to" follows "want" about 65.6% of the time.
- Count quality depends strongly on corpus domain and size.

Why Use Log Probabilities

Score sequences in log space

Product space

$$p_1 \times p_2 \times \dots$$

underflow

Log space

$$\log p_1 + \log p_2 + \dots$$

stable sums

Why Use Log Probabilities

- Multiplying many small probabilities causes numerical underflow.
- Log transform converts multiplication into stable addition.
- Sequence scoring becomes computationally robust and efficient.

Why Use Log Probabilities

Original Space	Log Space
$P = p_1 * p_2 * ... * p_k$	$\log P = \log p_1 + \log p_2 + ... + \log p_k$
Very small numbers	Numerically stable sums

Perplexity (PP): Interpretation

$$\text{PP}(W) = P(w_1, \dots, w_T)^{-\frac{1}{T}} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log P(w_t | h_t)\right)$$

- Perplexity is the average branching uncertainty of the model.
- Lower PP means better predictive confidence on unseen text.
- Use identical test sets when comparing models.

Raw sentence probabilities are length-sensitive; perplexity normalizes by token count.

Static vs Contextualized Word Representations

Polysemy needs context, not a single lookup table

Static · one vector

bank

river and finance collide

Contextual · depends on neighbors

bank

river

bank

finance

Static vs Contextualized Word Representations

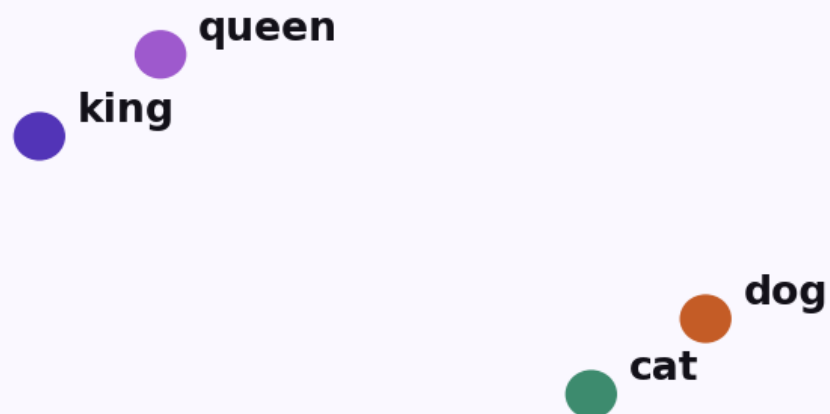
Property	Static Embeddings (Word2Vec/GloVe)	Contextualized Embeddings (ELMo/BERT/GPT)
Vector per word	One fixed vector	Different vectors per context
Polysemy handling	Weak	Strong
Context direction	Usually local/global corpus only	Bidirectional or autoregressive sequence context
Task transfer	Moderate	High with pretraining + fine-tuning

Contextualization is a core enabler for modern LLM quality.

How Contextualized Embeddings Work

Words as vectors in embedding space

Words as vectors in embedding space



How Contextualized Embeddings Work

Words as vectors in embedding space

- Static embeddings: one vector per word type.
- Contextual: vector depends on surrounding tokens (ELMo, BERT-style).

How Contextualized Embeddings Work

Architecture Layers

- Token/base embedding layer maps input ids to dense vectors.
- Context encoder layers (often Transformer blocks) enrich each token with sequence context.
- Task head uses contextual features for prediction.

How Contextualized Embeddings Work

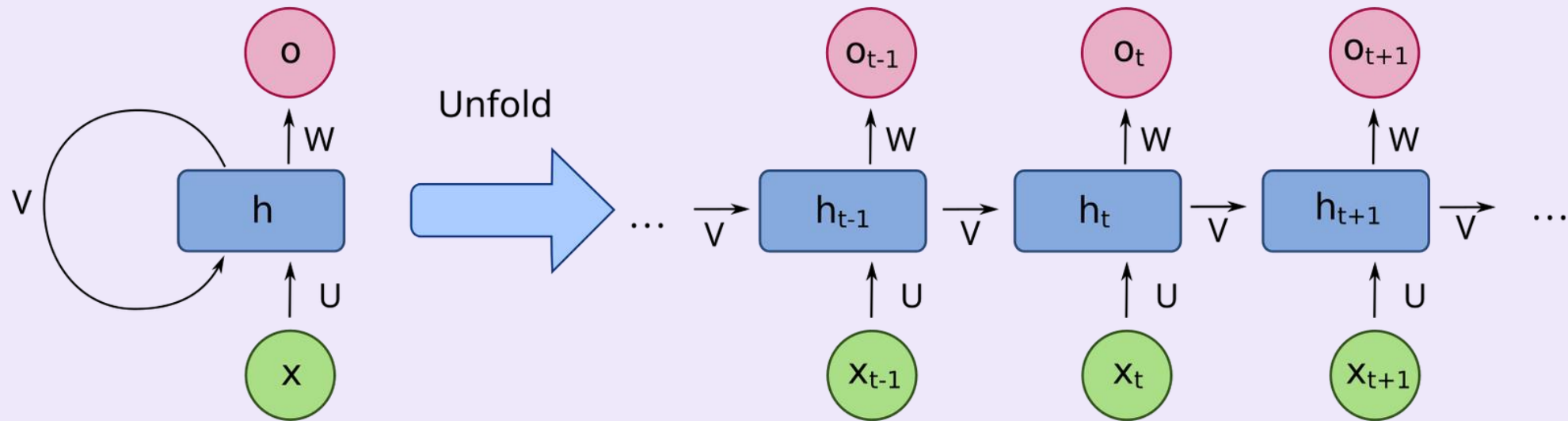
Layer Semantics

- Lower layers capture local syntax/patterns.
- Middle layers improve sense disambiguation.

RNN Mechanism and Weight Sharing

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

RNN Mechanism and Weight Sharing



Common RNN Input/Output Patterns

RNN input/output patterns

1→1
one in, one out

1→many
captioning

many→1
sentiment

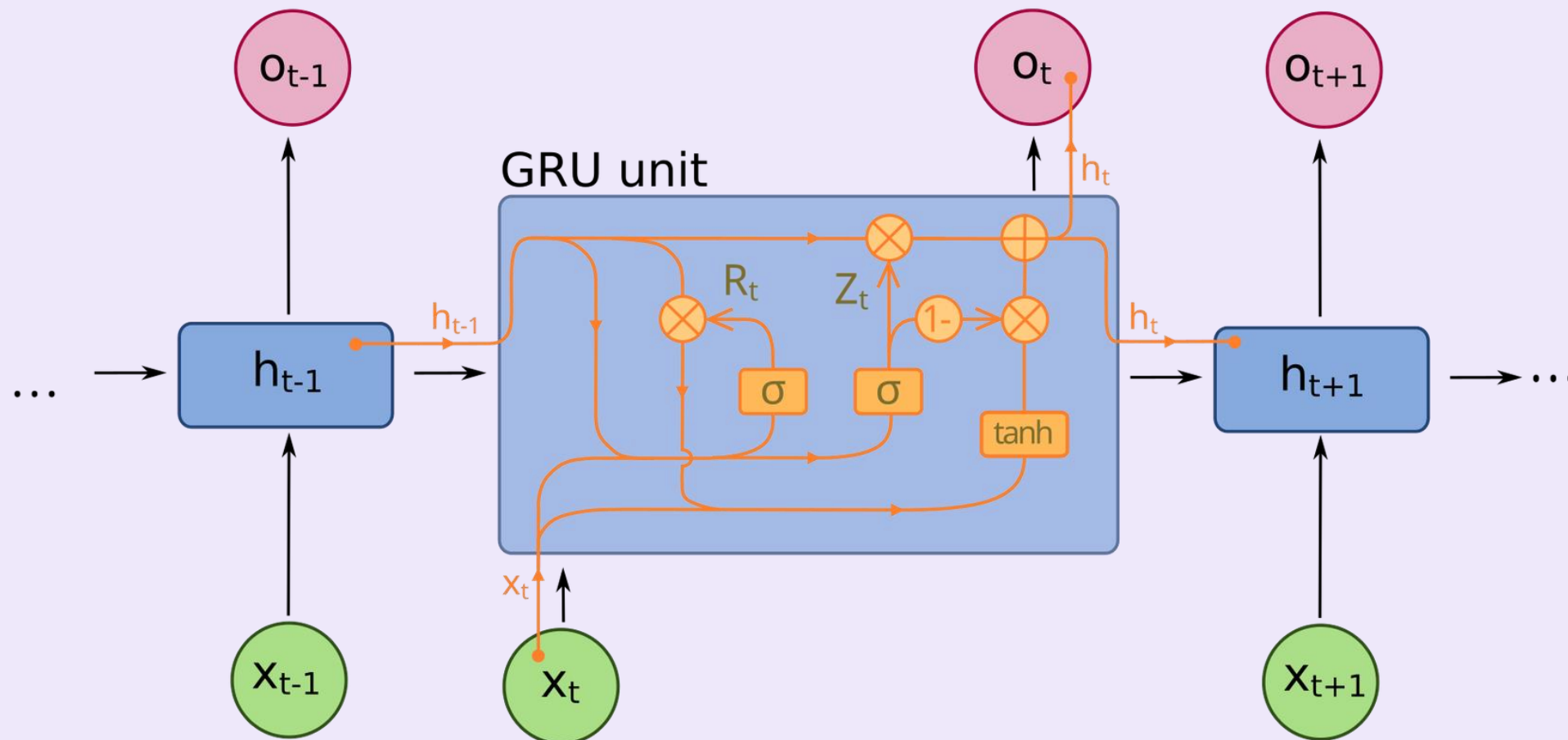
many→many
translation

Common RNN Input/Output Patterns

Pattern	Mapping	Example Task
One-to-One	single input -> single output	basic regression/classification
One-to-Many	single input -> sequence output	image captioning
Many-to-One	sequence input -> single output	sentiment classification
Many-to-Many	sequence input -> sequence output	machine translation

Sequence-aware patterns are where recurrent models provide clear value.

RNN Family: Vanilla, GRU, BiRNN, LSTM



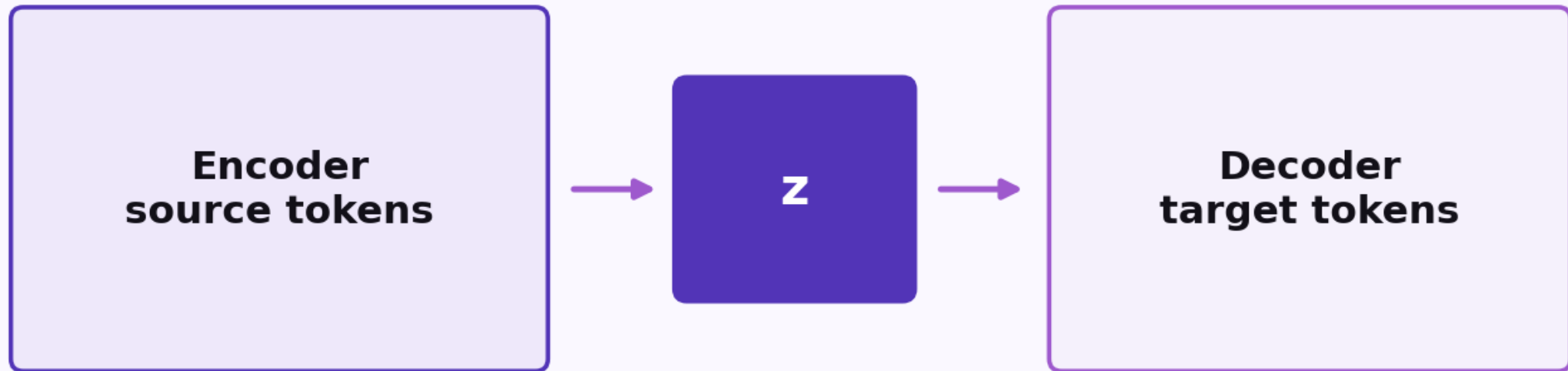
RNN Family: Vanilla, GRU, BiRNN, LSTM

Model	Vanishing Gradient Risk	Key Strength	Typical Limitation
Vanilla RNN	High	Simple architecture	Poor long-range memory
GRU	Lower	Efficient gating with fewer parameters	Can still degrade on very long contexts
BiRNN	Medium	Uses past and future context	Higher compute and memory
LSTM	Low (relative)	Strong long-term dependency handling	Heavier than GRU

NLP Seq2Seq for Neural Machine Translation

Encoder-Decoder Modeling, Decoding Strategies, and Evaluation

Seq2Seq maps variable length → variable length



NLP Seq2Seq for Neural Machine Translation

Encoder-Decoder Modeling, Decoding Strategies, and Evaluation

- Seq2Seq maps variable-length input sequences to variable-length outputs.
- Encoder-decoder models were foundational for neural machine translation.
- This module covers training, decoding, bottlenecks, attention, and metrics.

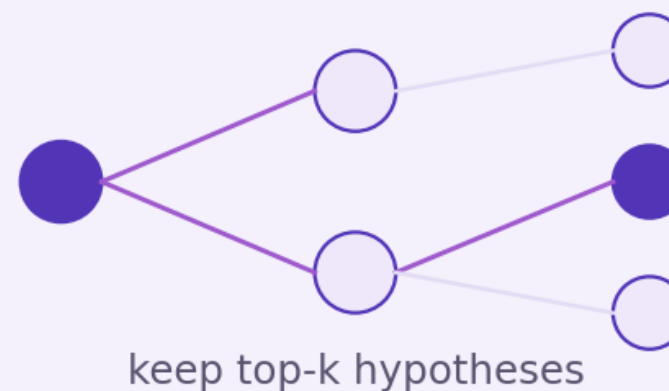
Inference: Greedy Decoding vs Beam Search

Decoding: local pick vs shortlist of sequences

Greedy



Beam search



Inference: Greedy Decoding vs Beam Search

Method	Decision Rule	Strength	Risk
Greedy	Pick top token each step	Fast and simple	May miss globally better sequence
Beam Search	Track top-k hypotheses each step	Better global sequence quality	Higher compute; can become generic/bland

Typical beam sizes are moderate (e.g., 4-10) to balance quality and cost.

Information Bottleneck in Basic Seq2Seq

Attention removes the fixed-vector bottleneck

Bottleneck



one vector for the whole source

Attention



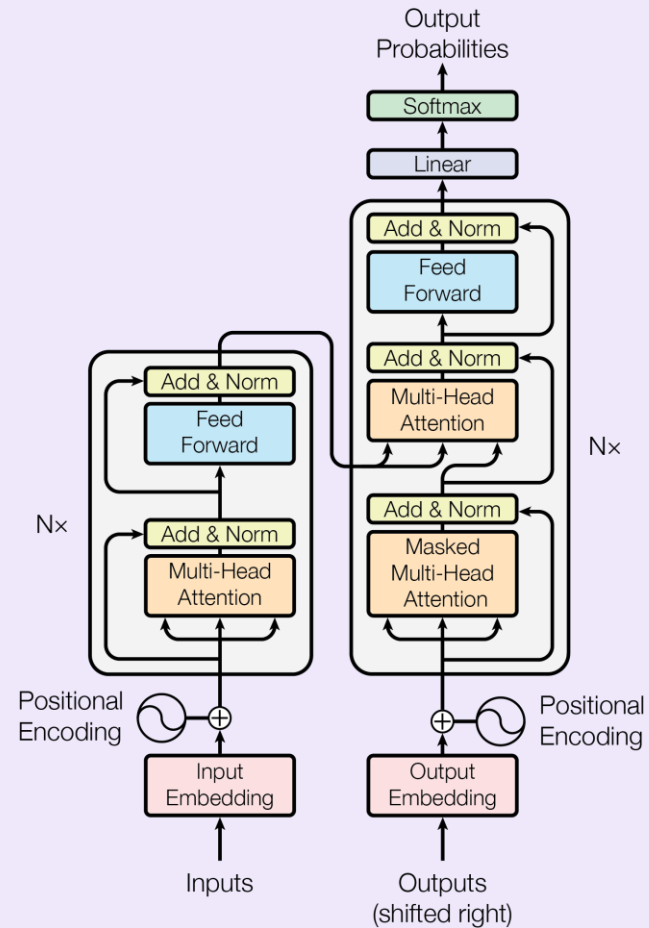
decoder looks at relevant encoder states

Information Bottleneck in Basic Seq2Seq

- Compressing a full source sentence into one fixed vector can lose detail.
- Longer/complex inputs worsen the bottleneck effect.
- Decoder needs different source details at different output steps.

This motivates attention over all encoder hidden states.

Attention as the Bottleneck Solution



Attention as the Bottleneck Solution

- Decoder attends to relevant encoder positions at each generation step.
- Dynamic alignment improves translation adequacy and fluency.
- Attention laid the foundation for transformer-dominant NMT systems.

NMT Evaluation Metrics: BLEU and ROUGE

Metric	Orientation	What It Measures	Common Limitation
BLEU	Precision-oriented	n-gram overlap from candidate to reference	Weak semantic/syntactic sensitivity
ROUGE-N	Recall-oriented	n-gram overlap from reference to candidate	Can reward lexical overlap over meaning
F1 (with overlap metrics)	Balance precision/recall	Harmonic tradeoff view	Still limited on deep semantics

Transformer Encoder-Decoder Overview

Bridge to Week 4 GenAI — encoder-decoder intuition before BERT/GPT

Transformer encoder-decoder — bridge to Week 4 GenAI

**Encoder
contextual source states**



**Decoder
autoregressive target + attention**

Transformer Encoder-Decoder Overview

Bridge to Week 4 GenAI — encoder-decoder intuition before BERT/GPT

- Encoder builds contextual token representations from source text.
- Decoder generates target tokens autoregressively with attention to encoder states.
- Foundation architecture for translation, summarization, and many LLM pipelines.

Attention mechanisms reduce reliance on recurrence for sequence modeling.

3 Ideas to Keep

Leave with these

- 1 Language is ambiguous — cleaning and tokenization choices persist through the pipeline.
- 2 N-grams and RNNs model sequences; attention removes the seq2seq bottleneck.
- 3 Subword tokenizers and encoder–decoder attention are the bridge to Transformers.

Thank you

Labs: code/16- NLP

Week 4 · Session 1

GenAI

GenAI: BERT, T5, GPT, Attention

Where Are We in the Bootcamp?

Week 4 Session 1 — GenAI: BERT, T5, GPT, Attention.

Week 1

- S1 Foundations & Preprocessing
- S2 Regression Models
- S3 Classification Basics
- S4 Naive Bayes & Trees
- S5 SVM & Kernels
- S6 Clustering & PCA

Week 2

- S7 Deep Learning

Week 3

- S8 NLP Fundamentals
- S9 Tokenization
- S10 Text Analysis & NER
- S11 Language Modeling
- S12 Embeddings & RNNs
- S13 Seq2Seq & NMT

Week 4

- S14 Generative AI**
- S15 RAG Systems
- S16 MLOps

Week 4 · Session 1

GenAI

BERT · T5 · GPT · Attention

Core Concepts

Transfer Learning

BERT

T5

GPT

Attention

Scaling & Practice

BERT · T5 · GPT · Attention

What we will cover in this session

- Core Concepts
- Transfer Learning
- BERT
- T5
- GPT
- Attention
- Scaling & Practice

Generative AI: Core Concepts

BERT, T5, GPT, LLMs, and Attention (Focused Edition)

Choose architecture by task family

BERT

encoder-only

bidirectional
understanding

T5

encoder-decoder

text-to-text
multitask

GPT

decoder-only

causal
generation

Generative AI: Core Concepts

BERT, T5, GPT, LLMs, and Attention (Focused Edition)

- This section extracts the most important ideas into a compact teaching flow.
- Focus areas: transfer learning, model families, attention mechanics, scaling, and practical deployment.
- Designed for bootcamp delivery within roughly 20 slides.

Transfer Learning in Generative AI

Transfer learning stages



unlabeled corpora → task labels → user prompt

Transfer Learning in Generative AI

Stage	Data Type	Goal
Pretraining	Large mostly unlabeled corpora	Learn general language representations
Fine-tuning	Task-labeled dataset	Adapt model to specific downstream objective
Inference	User prompt/context	Generate or classify outputs for real tasks

Transfer learning reduces labeled data needs and accelerates convergence.

Feature-Based Transfer vs Fine-Tuning

What changes in the pretrained model

Feature-based

frozen backbone

embeddings as features

Fine-tuning

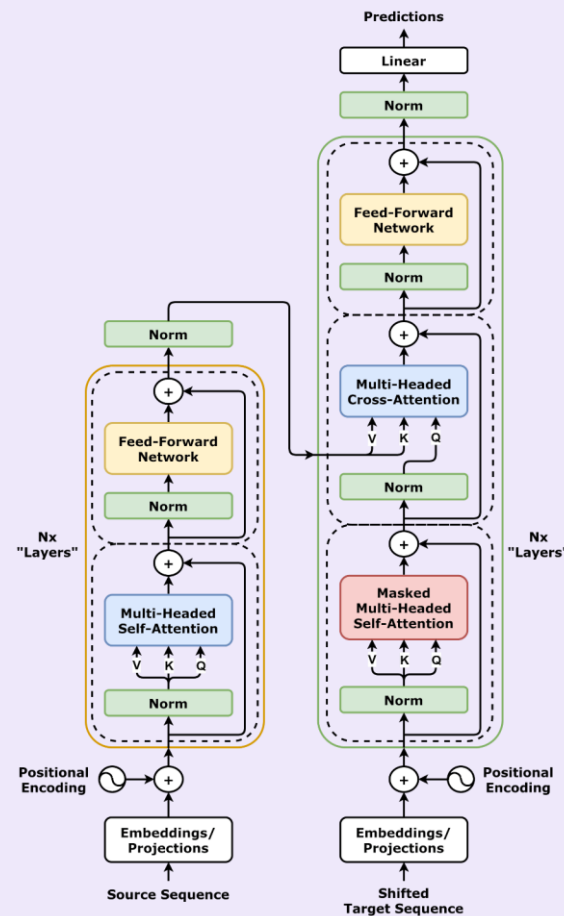
update weights + task head

more adaptation, more compute

Feature-Based Transfer vs Fine-Tuning

Strategy	What Changes	Pros	Tradeoff
Feature-based	Freeze pretrained backbone; use embeddings as input features	Fast and lightweight	Less task adaptation
Fine-tuning	Update model weights (often with a task head)	Higher task performance	More compute and tuning effort

BERT: Encoder-Only Bidirectional Understanding



BERT: Encoder-Only Bidirectional Understanding

- BERT learns contextual token representations using both left and right context.
- Strong for understanding tasks: classification, NER, QA, and retrieval-oriented use cases.
- Pretrained at scale (Wikipedia + BooksCorpus), then adapted to downstream tasks.

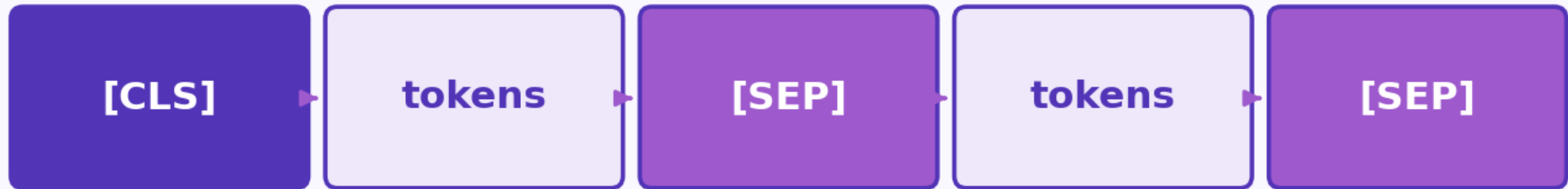
BERT Pretraining Objectives

Objective	How It Works	What It Teaches
MLM (Masked Language Modeling)	Mask subset of tokens and predict originals	Bidirectional contextual semantics
NSP (Next Sentence Prediction)	Predict whether sentence B follows sentence A	Inter-sentence coherence signals

MLM and NSP were jointly used in original BERT training setup.

BERT Special Tokens: [CLS] and [SEP]

[CLS] and [SEP] unify single- and pair-sentence inputs



[CLS] = sequence summary · [SEP] = segment boundary

BERT Special Tokens: [CLS] and [SEP]

- [CLS] is prepended and its final embedding is used as a sequence-level summary.
- [SEP] separates sentence segments and marks boundaries.
- In classification tasks, [CLS] passes through a lightweight prediction head.

Token design helps unify single- and pair-sentence tasks.

Arabic Adaptation Example: AraBERT

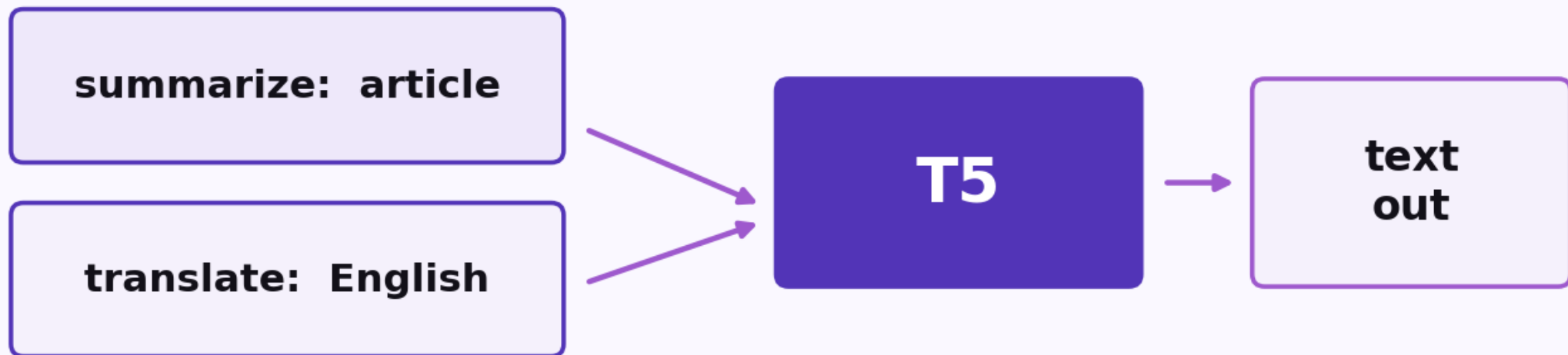
- AraBERT follows BERT-base style architecture with Arabic-specific preprocessing.
- Segmentation and Arabic-tailored tokenization improve vocabulary coverage.
- Domain/language adaptation quality can matter more than raw parameter count.

Arabic Adaptation Example: AraBERT

Component	Adaptation Benefit
Farasa-style segmentation	Better handling of prefixes/suffixes and morphology
Arabic-focused vocabulary	Lower fragmentation and stronger lexical coverage
Arabic corpus curation	Improved contextual understanding in target language

T5: Unified Text-to-Text Framework

Every NLP task is text input → text output



T5: Unified Text-to-Text Framework

- T5 reframes every NLP task as text input -> text output.
- Task instruction is included as a prefix (e.g., "summarize:", "translate:").
- One shared architecture/training recipe serves multiple tasks.

This design strongly influenced modern prompt-based LLM usage.

T5 Pretraining Objective: Span Corruption

Corruption style: tokens vs spans

BERT · token mask



random individual tokens

T5 · span corruption



contiguous span + sentinel

T5 Pretraining Objective: Span Corruption

- Remove contiguous text spans from input and replace with sentinel tokens.
- Train model to reconstruct missing spans autoregressively.
- Encourages stronger generative behavior than token-only masking.

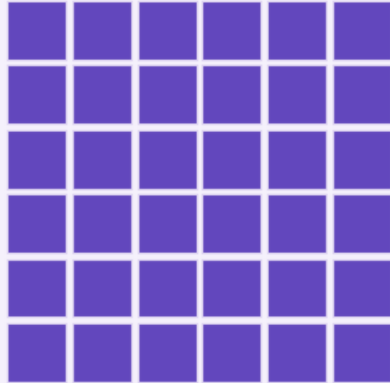
T5 Pretraining Objective: Span Corruption

Model	Corruption Style	Typical Strength
BERT	Random masked individual tokens	Language understanding tasks
T5	Contiguous span corruption	Generation + multitask transfer

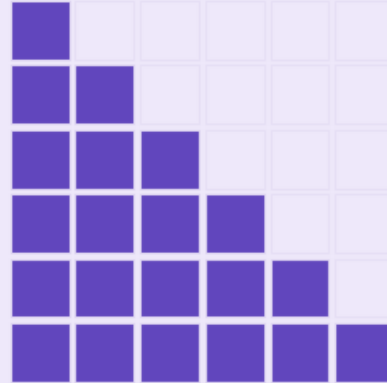
T5 Attention Strategies (Encoder/Decoder)

T5 attention visibility

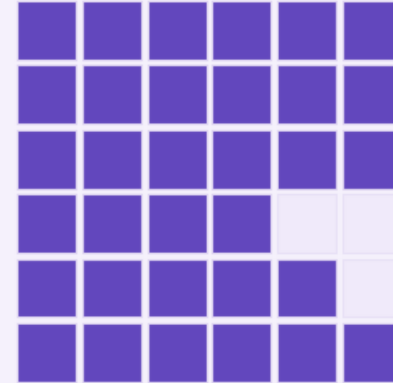
Fully visible



Causal



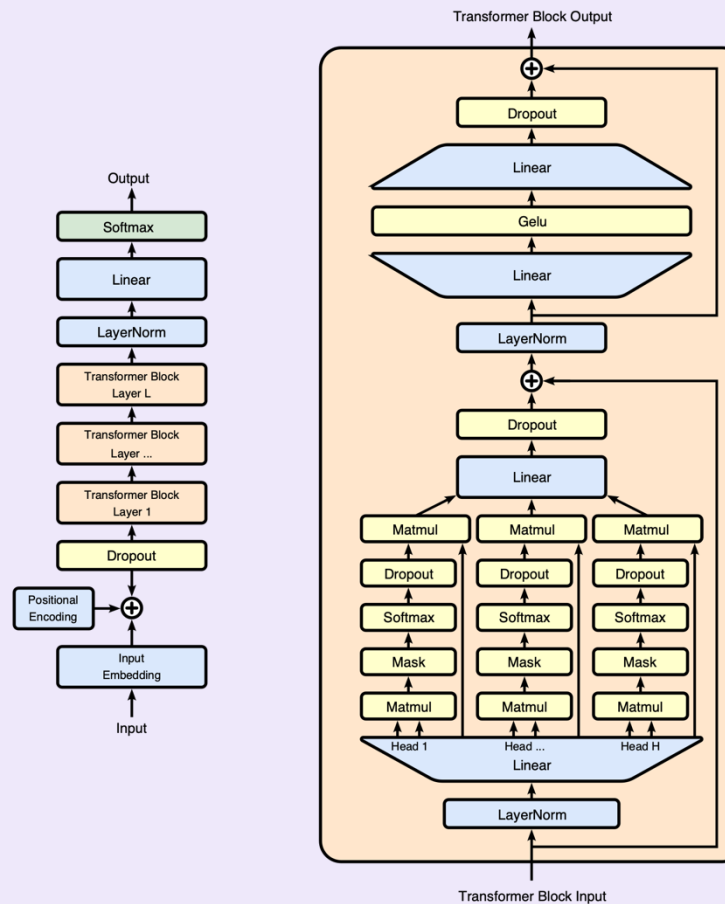
Prefix-causal



T5 Attention Strategies (Encoder/Decoder)

Attention Type	Visibility	Used In
Fully visible	All tokens attend to all tokens	Encoder representations
Causal	Token attends to past only	Autoregressive generation
Prefix-causal hybrid	Full source access + causal target decoding	Seq2Seq decoding in T5

GPT: Decoder-Only Autoregressive Generation



GPT: Decoder-Only Autoregressive Generation

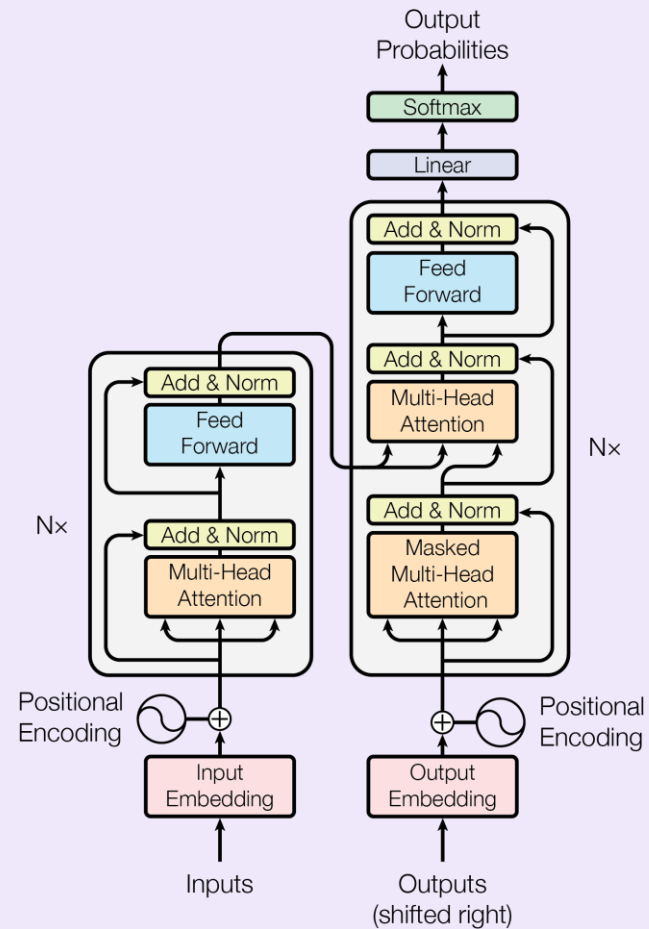
- GPT predicts next token from left context only (causal modeling).
- Uses masked self-attention in decoder stack for no-lookahead generation.
- Excellent for open-ended completion, instruction following, and synthesis.

GPT Evolution and Foundation Model Scale

Generation	Approx. Parameters	Key Theme
GPT-1	117M	Proof of transfer-learning viability
GPT-2	1.5B	Large-scale web pretraining for generation
GPT-3	175B	Few-shot in-context capabilities at scale

Model utility scales with data, architecture, and training compute quality.

Attention: Why It Was a Breakthrough



Attention: Why It Was a Breakthrough

- Attention solves fixed-vector bottlenecks in sequence transduction.
- At each step, model dynamically focuses on most relevant source positions.
- Entire mechanism is differentiable and learned end-to-end.

Attention Computation Pipeline

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V$$

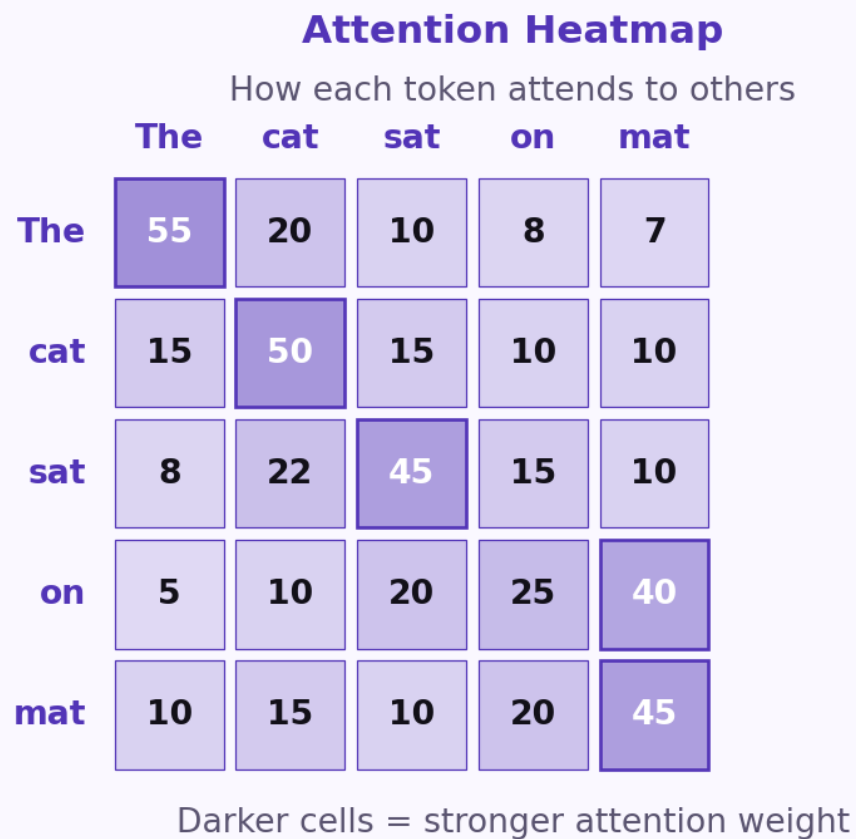
- Compute relevance scores between query state and source states.
- Apply softmax to obtain attention weights (probability distribution).
- Return weighted sum of value/source states as contextual output.

Bahdanau vs Luong Attention (Classic RNN Era)

Variant	Scoring Function	Typical Placement
Luong	Bilinear/dot-style	Attention after decoder state update
Bahdanau	MLP/additive score	Attention integrated before decoder update

Both approximate alignment; design choice affects speed and accuracy tradeoffs.

Self-Attention, Q/K/V, and Multi-Head



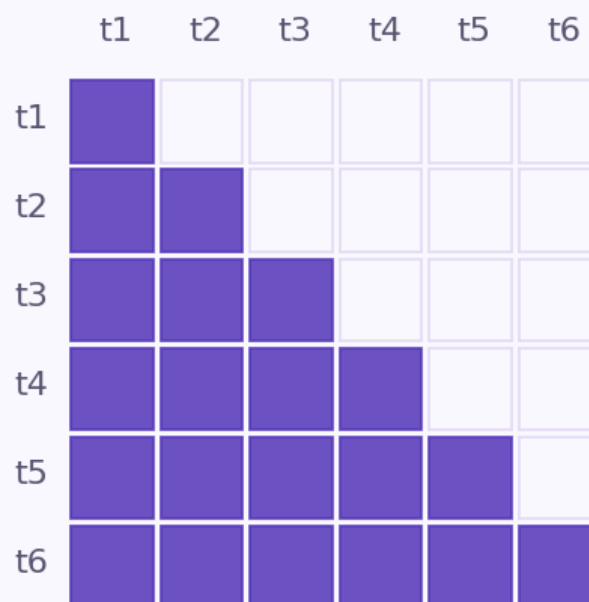
Self-Attention, Q/K/V, and Multi-Head

- Query asks for relevant context; Key indexes available context; Value carries content.
- Self-attention lets each token aggregate information from other tokens in sequence.
- Multi-head attention captures different linguistic relations in parallel.

Different heads can specialize in syntax, agreement, locality, or semantics.

Masked Self-Attention in Decoders

Causal mask — no lookahead



token attends to past only

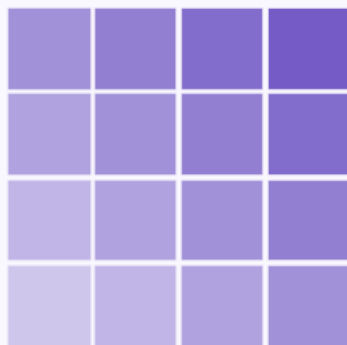
Masked Self-Attention in Decoders

- Decoder cannot access future tokens during generation.
- Causal mask enforces left-to-right consistency and prevents information leakage.
- Enables parallel training over full target sequence while preserving autoregressive objective.

Long-Sequence Challenge in Transformers

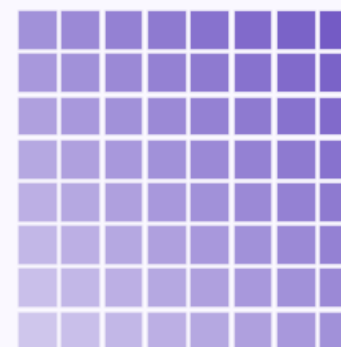
Attention map grows with sequence length

short L



$O(L^2)$

long L



$O(L^2)$ explodes

Long-Sequence Challenge in Transformers

Bottleneck	Complexity	Practical Impact
Attention map computation	$O(L^2)$	Time/memory explode as context grows
Activation storage (training)	$O(N * L * d_{\text{model}})$	High VRAM demand for deep long-context models

Long-context efficiency is now a central LLM engineering topic.

Scaling Laws and Compute-Optimal Training

- Performance improves with data, model size, and compute, but with diminishing returns.
- Compute-optimal training balances parameter count and token budget.
- Chinchilla-style insight: many large models are under-trained relative to their size.

Scaling Laws and Compute-Optimal Training

Principle	Implication
Fixed compute budget	Tune model size and data volume jointly
Data-quality filtering	Small fraction of tokens may dominate useful learning
Chinchilla ratio (rule of thumb)	Roughly ~20 training tokens per parameter

Generative AI Practical Takeaways

- Choose architecture by task type: encoder-only, encoder-decoder, or decoder-only.
- Prefer transfer learning and fine-tuning over training from scratch when possible.
- Use attention-aware designs and decoding strategy (greedy/beam) based on product constraints.
- Plan for memory/compute early: quantization, context limits, and deployment targets.
- For domain-heavy use cases, adapt with specialized corpora and evaluation protocols.

Strong GenAI systems come from balanced choices across data, architecture, compute, and evaluation.

3 Ideas to Keep

Leave with these

- 1 Choose architecture by task: encoder-only (BERT), encoder-decoder (T5), or decoder-only (GPT).
- 2 Prefer transfer learning and fine-tuning over training from scratch.
- 3 Attention, decoding strategy, and compute limits shape what you can ship.

Thank you

Labs: code/17- GenAI